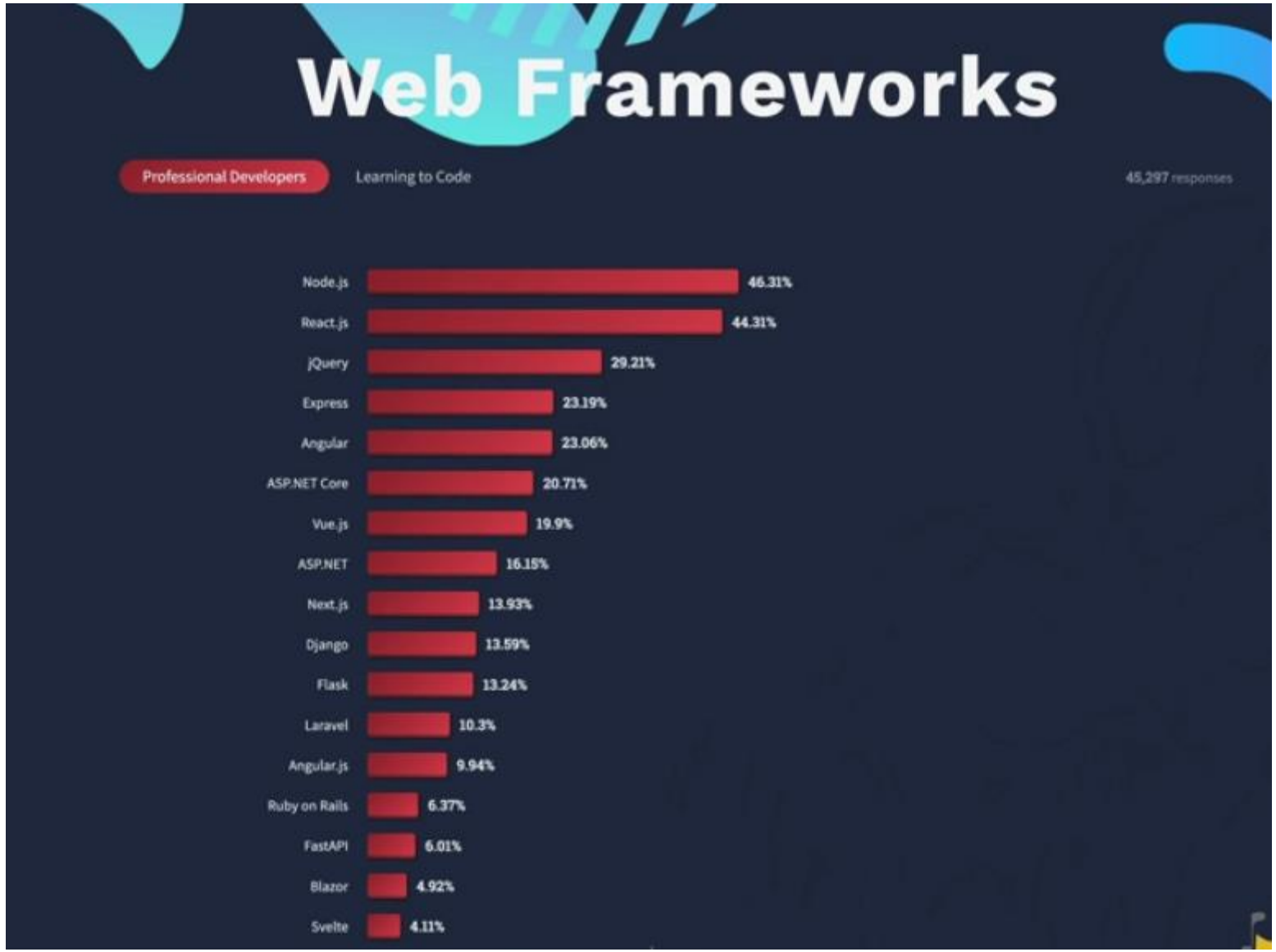
Express Framework

A framework on top of Node




Web Frameworks
Professional Developers
Learning to Code
45,297 responses
Node.js 46.31%
React.js 44.31%
jQuery 29.21%
Express 23.19%
Angular 23.06%
ASP.NET Core 20.71%
Vue.js 19.9%
ASP.NET 16.15%
Next.js 13.93%
Django 13.59%
Flask 13.24%
Laravel 10.3%
Angular.js 9.94%
Ruby on Rails 6.37%
FastAPI 6.01%
Blazor 4.92%
Svelte 4.11%

Back-end











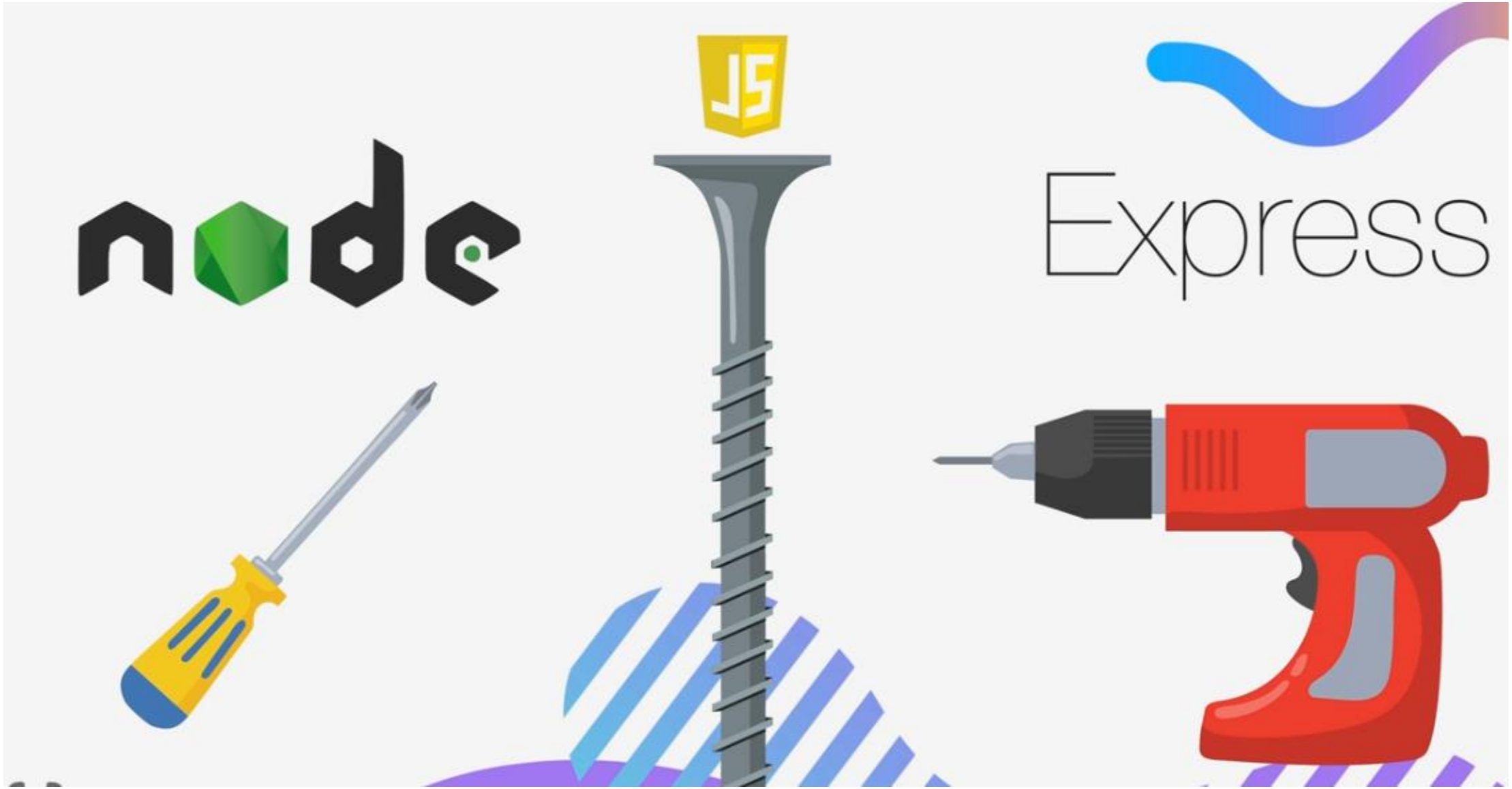


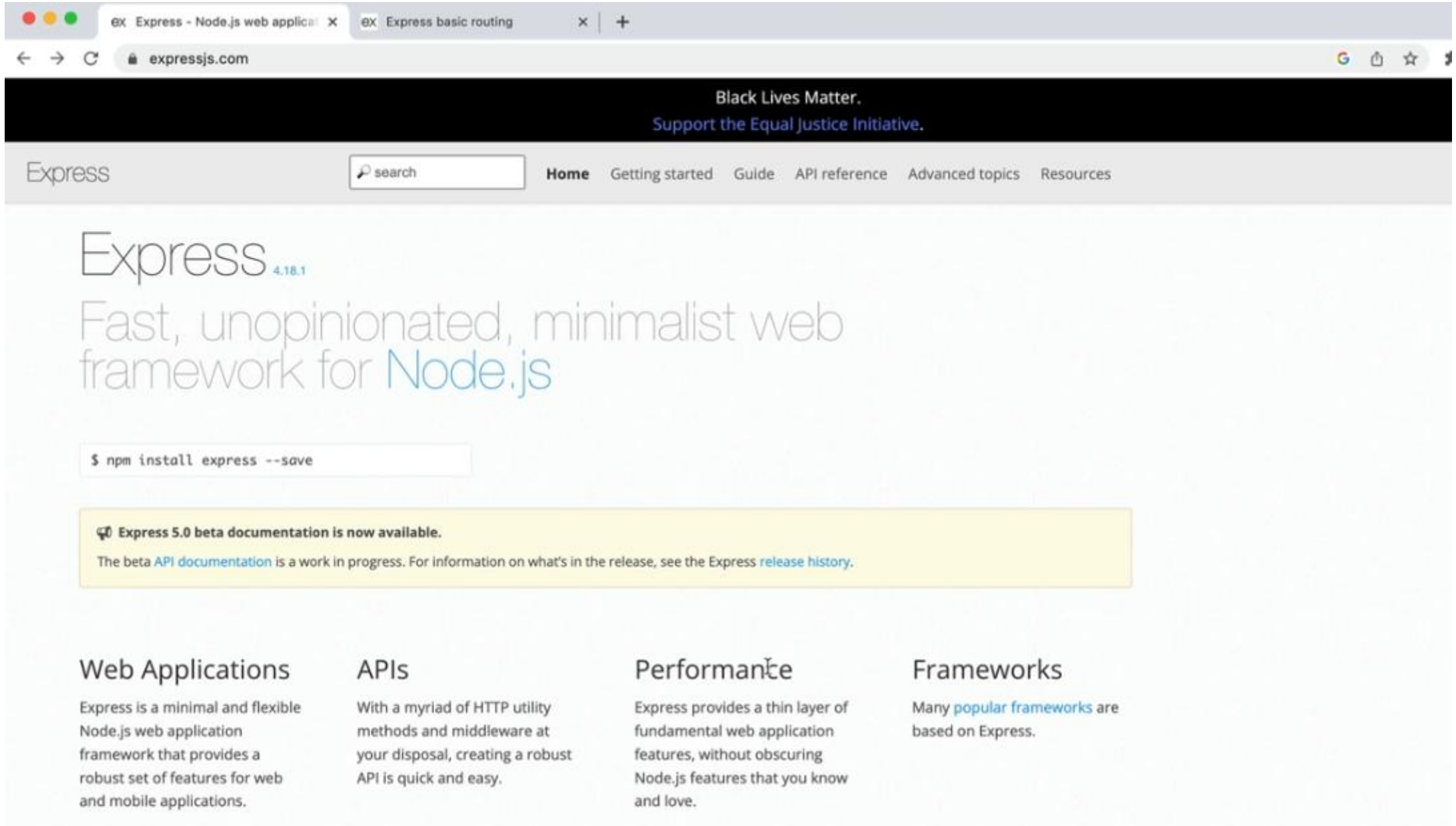




JS framework

Express


node
Express


Express - Node.js web applica
Express basic routing
expressjs.com
Black Lives Matter.
Support the Equal Justice Initiative.
Express
search
Home
Getting started
Guide
API reference
Advanced topics
Resources
Express 4.18.1
Fast, unopinionated, minimalist web framework for Node.js
$ npm install express --save
Express 5.0 beta documentation is now available.
The beta API documentation is a work in progress. For information on what's in the release, see the Express release history.
Web Applications
Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.
APIs
With a myriad of HTTP utility methods and middleware at your disposal, creating a robust API is quick and easy.
Performance
Express provides a thin layer of fundamental web application features, without obscuring Node.js features that you know and love.
Frameworks
Many popular frameworks are based on Express.

expressjs.com/en/starter/basic-routing.html

Black Lives Matter.
Support the Equal Justice Initiative.

Basic routing

Routing refers to determining how an application responds to a client request to a particular endpoint, which is a URI (or path) and a specific HTTP request method (GET, POST, and so on).

Each route can have one or more handler functions, which are executed when the route is matched.

Route definition takes the following structure:

```
app.METHOD(PATH, HANDLER)
```

Where:

- `app` is an instance of `express`.
- `METHOD` is an HTTP request method, in lowercase.
- `PATH` is a path on the server.
- `HANDLER` is the function executed when the route is matched.

> This tutorial assumes that an instance of `express` named `app` is created and the server is running. If you are not familiar with creating an app and starting it, see the Hello world example.

The following examples illustrate defining simple routes.

Respond with `Hello World!` on the homepage:

```
app.get('/', (req, res) => {
  res.send('Hello World!')
})
```

Node

index.js — Web Development Projects

index.js

Backend > 3.0 Intro to Express > index.js > ...

```
import http from "http";
import url from "url";

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true);

  if (parsedUrl.pathname === "/" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>Welcome to the homepage!</h1>");
  } else if (parsedUrl.pathname === "/about" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>About us</h1>");
  } else {
    res.writeHead(404, { "Content-Type": "text/html" });
    res.end("<h1>Page not found</h1>");
  }
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000/");
});

```

Node + Express

index.js — Web Development Projects

index.js

Backend > 3.0 Intro to Express > index.js > ...

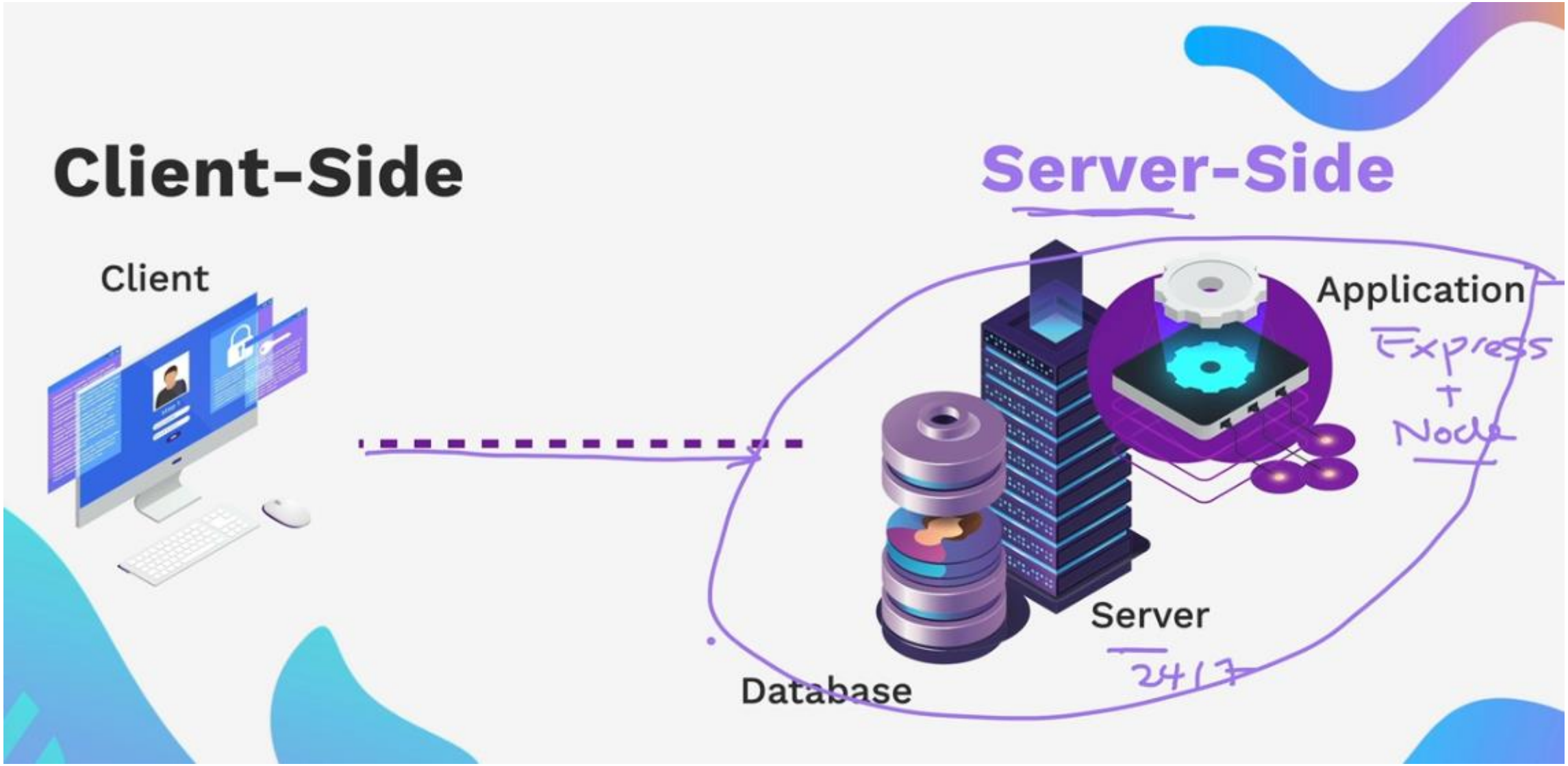
```
import express from "express";
const app = express();

app.get("/", (req, res) => {
  res.send("<h1>Welcome to the homepage!</h1>");
});

app.get("/about", (req, res) => {
  res.send("<h1>About us</h1>");
});

app.use((req, res) => {
  res.status(404).send("<h1>Page not found</h1>");
});

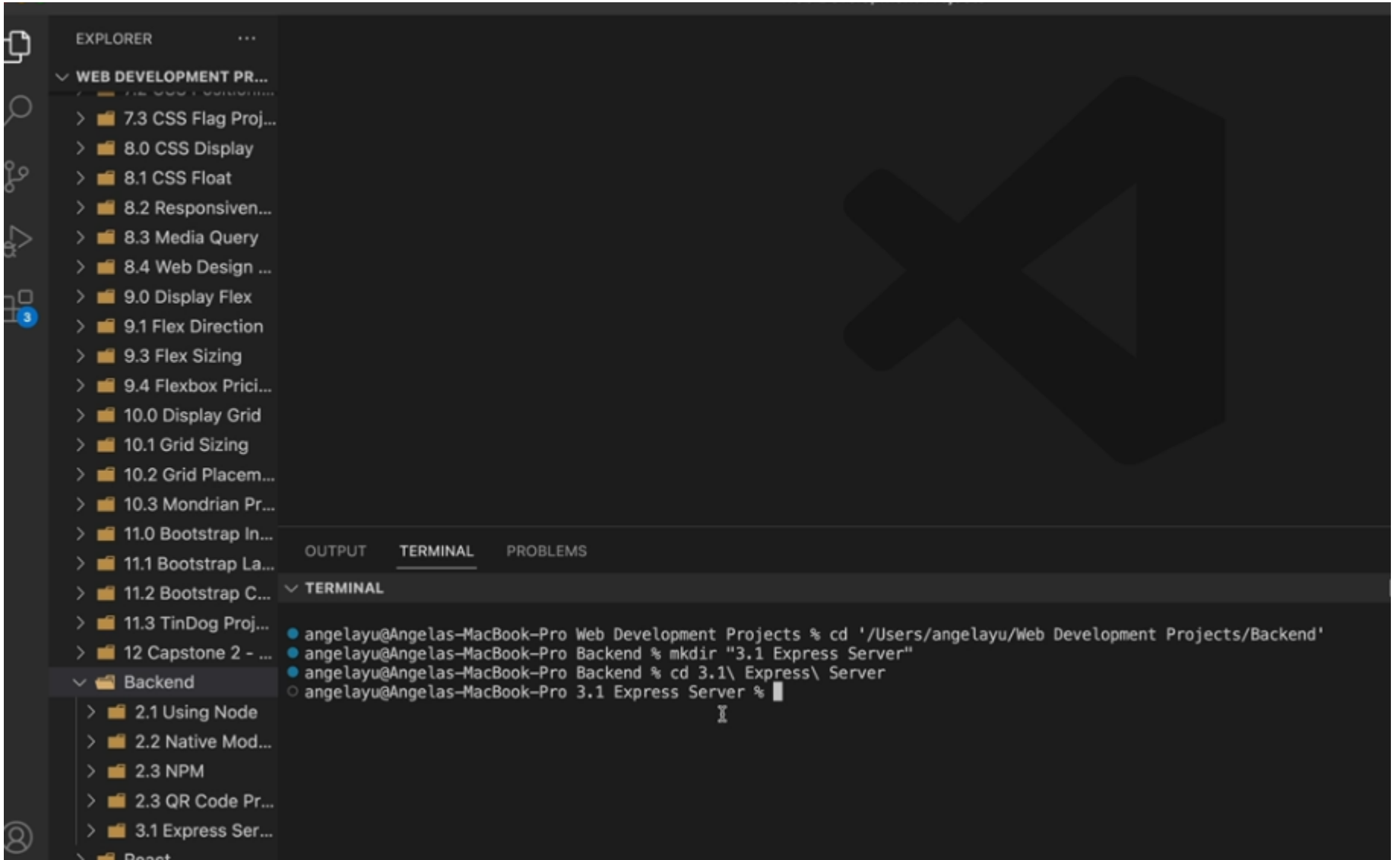
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000/");
});
```

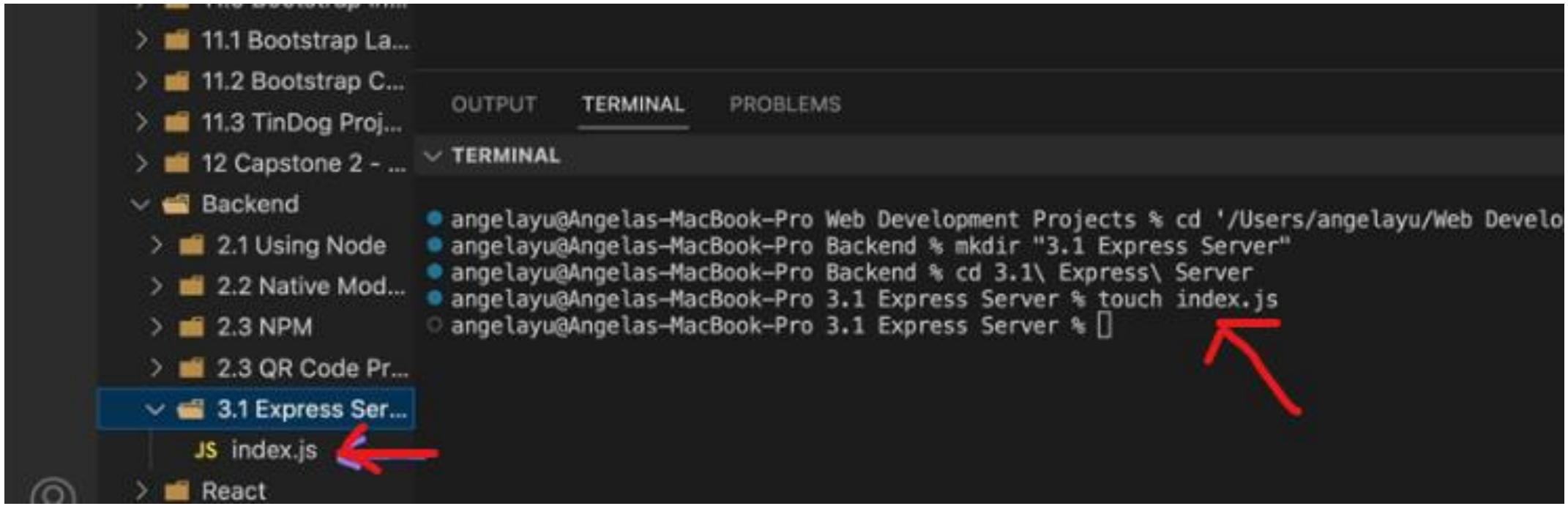

Client-Side
Server-Side
Client
Application
Express
+
Node
Server
24/7
Database

Creating an Express Server

1. Create directory.
2. Create index.js file.
3. Initialise NPM.
4. Install the Express package.
5. Write Server application in index.js.
6. Start server.

Create directory




11.1 Bootstrap La...
11.2 Bootstrap C...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.3 QR Code Pr...
3.1 Express Ser...
index.js
React
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro Web Development Projects % cd '/Users/angelayu/Web Develo
angelayu@Angelas-MacBook-Pro Backend % mkdir "3.1 Express Server"
angelayu@Angelas-MacBook-Pro Backend % cd 3.1\ Express\ Server
angelayu@Angelas-MacBook-Pro 3.1 Express Server % touch index.js
angelayu@Angelas-MacBook-Pro 3.1 Express Server %

Black Lives Matter.
Support the Equal Justice Initiative.

Installing

Assuming you've already installed Node.js, create a directory to hold your application, and make that your working directory.

```
$ mkdir myapp
$ cd myapp
```

Use the `npm init` command to create a `package.json` file for your application. For more information on how `package.json` works, see Specifics of npm's package.json handling.

```
$ npm init
```

This command prompts you for a number of things, such as the name and version of your application. For now, you can simply hit RETURN to accept the defaults for most of them, with the following exception:

```
entry point: (index.js)
```

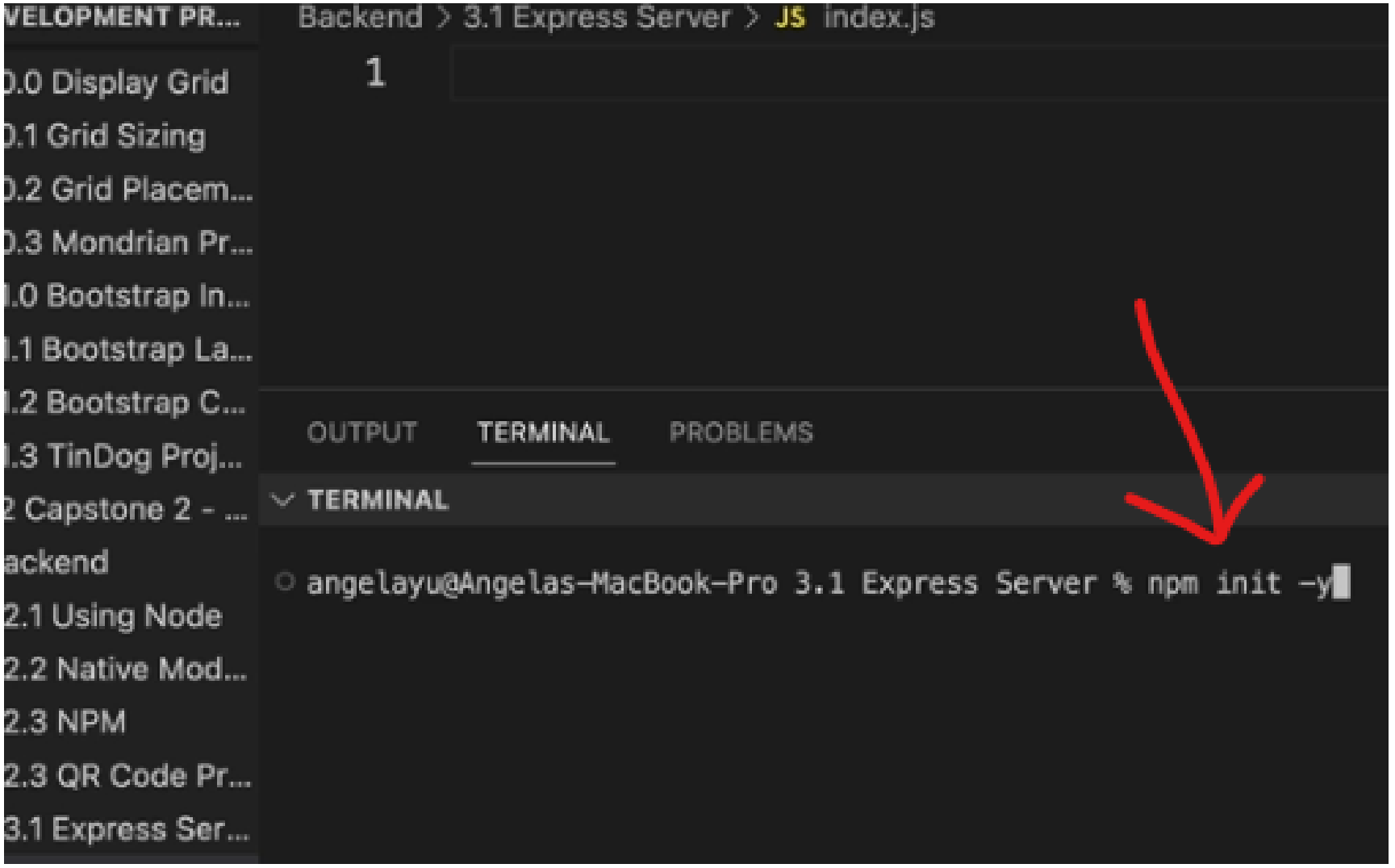
Enter `app.js`, or whatever you want the name of the main file to be. If you want it to be `index.js`, hit RETURN to accept the suggested default file name.

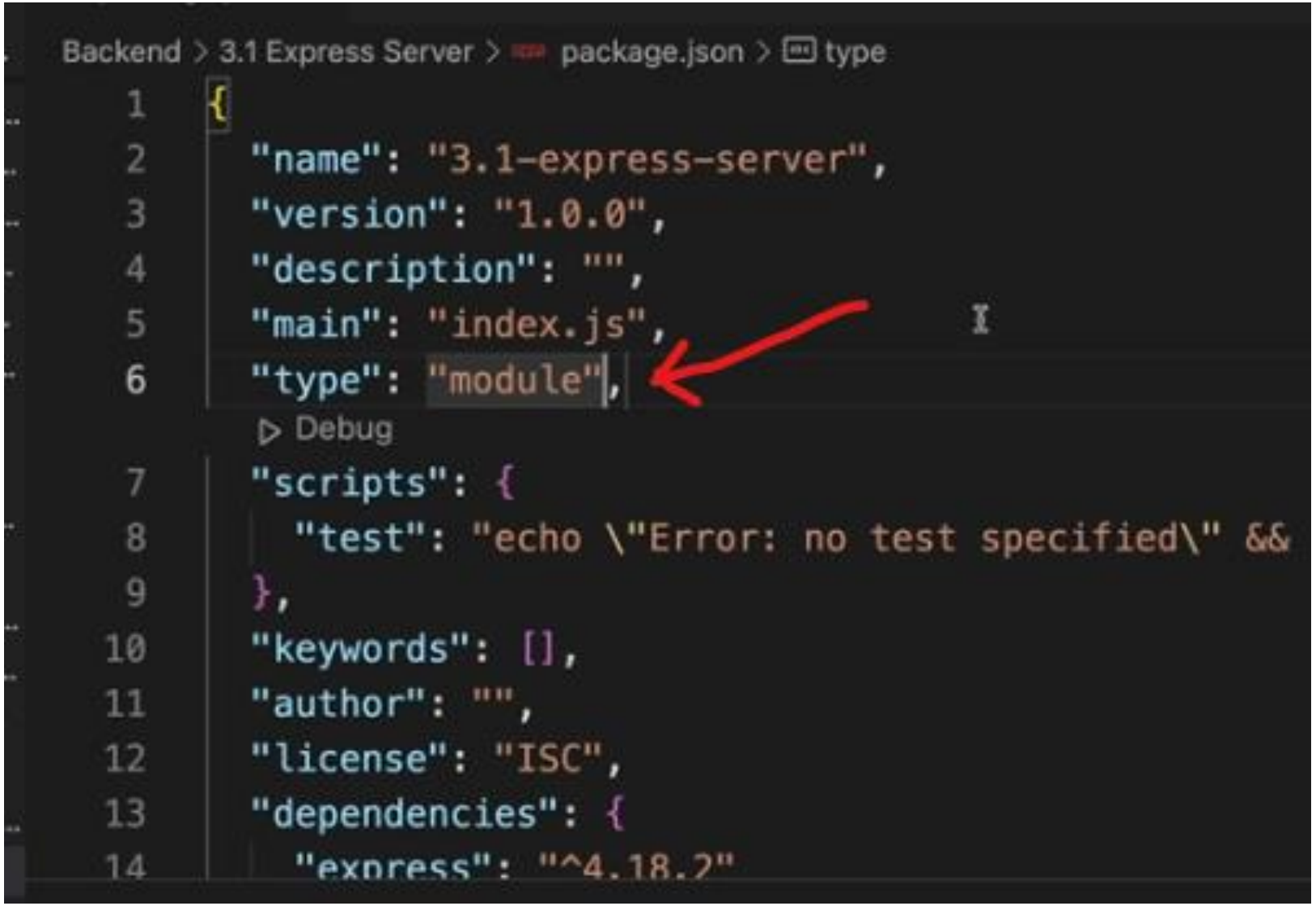
Now install Express in the `myapp` directory and save it in the dependencies list. For example:

```
$ npm install express
```

To install Express temporarily and not add it to the dependencies list:

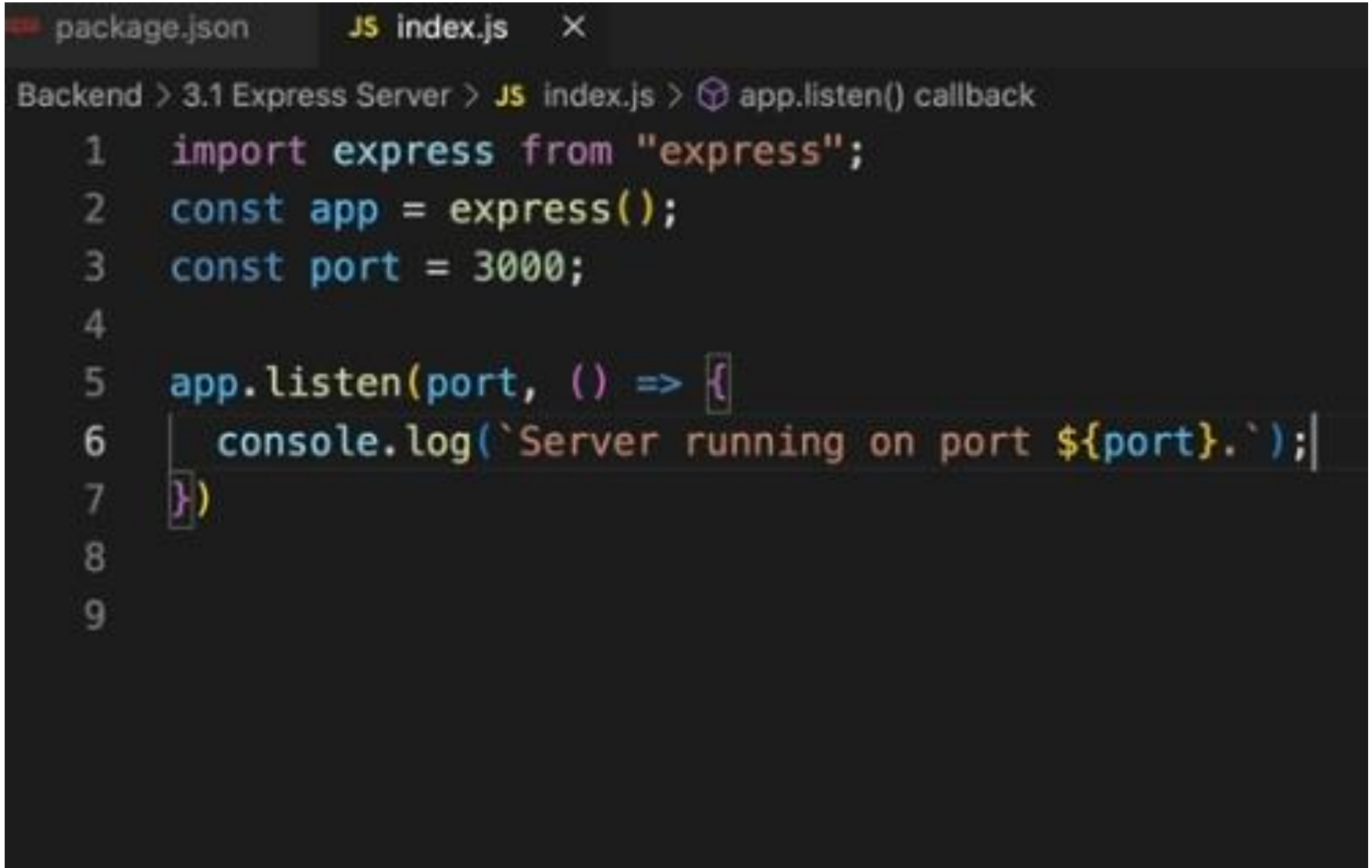
```
$ npm install express --no-save
```


VELOPMENT PR...
0.0 Display Grid
0.1 Grid Sizing
0.2 Grid Placem...
0.3 Mondrian Pr...
1.0 Bootstrap In...
1.1 Bootstrap La...
1.2 Bootstrap C...
1.3 TinDog Proj...
2 Capstone 2 - ...
ackend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.3 QR Code Pr...
3.1 Express Ser...
Backend > 3.1 Express Server > JS index.js
1
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro 3.1 Express Server % npm init -y


Backend > 3.1 Express Server > package.json > type
{
"name": "3.1-express-server",
"version": "1.0.0",
"description": "",
"main": "index.js",
"type": "module",
Debug
"scripts": {
"test": "echo \"Error: no test specified\" &&
},
"keywords": [],
"author": "",
"license": "ISC",
"dependencies": {
"express": "^4.18.2"

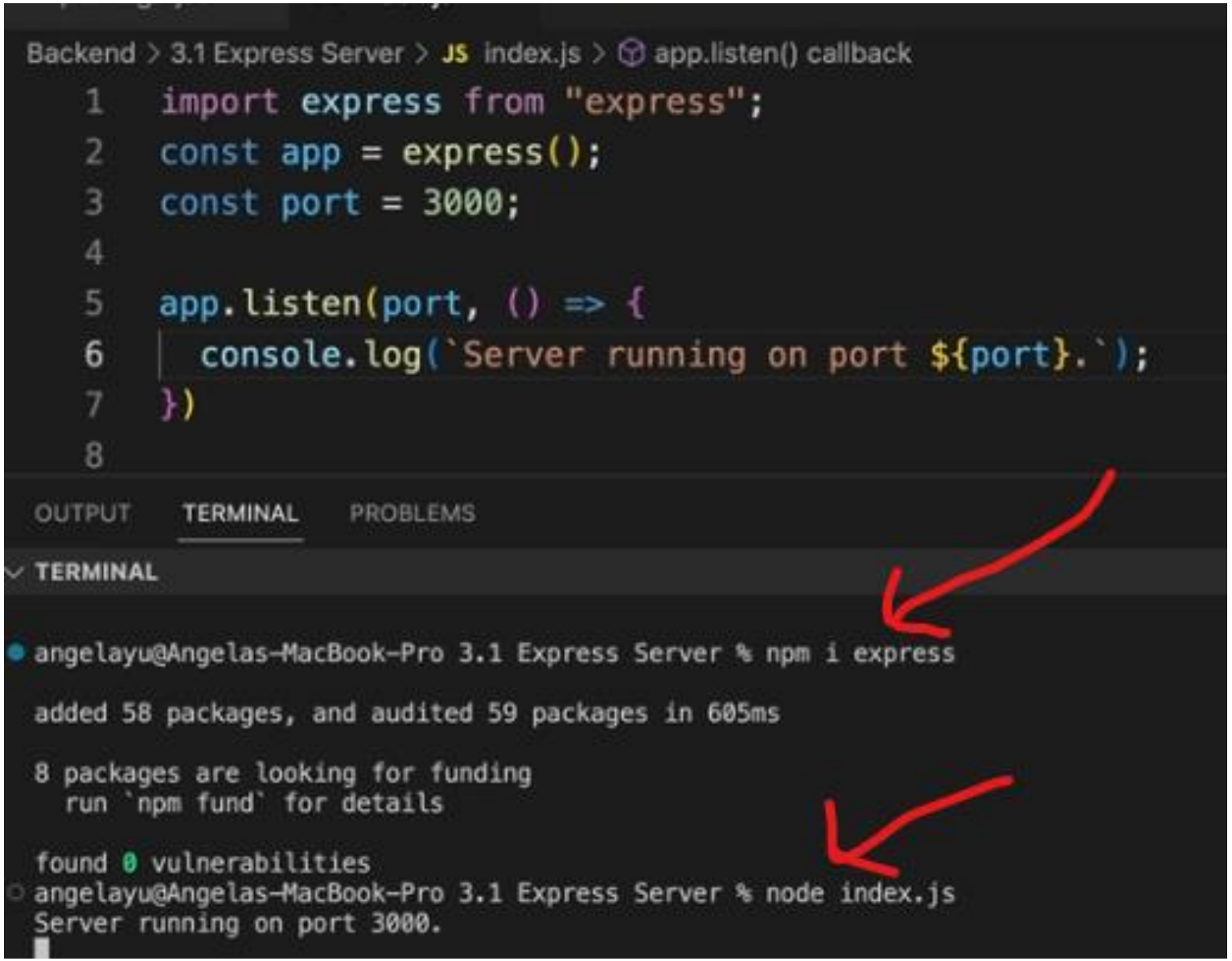

JS index.js
import express from "express";
const app = express();
1 Port
app.listen(3000, () => {
console.log("Server running on port 3000.");
});
2 callback

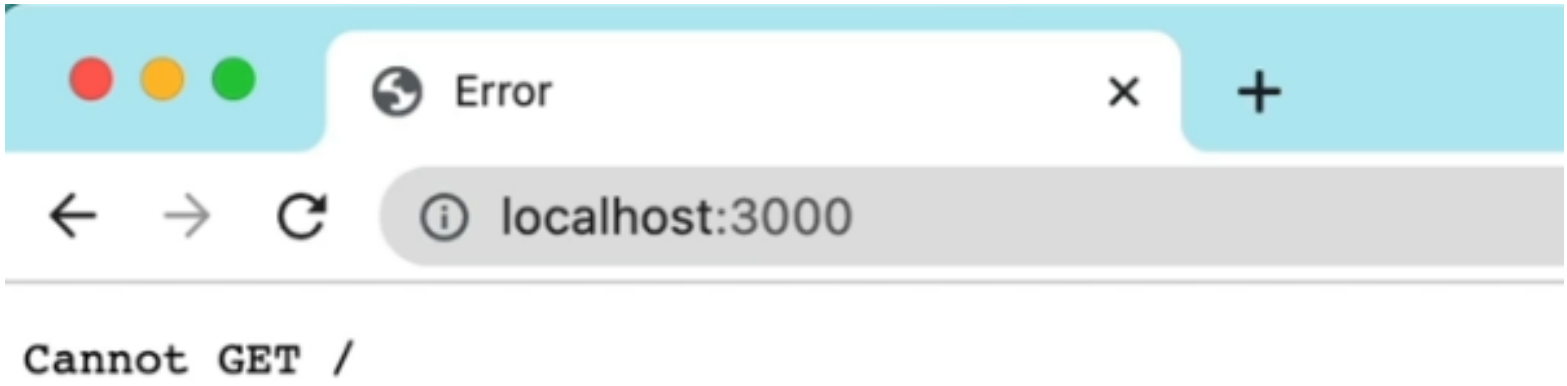
Express


package.json
JS index.js
Backend > 3.1 Express Server > JS index.js > app.listen() callback
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.listen(port, () => {
6 console.log(`Server running on port ${port}.`);
7 })
8
9

Run a Node.js program

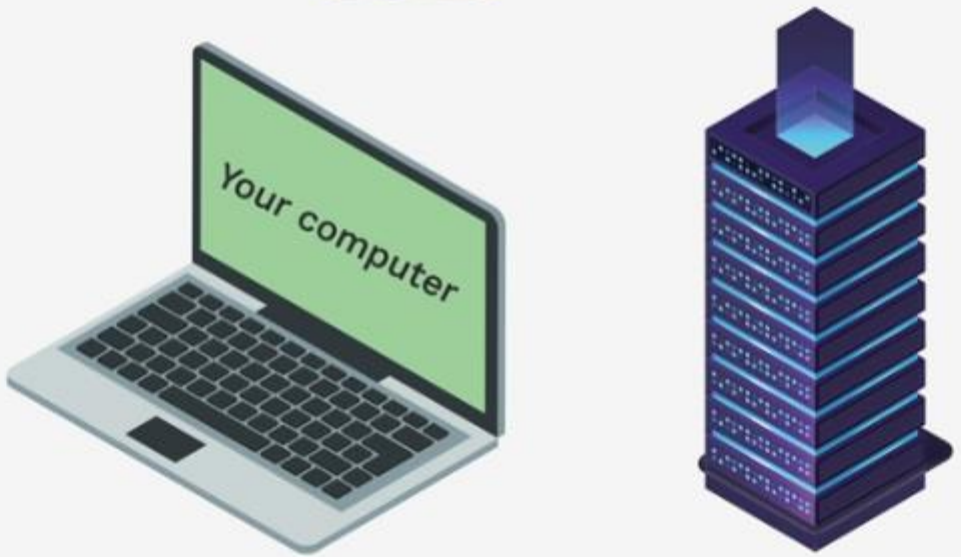
```
node your_app.js
```


Backend > 3.1 Express Server > JS index.js > app.listen() callback
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.listen(port, () => {
6 console.log(`Server running on port ${port}.`);
7 })
8
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro 3.1 Express Server % npm i express
added 58 packages, and audited 59 packages in 605ms
8 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
angelayu@Angelas-MacBook-Pro 3.1 Express Server % node index.js
Server running on port 3000.


Error
localhost:3000
Cannot GET /

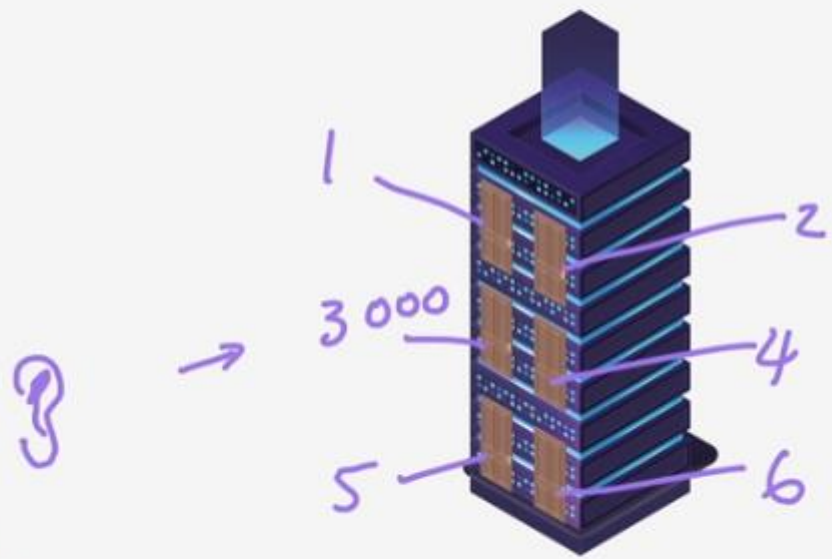
What is Localhost?

https://localhost:3000



Server

What is a Port?



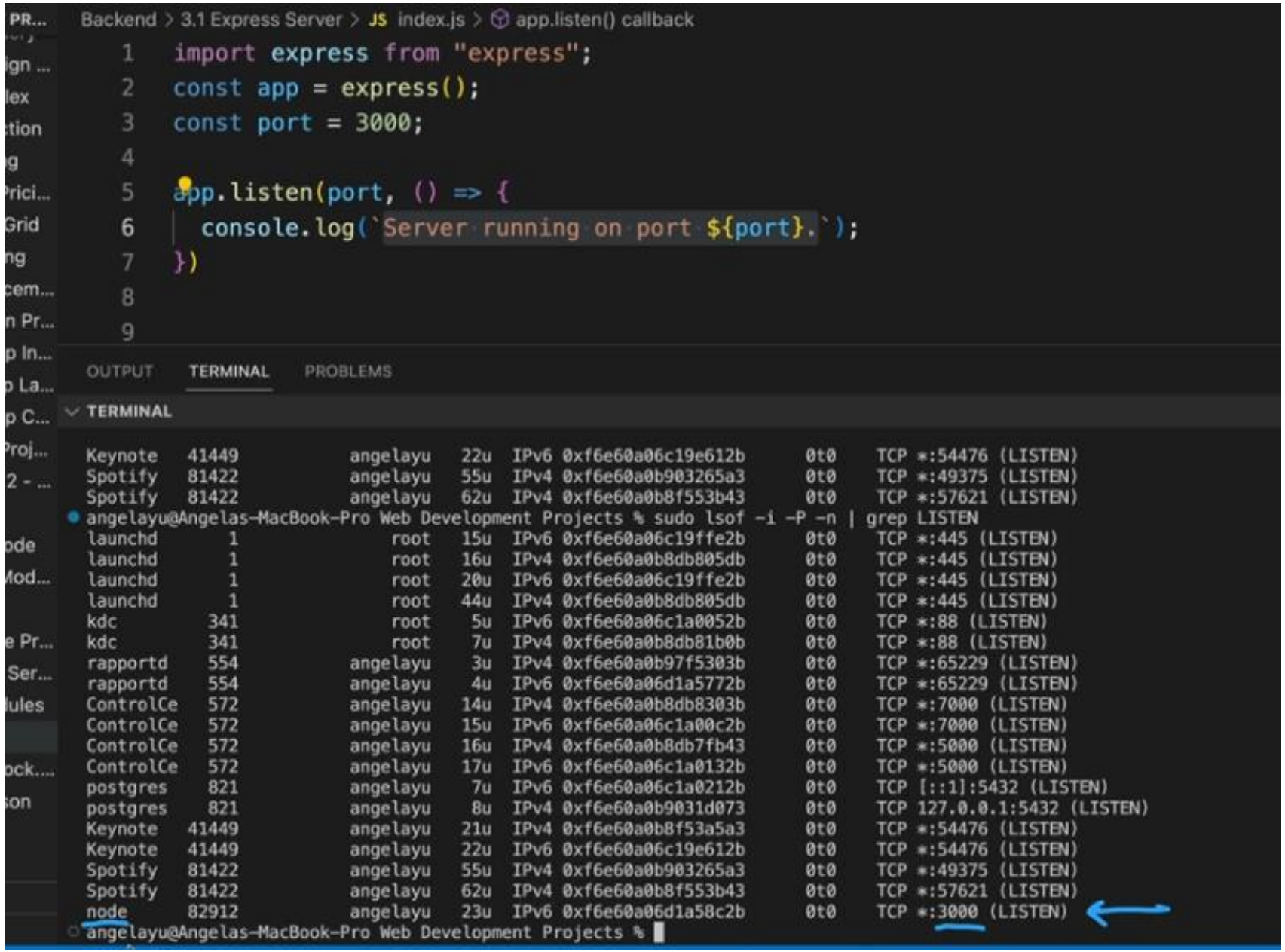
Server

Windows

```
netstat -ano | findstr "LISTENING"
```

Mac/Linux

```
sudo lsof -i -P -n | grep LISTEN
```

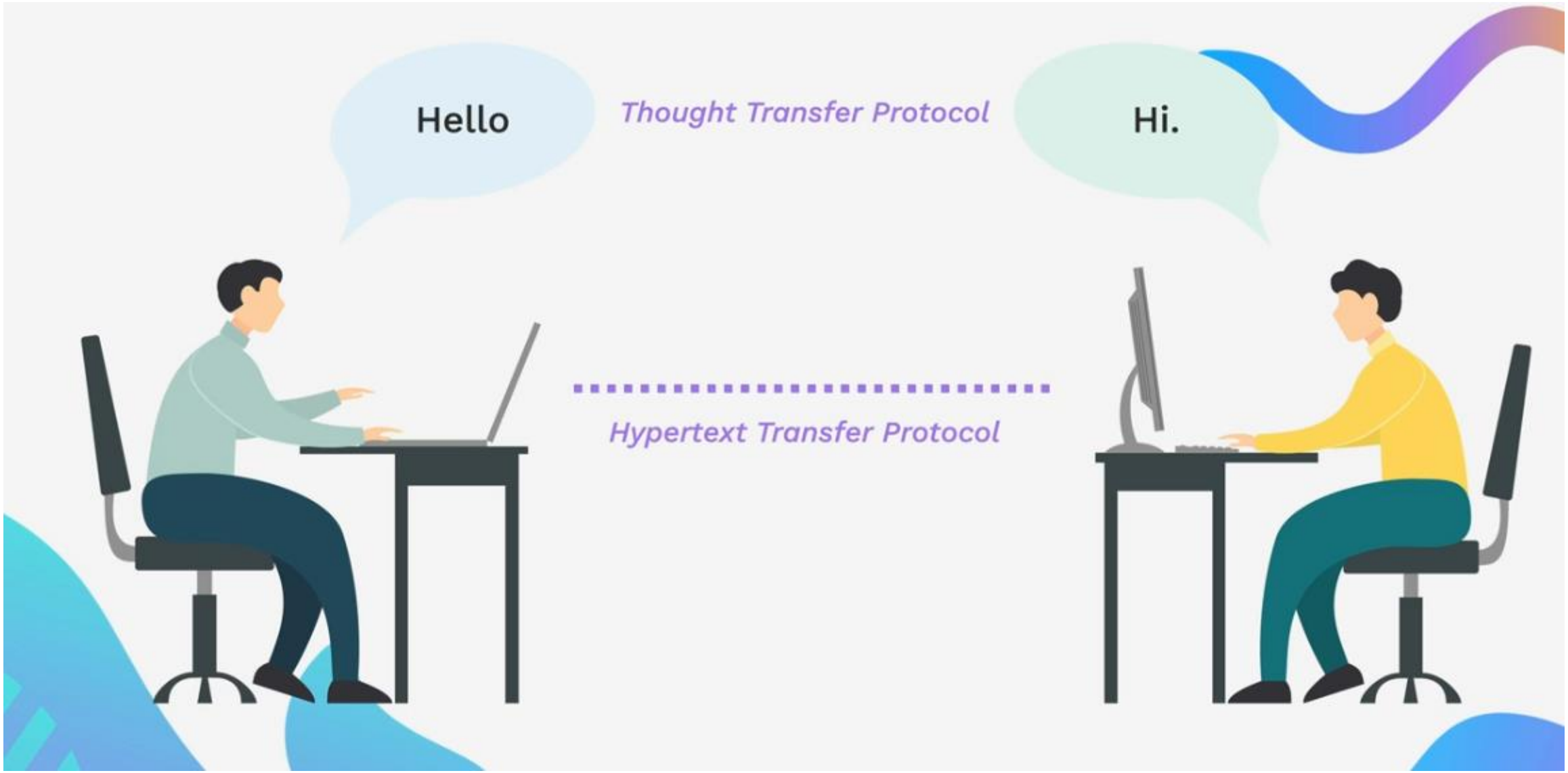

Backend > 3.1 Express Server > JS index.js > app.listen() callback
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.listen(port, () => {
6 console.log(`Server running on port ${port}.`);
7 })
8
9
OUTPUT TERMINAL PROBLEMS
TERMINAL
Keynote 41449 angelayu 22u IPv6 0xf6e60a06c19e612b 0t0 TCP *:54476 (LISTEN)
Spotify 81422 angelayu 55u IPv4 0xf6e60a0b903265a3 0t0 TCP *:49375 (LISTEN)
Spotify 81422 angelayu 62u IPv4 0xf6e60a0b8f553b43 0t0 TCP *:57621 (LISTEN)
angelayu@Angelas-MacBook-Pro Web Development Projects % sudo lsof -i -P -n | grep LISTEN
launchd 1 root 15u IPv6 0xf6e60a06c19ffe2b 0t0 TCP *:445 (LISTEN)
launchd 1 root 16u IPv4 0xf6e60a0b8db805db 0t0 TCP *:445 (LISTEN)
launchd 1 root 20u IPv6 0xf6e60a06c19ffe2b 0t0 TCP *:445 (LISTEN)
launchd 1 root 44u IPv4 0xf6e60a0b8db805db 0t0 TCP *:445 (LISTEN)
kdc 341 root 5u IPv6 0xf6e60a06c1a0052b 0t0 TCP *:88 (LISTEN)
kdc 341 root 7u IPv4 0xf6e60a0b8db81b0b 0t0 TCP *:88 (LISTEN)
rapportd 554 angelayu 3u IPv4 0xf6e60a0b97f5303b 0t0 TCP *:65229 (LISTEN)
rapportd 554 angelayu 4u IPv6 0xf6e60a06d1a5772b 0t0 TCP *:65229 (LISTEN)
ControlCe 572 angelayu 14u IPv4 0xf6e60a0b8db8303b 0t0 TCP *:7000 (LISTEN)
ControlCe 572 angelayu 15u IPv6 0xf6e60a06c1a00c2b 0t0 TCP *:7000 (LISTEN)
ControlCe 572 angelayu 16u IPv4 0xf6e60a0b8db7fb43 0t0 TCP *:5000 (LISTEN)
ControlCe 572 angelayu 17u IPv6 0xf6e60a06c1a0132b 0t0 TCP *:5000 (LISTEN)
postgres 821 angelayu 7u IPv6 0xf6e60a06c1a0212b 0t0 TCP [::1]:5432 (LISTEN)
postgres 821 angelayu 8u IPv4 0xf6e60a0b9031d073 0t0 TCP 127.0.0.1:5432 (LISTEN)
Keynote 41449 angelayu 21u IPv4 0xf6e60a0b8f53a5a3 0t0 TCP *:54476 (LISTEN)
Keynote 41449 angelayu 22u IPv6 0xf6e60a06c19e612b 0t0 TCP *:54476 (LISTEN)
Spotify 81422 angelayu 55u IPv4 0xf6e60a0b903265a3 0t0 TCP *:49375 (LISTEN)
Spotify 81422 angelayu 62u IPv4 0xf6e60a0b8f553b43 0t0 TCP *:57621 (LISTEN)
node 82912 angelayu 23u IPv6 0xf6e60a06d1a58c2b 0t0 TCP *:3000 (LISTEN)
angelayu@Angelas-MacBook-Pro Web Development Projects %

HTTP Requests

Learn the language to make requests to your server



HyperText Transfer Protocol


Hello
Thought Transfer Protocol
Hi.
Hypertext Transfer Protocol


お腹空いた


200
404
Get
Post

Request Vocab

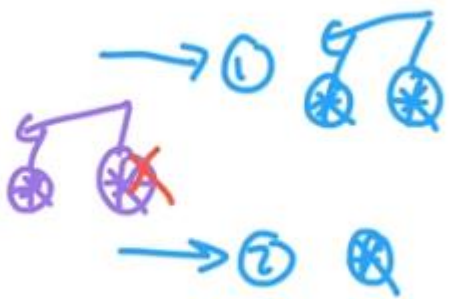
GET → Request resource

POST → Sending resource

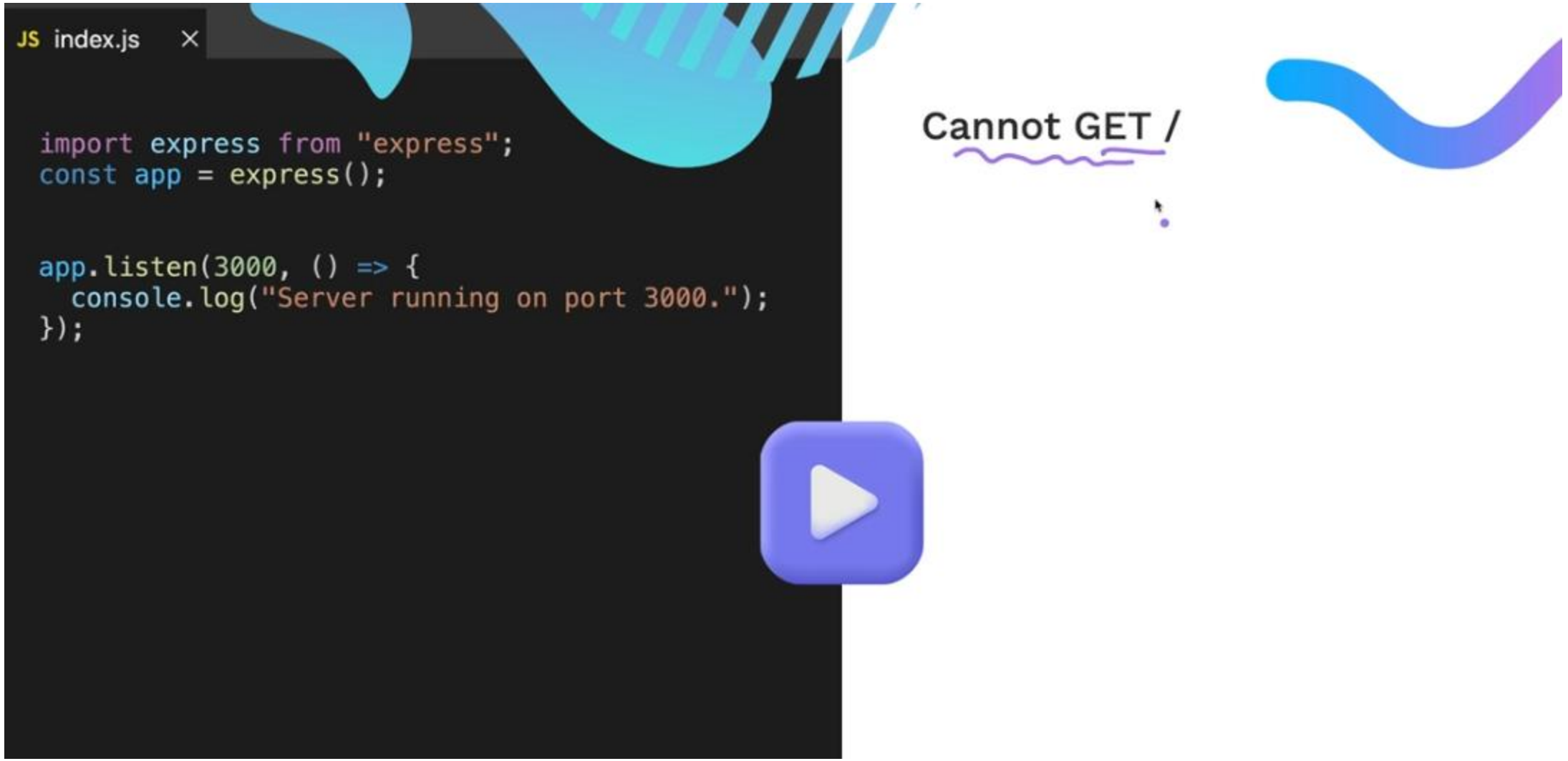
UPDATE

PUT → Replace resource

PATCH → Patch up a resource



DELETE → Delete resource

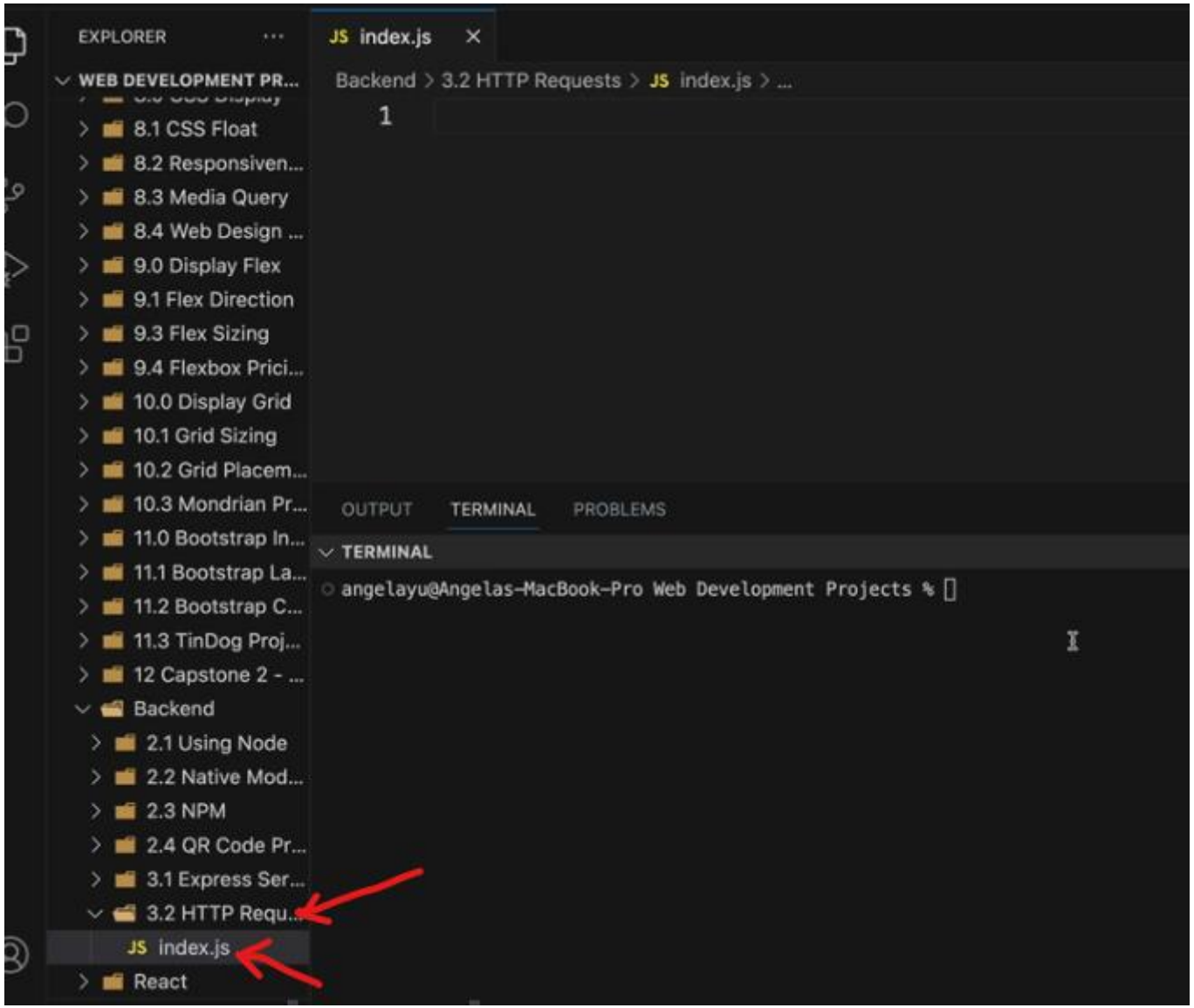

JS index.js
import express from "express";
const app = express();
app.listen(3000, () => {
console.log("Server running on port 3000.");
});
Cannot GET /

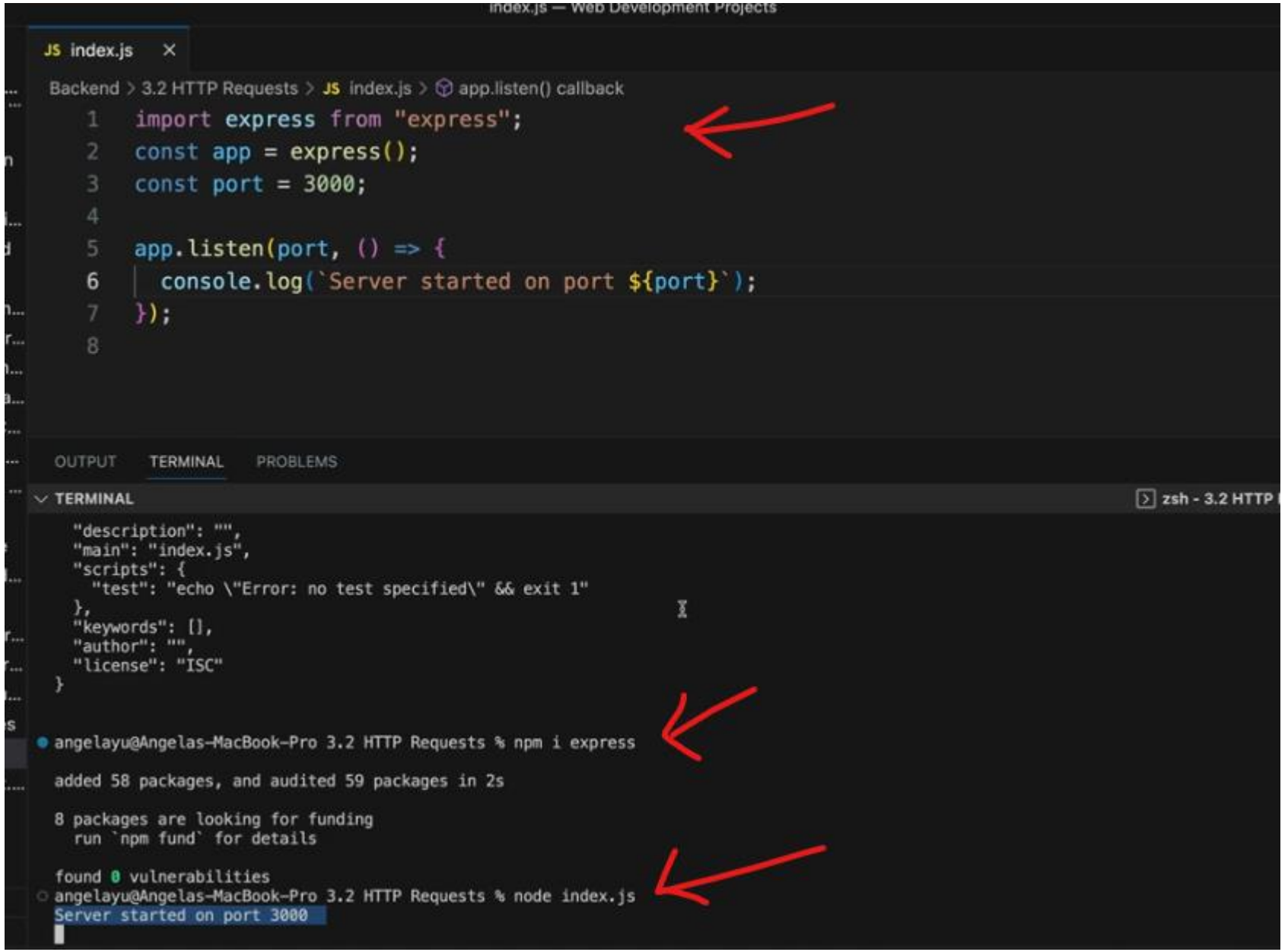

import express from "express";
const app = express();
app.get("/", (req, res) => {
res.send("Hello, World!");
});
app.listen(3000, () => {
console.log("Server running on port 3000.");
});

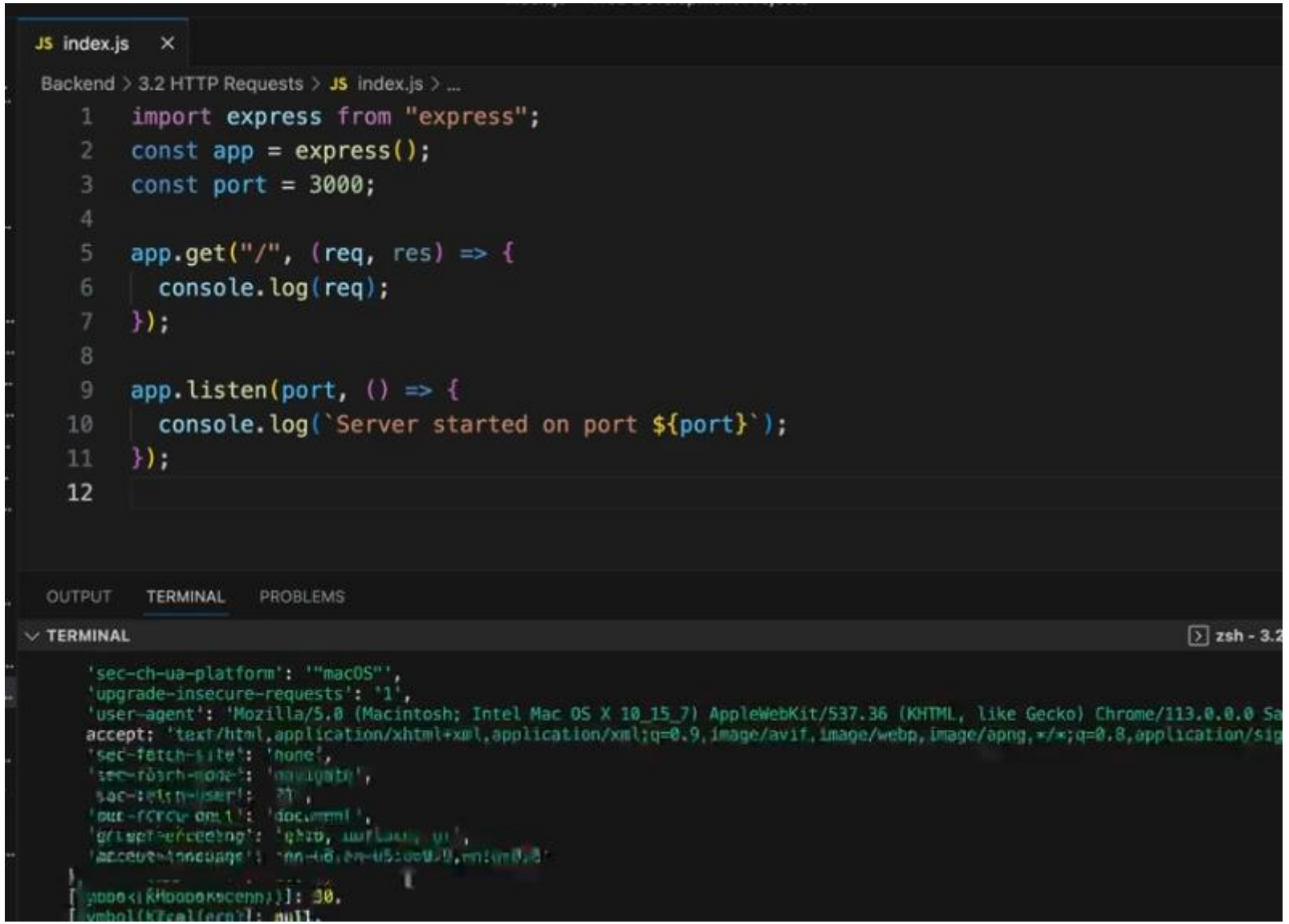
Cannot GET /

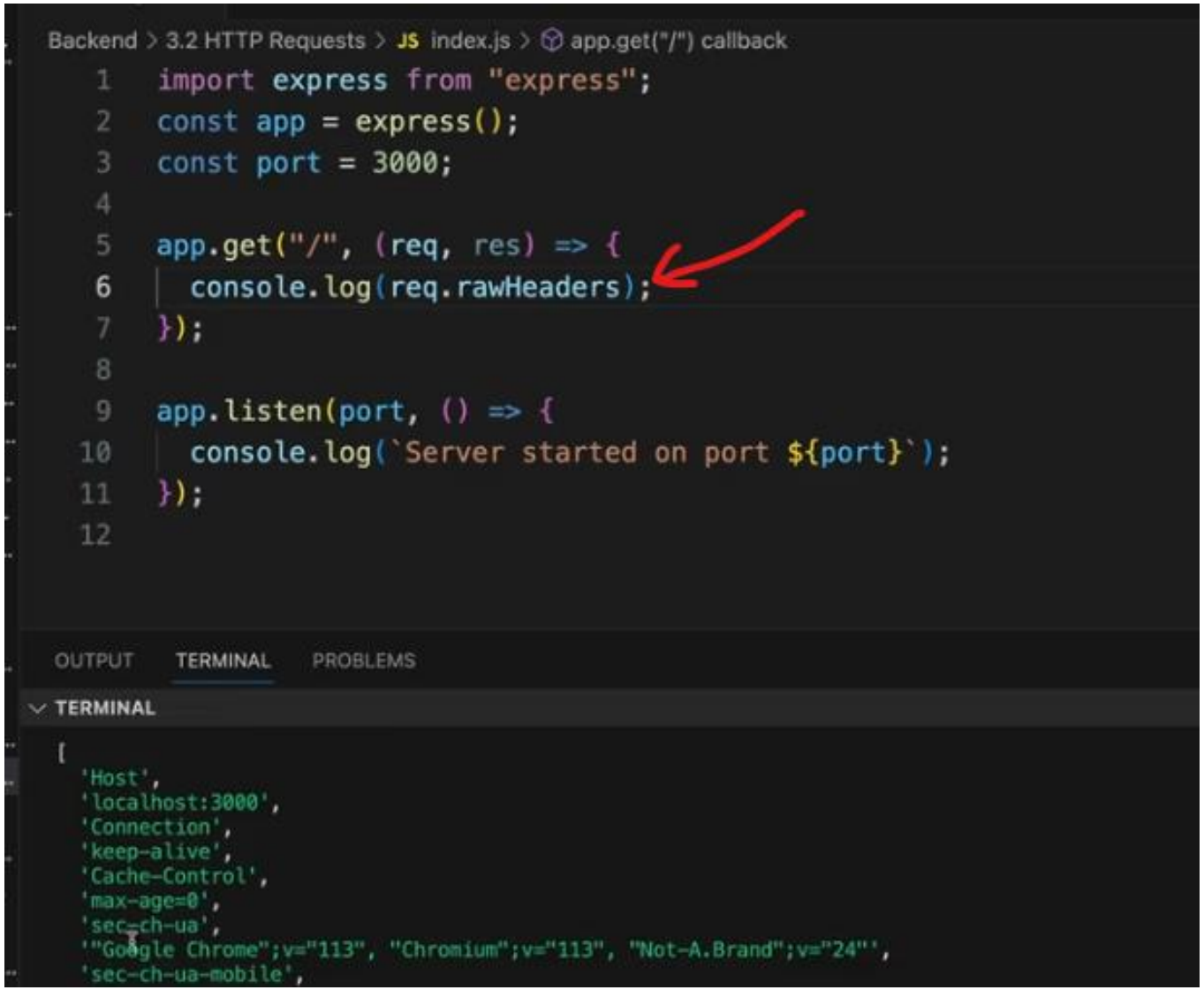
Exercise

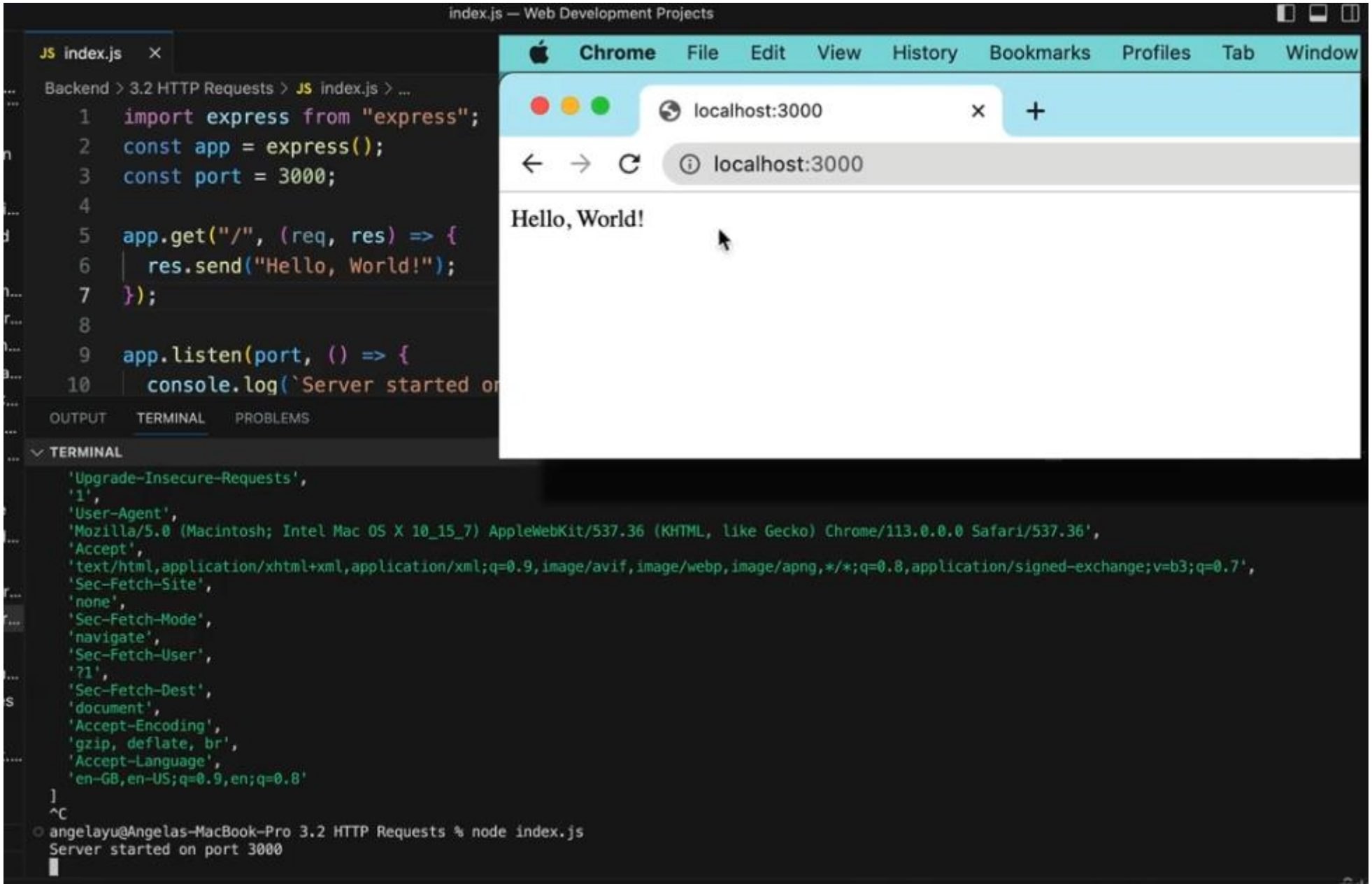
1. Create directory called 3.2 HTTP Requests.
2. Create index.js file.
3. Initialise NPM.
4. Install the Express package.
5. Write Server application in index.js.
6. Start server.

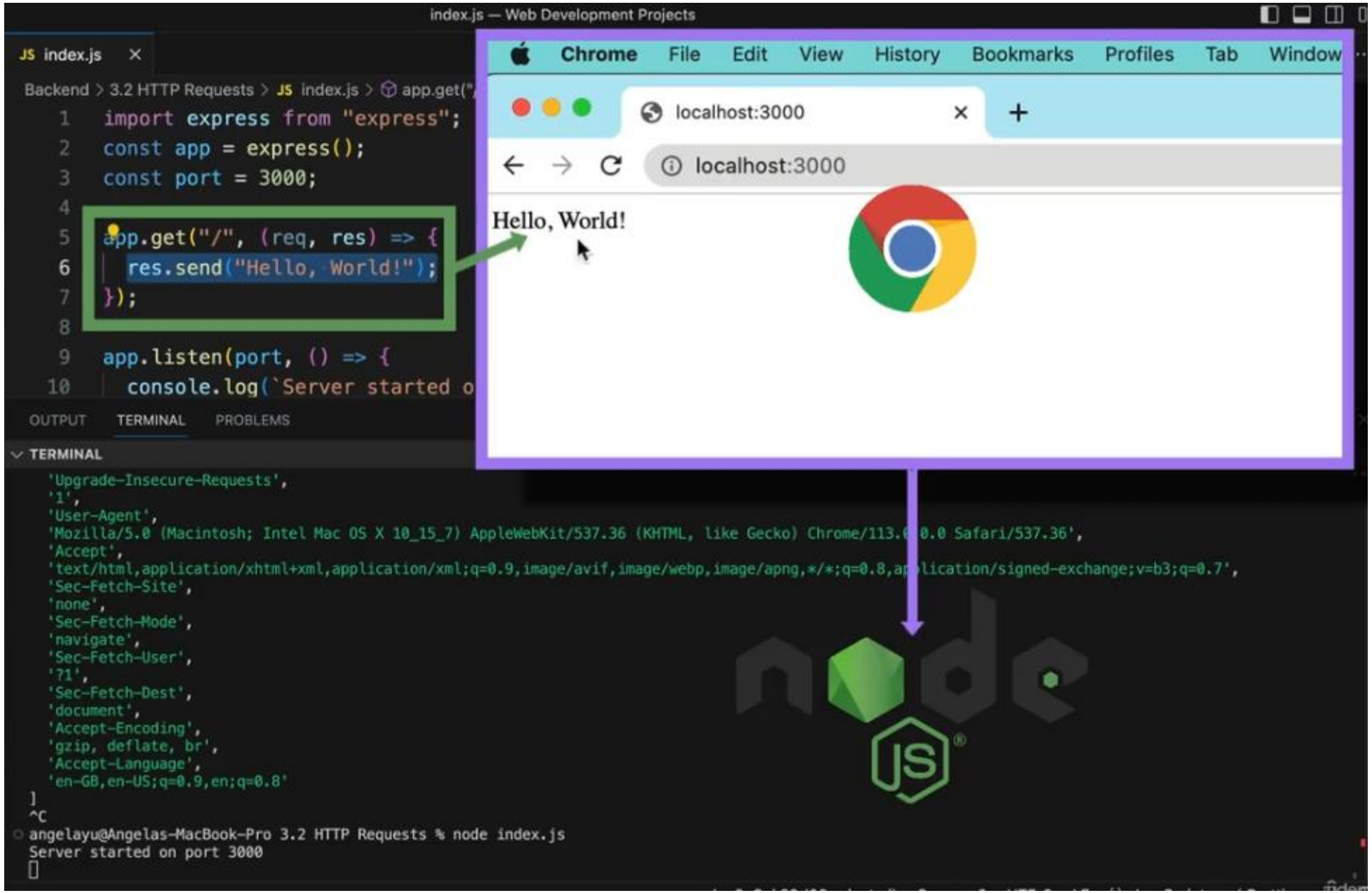

EXPLORER
WEB DEVELOPMENT PR...
8.1 CSS Float
8.2 Responsiven...
8.3 Media Query
8.4 Web Design ...
9.0 Display Flex
9.1 Flex Direction
9.3 Flex Sizing
9.4 Flexbox Prici...
10.0 Display Grid
10.1 Grid Sizing
10.2 Grid Placem...
10.3 Mondrian Pr...
11.0 Bootstrap In...
11.1 Bootstrap La...
11.2 Bootstrap C...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
index.js
React
index.js
Backend > 3.2 HTTP Requests > index.js > ...
1
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro Web Development Projects %


index.js — Web Development Projects
index.js
Backend > 3.2 HTTP Requests > index.js > app.listen() callback
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.listen(port, () => {
6 console.log(`Server started on port ${port}`);
7 });
8
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
zsh - 3.2 HTTP
"description": "",
"main": "index.js",
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC"
}
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % npm i express
added 58 packages, and audited 59 packages in 2s
8 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % node index.js
Server started on port 3000


index.js
Backend > 3.2 HTTP Requests > index.js > ...
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.get("/", (req, res) => {
6 console.log(req);
7 });
8
9 app.listen(port, () => {
10 console.log(`Server started on port ${port}`);
11 });
12
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
'sec-ch-ua-platform': '"macOS"',
'upgrade-insecure-requests': '1',
'user-agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Sa
accept: 'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/sig
'sec-fetch-site': 'none',
'sec-fetch-mode': 'navigate',
'sec-fetch-user': '?1',
'sec-ch-ua': 'doc.mml',
'accept-encoding': 'gzip, deflate, br',
'accept-language': 'en-US,en;q=0.9,en-US;q=0.8,en;q=0.8'


Backend > 3.2 HTTP Requests > index.js > app.get("/") callback
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.get("/", (req, res) => {
6 console.log(req.rawHeaders);
7 });
8
9 app.listen(port, () => {
10 console.log(`Server started on port ${port}`);
11 });
12
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
[
'Host',
'localhost:3000',
'Connection',
'keep-alive',
'Cache-Control',
'max-age=0',
'sec-ch-ua',
'"Google Chrome";v="113", "Chromium";v="113", "Not-A.Brand";v="24"',
'sec-ch-ua-mobile',


index.js — Web Development Projects
index.js
Backend > 3.2 HTTP Requests > index.js > ...
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.get("/", (req, res) => {
6 res.send("Hello, World!");
7 });
8
9 app.listen(port, () => {
10 console.log(`Server started or
Chrome File Edit View History Bookmarks Profiles Tab Window
localhost:3000
localhost:3000
Hello, World!
OUTPUT TERMINAL PROBLEMS
TERMINAL
'Upgrade-Insecure-Requests',
'1',
'User-Agent',
'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0.0.0 Safari/537.36',
'Accept',
'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7',
'Sec-Fetch-Site',
'none',
'Sec-Fetch-Mode',
'navigate',
'Sec-Fetch-User',
'?1',
'Sec-Fetch-Dest',
'document',
'Accept-Encoding',
'gzip, deflate, br',
'Accept-Language',
'en-GB,en-US;q=0.9,en;q=0.8'
]
^C
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % node index.js
Server started on port 3000


index.js — Web Development Projects
index.js
Backend > 3.2 HTTP Requests > index.js > app.get("
import express from "express";
const app = express();
const port = 3000;
app.get("/", (req, res) => {
res.send("Hello, World!");
});
app.listen(port, () => {
console.log(`Server started o
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
'Upgrade-Insecure-Requests',
'1',
'User-Agent',
'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/113.0 0.0 Safari/537.36',
'Accept',
'text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,a lication/signed-exchange;v=b3;q=0.7',
'Sec-Fetch-Site',
'none',
'Sec-Fetch-Mode',
'navigate',
'Sec-Fetch-User',
'?1',
'Sec-Fetch-Dest',
'document',
'Accept-Encoding',
'gzip, deflate, br',
'Accept-Language',
'en-GB,en-US;q=0.9,en;q=0.8'
]
^C
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % node index.js
Server started on port 3000
Chrome File Edit View History Bookmarks Profiles Tab Window
localhost:3000
localhost:3000
Hello, World!
node
JS

nodemon DT

2.0.22 • Public • Published 2 months ago

Readme | Code Beta | 10 Dependencies

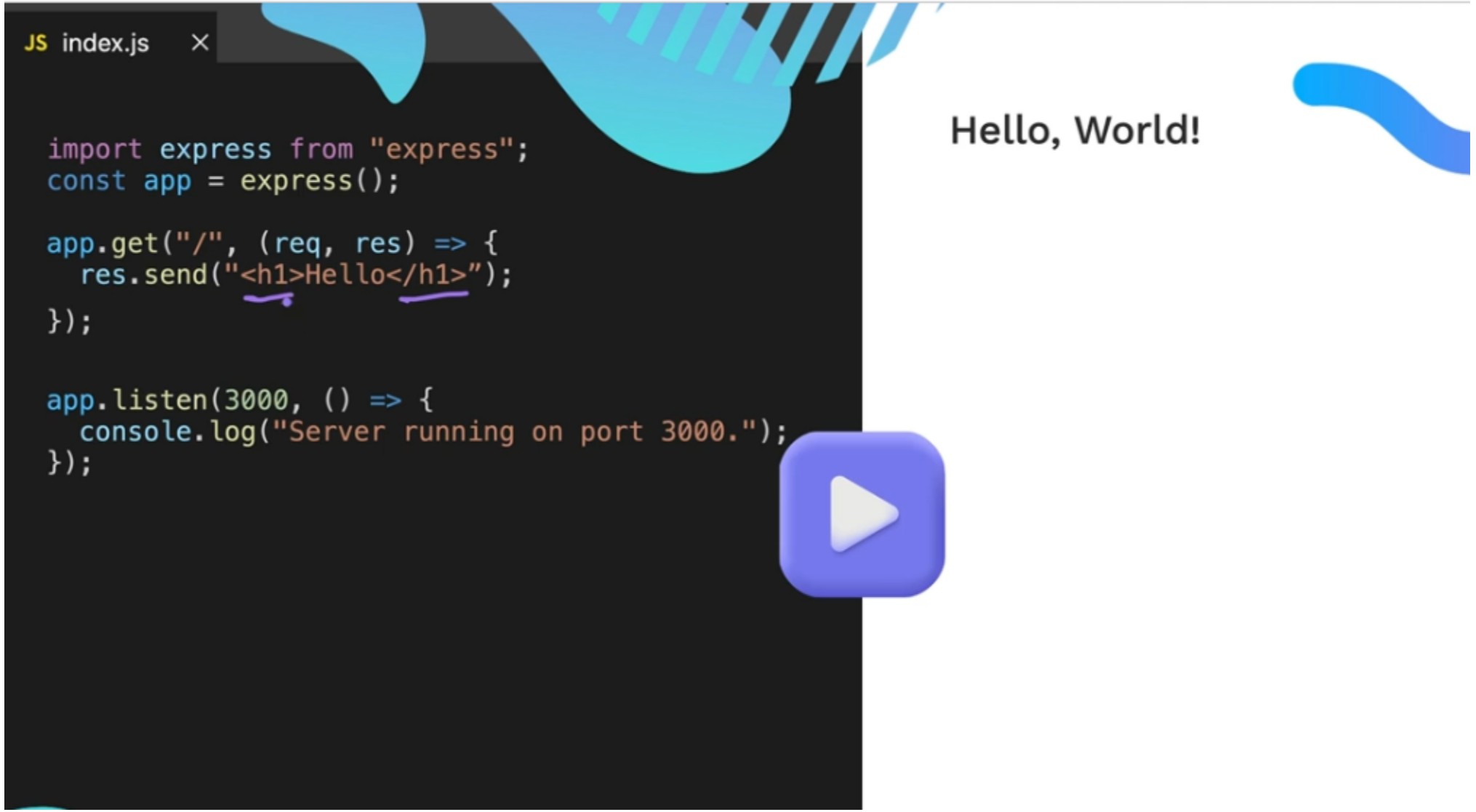


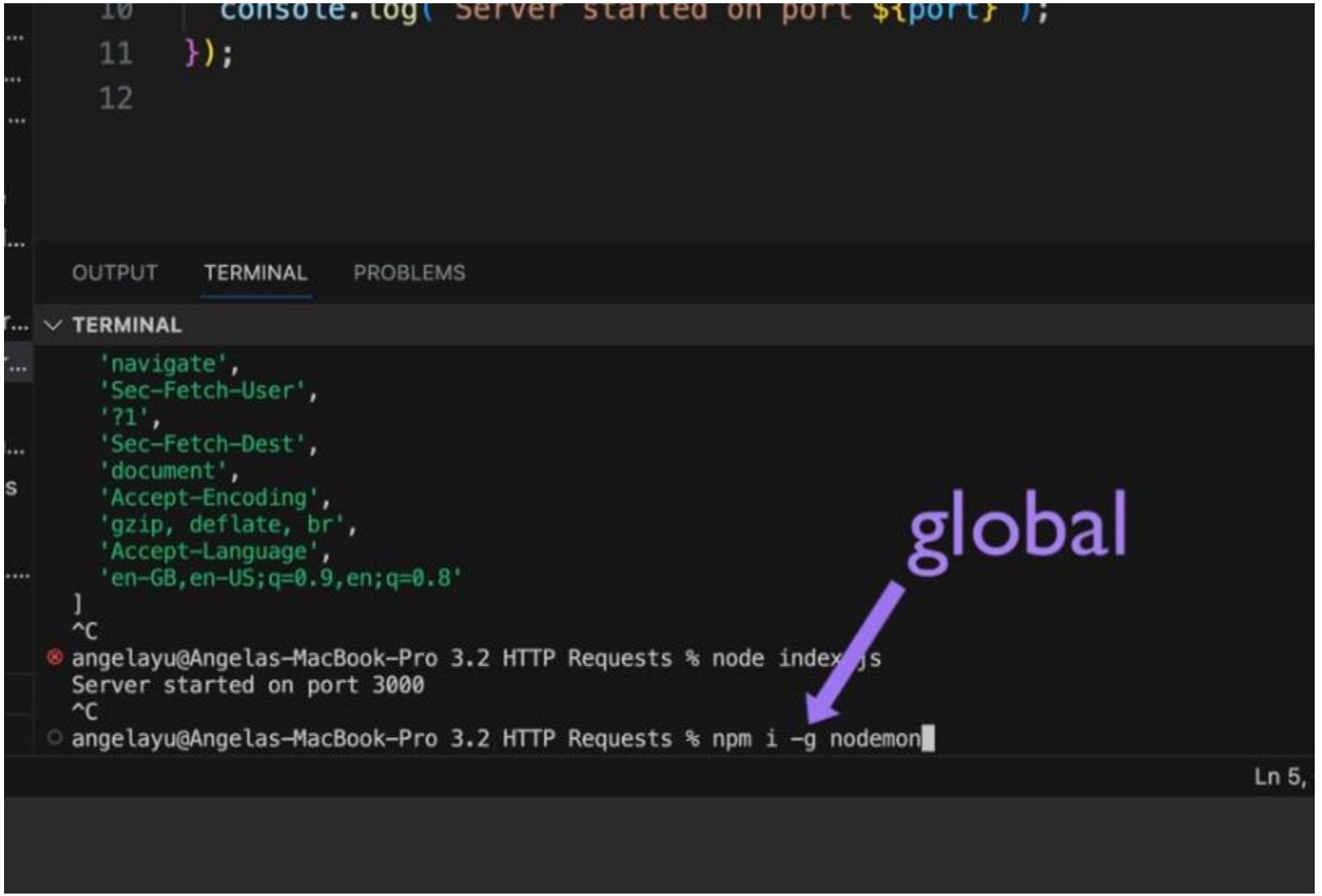
nodemon

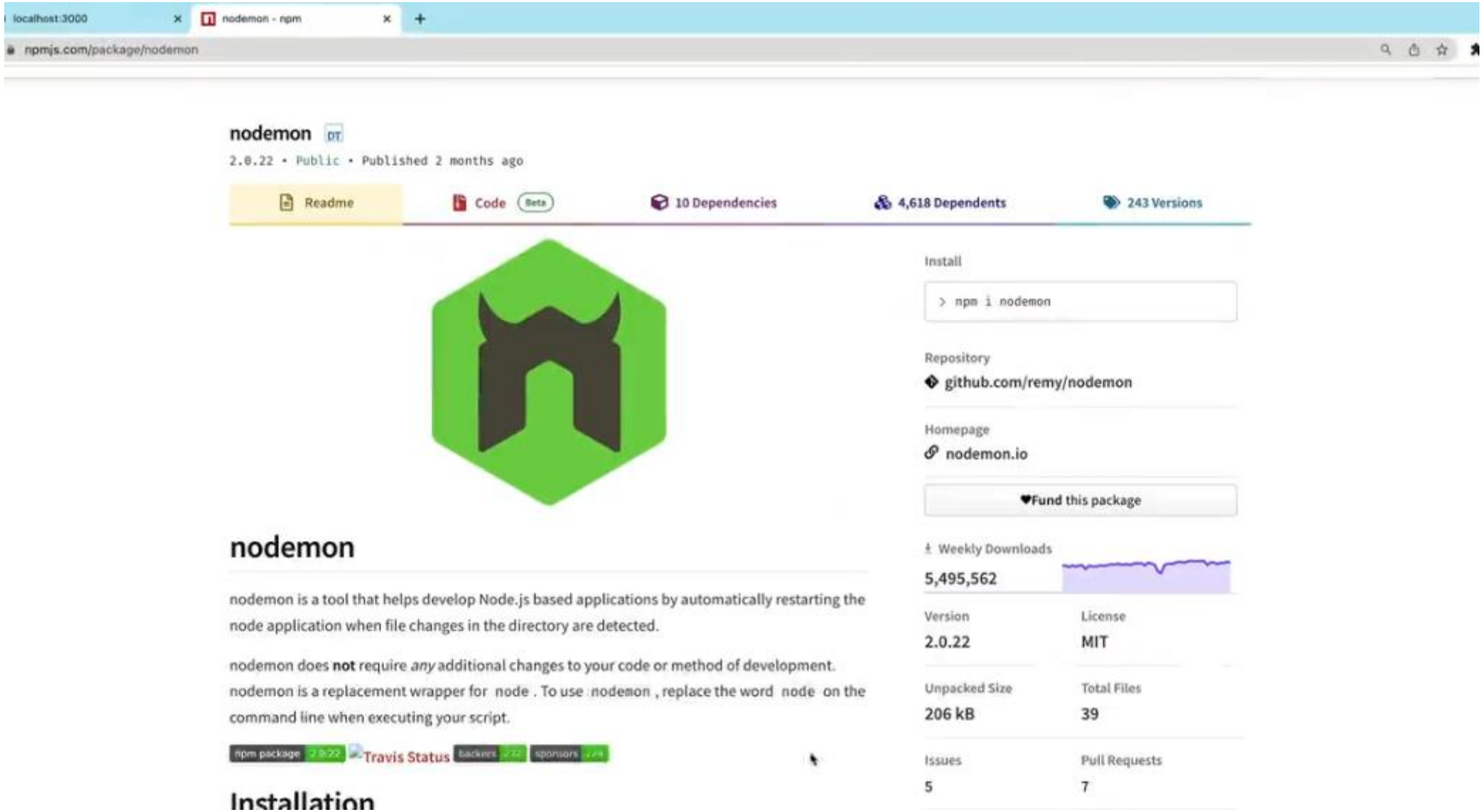
nodemon is a tool that helps develop Node.js based applications by automatically restarting the node application when file changes in the directory are detected.

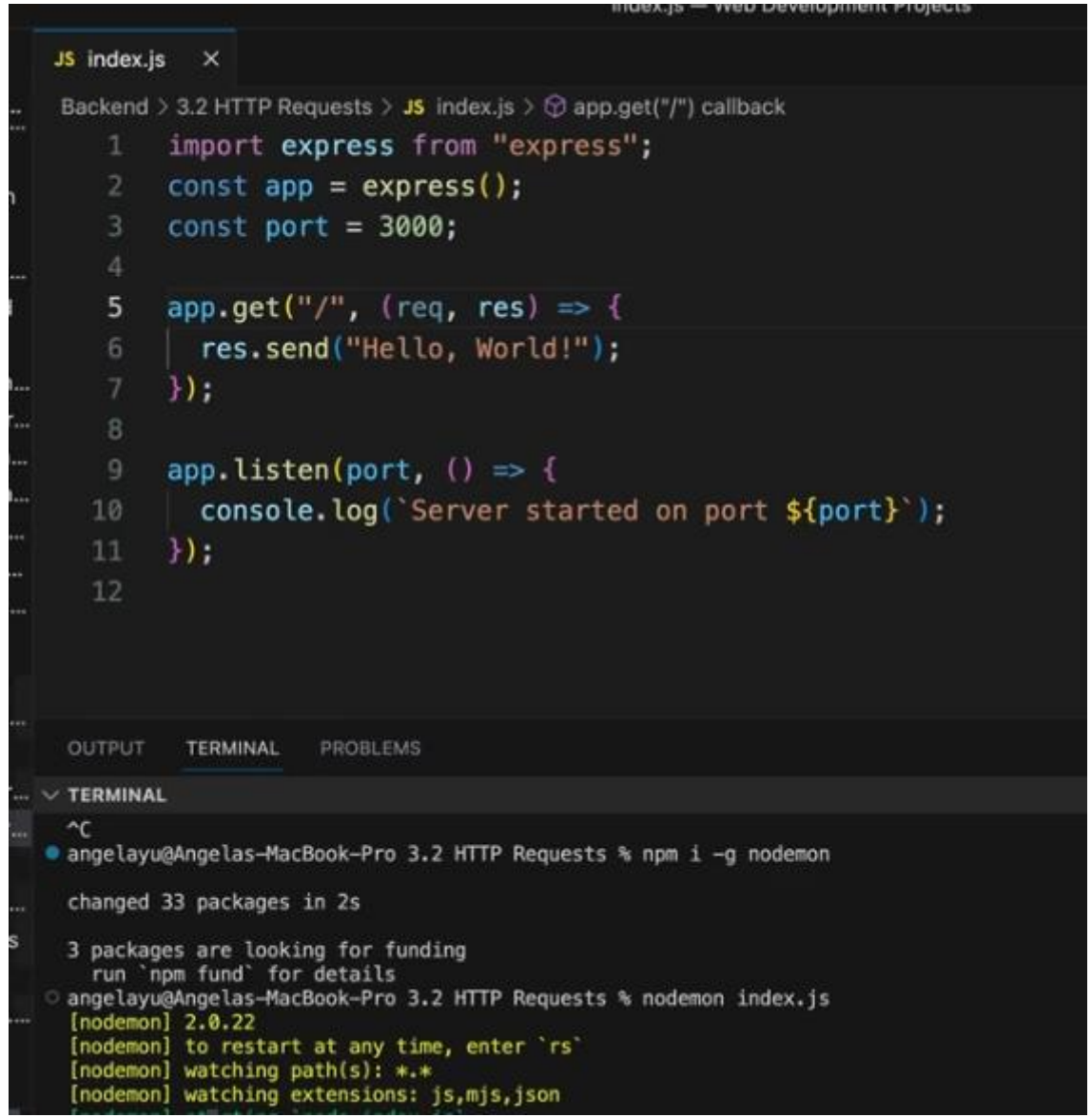
nodemon does **not** require *any* additional changes to your code or method of development. nodemon is a replacement wrapper for `node`. To use `nodemon`, replace the word `node` on the command line when executing your script.

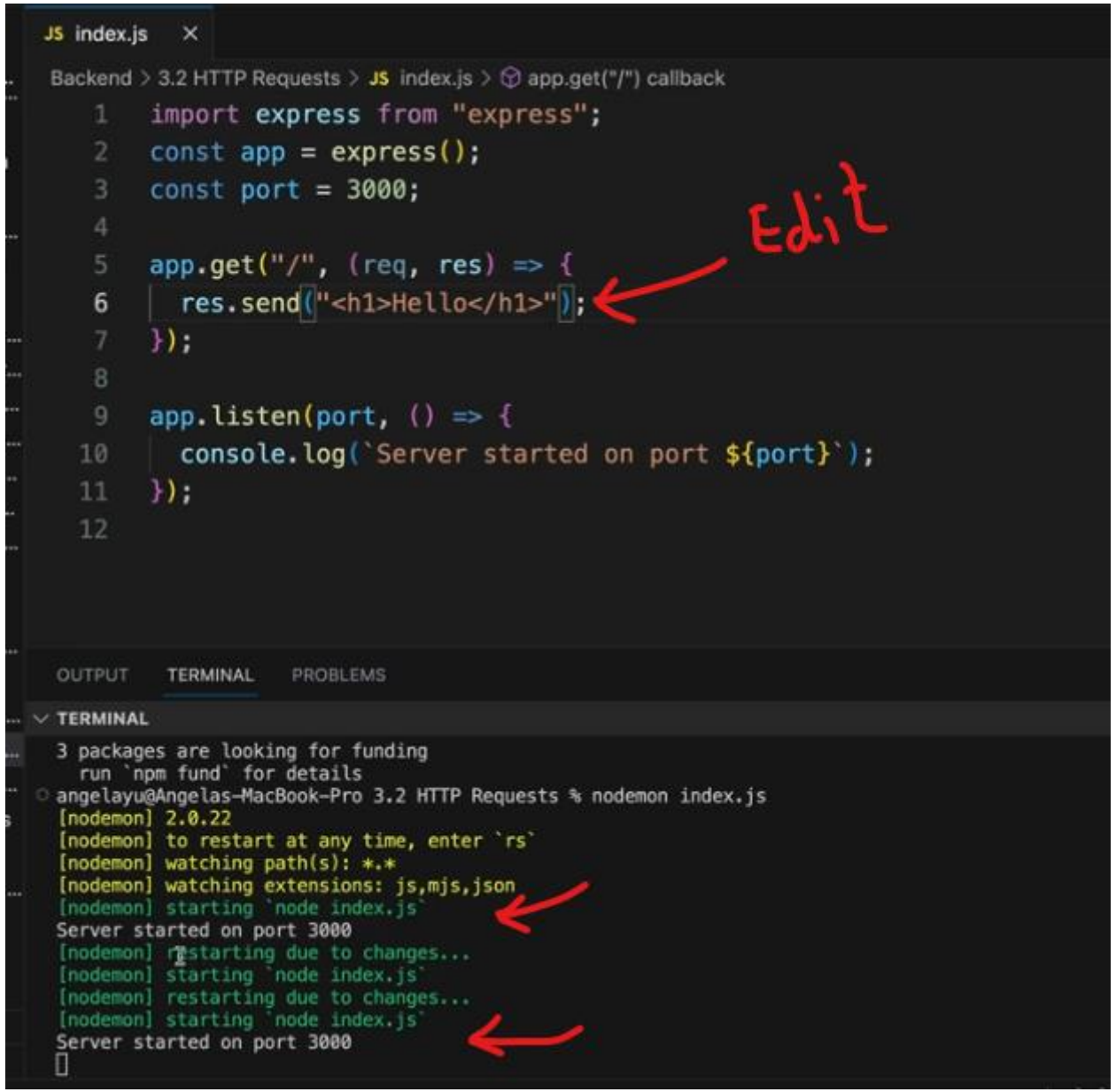

nodemon index.js


JS index.js
import express from "express";
const app = express();
app.get("/", (req, res) => {
res.send("<h1>Hello</h1>");
});
app.listen(3000, () => {
console.log("Server running on port 3000.");
});
Hello, World!


console.log(Server started on port ${port});
});
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
'navigate',
'Sec-Fetch-User',
'?1',
'Sec-Fetch-Dest',
'document',
'Accept-Encoding',
'gzip, deflate, br',
'Accept-Language',
'en-GB,en-US;q=0.9,en;q=0.8'
]
^C
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % node index.js
Server started on port 3000
^C
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % npm i -g nodemon
global
Ln 5,


localhost:3000
nodemon - npm
npmjs.com/package/nodemon
nodemon
DT
2.0.22 • Public • Published 2 months ago
Readme
Code
Beta
10 Dependencies
4,618 Dependents
243 Versions
nodemon
nodemon is a tool that helps develop Node.js based applications by automatically restarting the node application when file changes in the directory are detected.
nodemon does not require any additional changes to your code or method of development. nodemon is a replacement wrapper for node . To use nodemon , replace the word node on the command line when executing your script.
npm package
Travis Status
backers
sponsors
Installation
Install
> npm i nodemon
Repository
github.com/remy/nodemon
Homepage
nodemon.io
❤Fund this package
Weekly Downloads
5,495,562
Version
2.0.22
License
MIT
Unpacked Size
206 kB
Total Files
39
Issues
5
Pull Requests
7


index.js — Web Development Projects
index.js
Backend > 3.2 HTTP Requests > index.js > app.get("/") callback
1 import express from "express";
2 const app = express();
3 const port = 3000;
4
5 app.get("/", (req, res) => {
6 res.send("Hello, World!");
7 });
8
9 app.listen(port, () => {
10 console.log(`Server started on port ${port}`);
11 });
12
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
^C
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % npm i -g nodemon
changed 33 packages in 2s
3 packages are looking for funding
run `npm fund` for details
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % nodemon index.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json


index.js
Backend > 3.2 HTTP Requests > index.js > app.get("/") callback
import express from "express";
const app = express();
const port = 3000;
app.get("/", (req, res) => {
res.send("<h1>Hello</h1>");
});
app.listen(port, () => {
console.log(`Server started on port ${port}`);
});
Edit
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
3 packages are looking for funding
run `npm fund` for details
angelayu@Angelas-MacBook-Pro 3.2 HTTP Requests % nodemon index.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Server started on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server started on port 3000

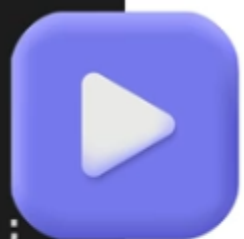
JS index.js

```
import express from "express";
const app = express();

app.get("/", (req, res) => {
  res.send("<h1>Home Page</h1>");
});


app.get("/about", (req, res) => {
  res.send("<h1>About Me</h1>");
});

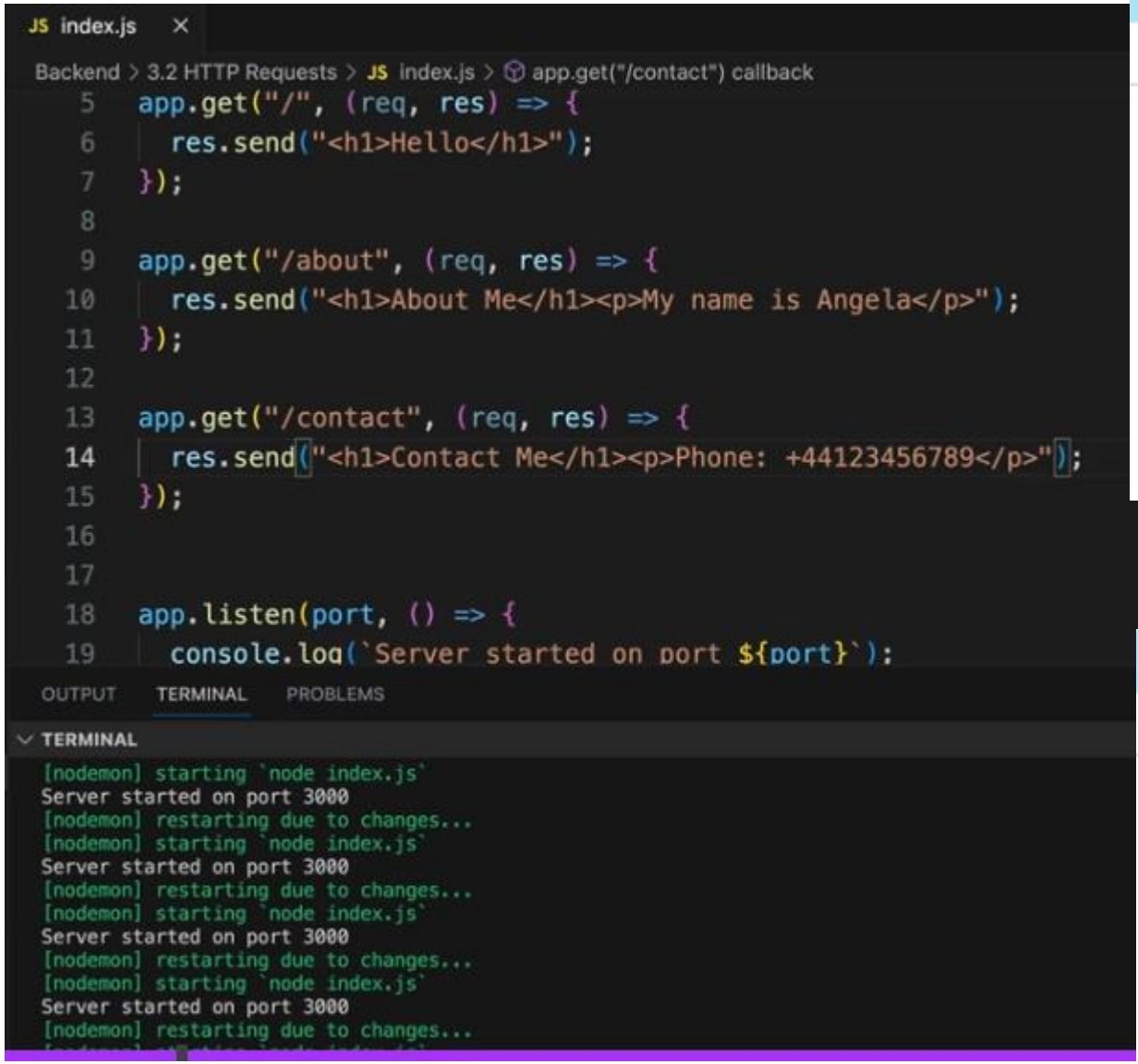

app.listen(3000, () => {
  console.log("Server running on port 3000.");
});
```

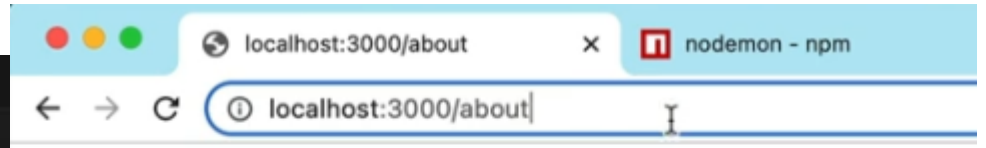


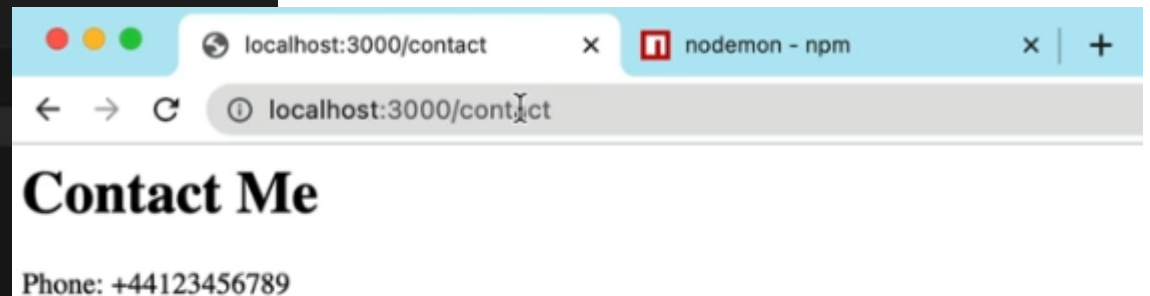


Exercise

1. Create directory.
2. Create index.js file.
3. Initialise NPM.
4. Install the Express package.
5. Write Server application in index.js.
6. Server should be able to handle “/” “/contact” and “/about” endpoints.
7. Start server.

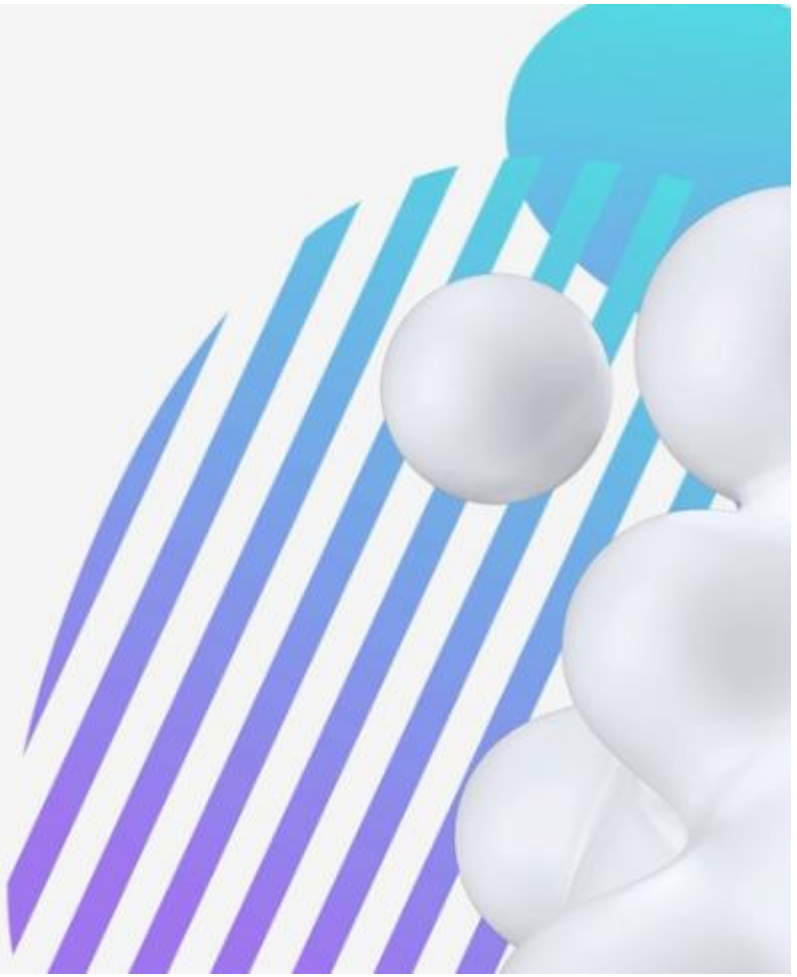

index.js
Backend > 3.2 HTTP Requests > index.js > app.get("/contact") callback
app.get("/", (req, res) => {
res.send("<h1>Hello</h1>");
});
app.get("/about", (req, res) => {
res.send("<h1>About Me</h1> My name is Angela ");
});
app.get("/contact", (req, res) => {
res.send("<h1>Contact Me</h1> Phone: +44123456789 ");
});
app.listen(port, () => {
console.log(`Server started on port ${port}`);
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
[nodemon] starting `node index.js`
Server started on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server started on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server started on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server started on port 3000
[nodemon] restarting due to changes...


localhost:3000/about
nodemon - npm
localhost:3000/about
About Me
My name is Angela


localhost:3000/contact
nodemon - npm
localhost:3000/contact
Contact Me
Phone: +44123456789

Making Requests

- Postman and making other HTTP requests



www.appbrewery.com

Client-Side

Server-Side



Client

HTTP Req

Response

"Hello"

<h1> Hello </h1>



Application

Server

Database

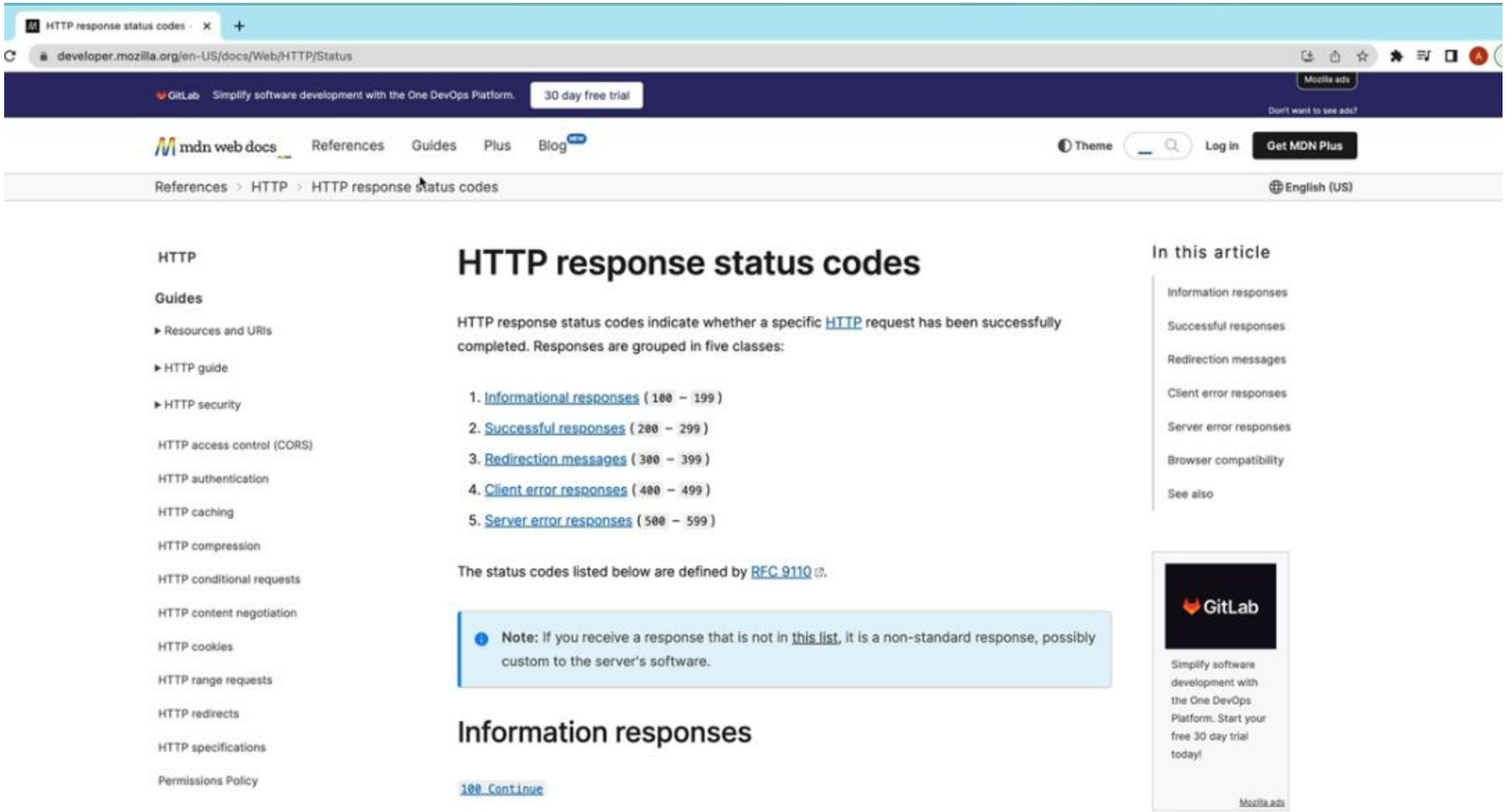
HTTP Response Status Codes

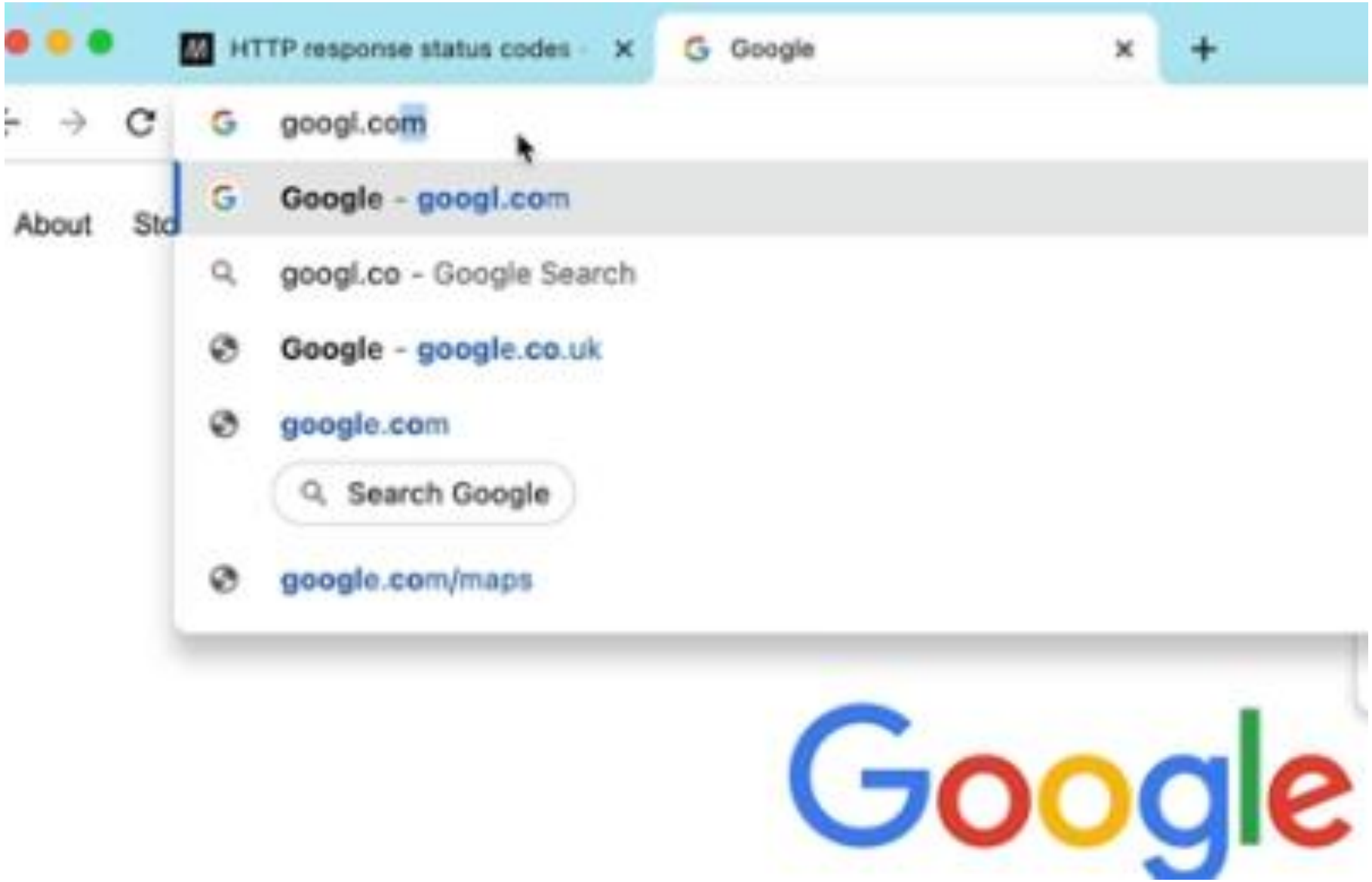
HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:

1. Informational responses (`100` – `199`)
2. Successful responses (`200` – `299`)
3. Redirection messages (`300` – `399`)
4. Client error responses (`400` – `499`)
5. Server error responses (`500` – `599`)

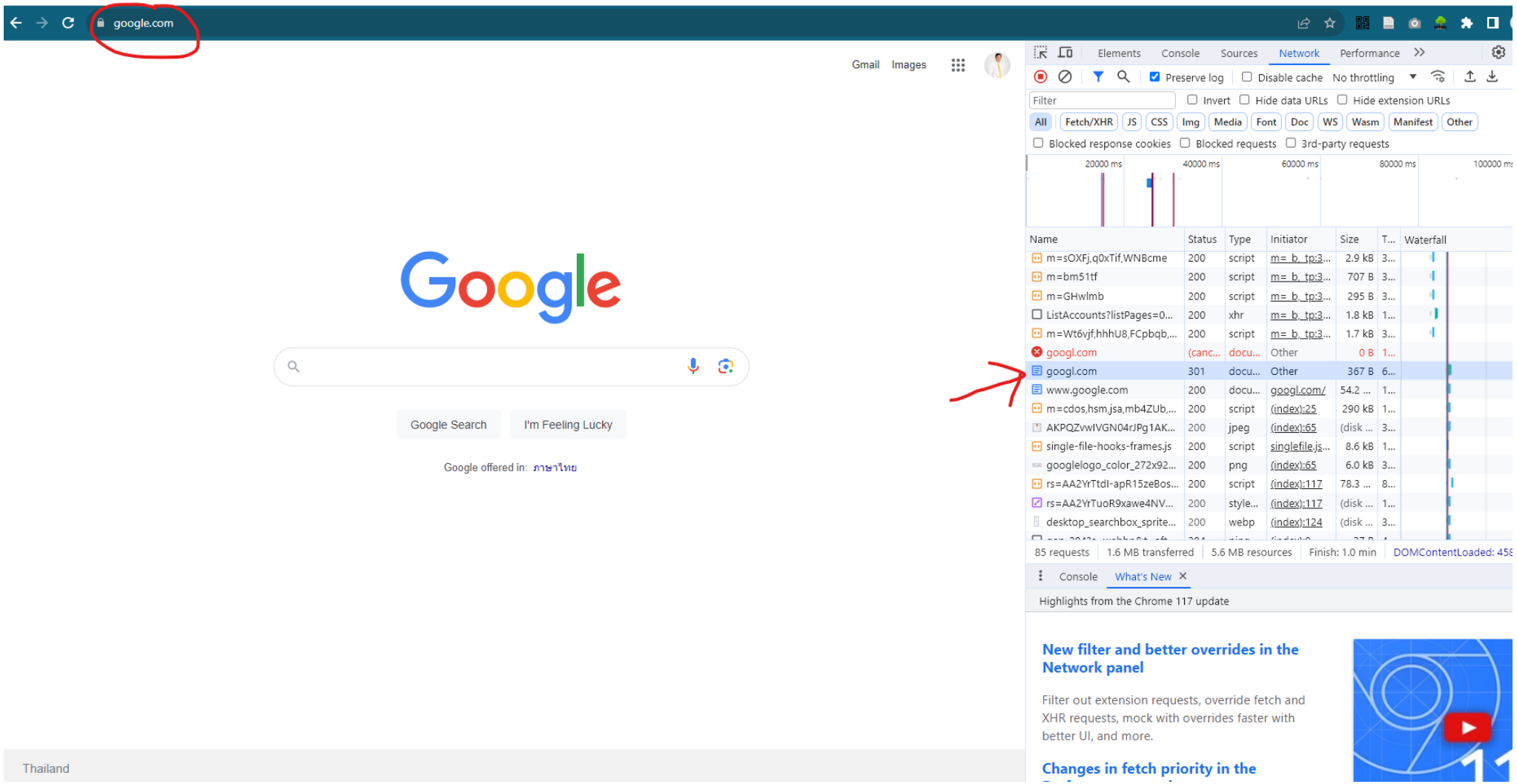
The status codes listed below are defined by RFC 9110.

developer.mozilla.org/docs/Web/HTTP/Status


HTTP response status codes
developer.mozilla.org/en-US/docs/Web/HTTP/Status
GitLab Simplify software development with the One DevOps Platform.
30 day free trial
Mozilla ads
Don't want to see ads?
mdn web docs
References
Guides
Plus
Blog
Theme
Log in
Get MDN Plus
References > HTTP > HTTP response status codes
English (US)
HTTP
Guides
▸ Resources and URIs
▸ HTTP guide
▸ HTTP security
HTTP access control (CORS)
HTTP authentication
HTTP caching
HTTP compression
HTTP conditional requests
HTTP content negotiation
HTTP cookies
HTTP range requests
HTTP redirects
HTTP specifications
Permissions Policy
HTTP response status codes
HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:
1. Informational responses (100 – 199)
2. Successful responses (200 – 299)
3. Redirection messages (300 – 399)
4. Client error responses (400 – 499)
5. Server error responses (500 – 599)
The status codes listed below are defined by RFC 9110.
Note: If you receive a response that is not in this list, it is a non-standard response, possibly custom to the server's software.
Information responses
100 Continue
In this article
Information responses
Successful responses
Redirection messages
Client error responses
Server error responses
Browser compatibility
See also
GitLab
Simplify software development with the One DevOps Platform. Start your free 30 day trial today!
Mozilla ads


HTTP response status codes
Google
googl.com
About
Std
Google - googl.com
googl.co - Google Search
Google - google.co.uk
google.com
Search Google
google.com/maps
Google

Monitor HTTP status



HTTP

Guides

▸ Resources and URIs

▸ HTTP guide

▸ HTTP security

HTTP access control (CORS)

HTTP authentication

HTTP caching

HTTP compression

HTTP conditional requests

HTTP content negotiation

HTTP cookies

HTTP range requests

HTTP redirects

HTTP specifications

Permissions Policy

References

▸ HTTP headers

The server has fulfilled a GET request for the resource, and the response is a representation of the result of one or more instance-manipulations applied to the current instance.

Redirection messages

`300 Multiple Choices`

The request has more than one possible response. The user agent or user should choose one of them. (There is no standardized way of choosing one of the responses, but HTML links to the possibilities are recommended so the user can pick.)

`301 Moved Permanently`

The URL of the requested resource has been changed permanently. The new URL is given in the response.

`302 Found`

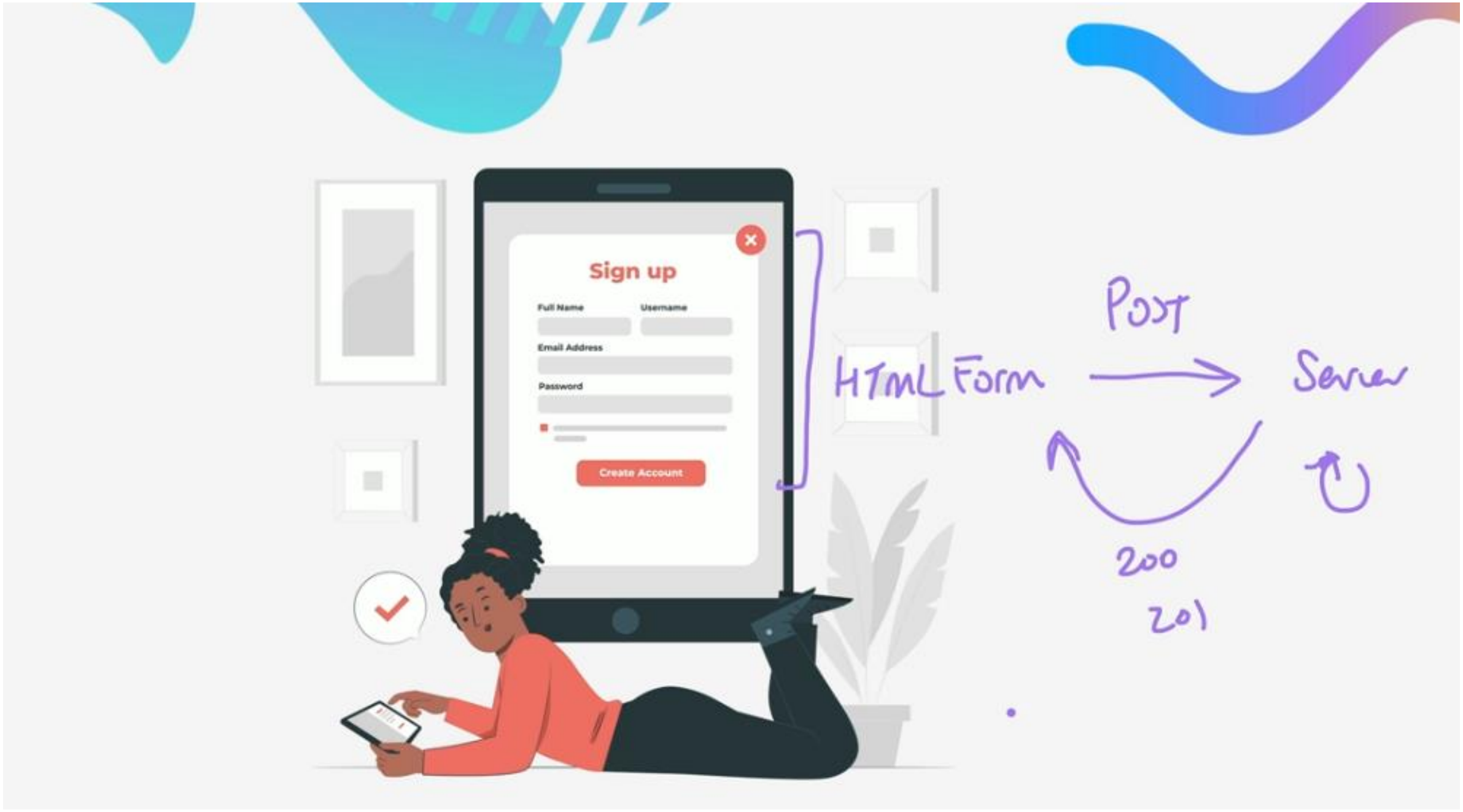
This response code means that the URI of requested resource has been changed *temporarily*. Further changes in the URI might be made in the future. Therefore, this same URI should be used by the client in future requests.

`303 See Other`

The server sent this response to direct the client to get the requested resource at another URI with a GET request.

`304 Not Modified`

This is used for caching purposes. It tells the client that the response has not been modified, so


Sign up
Full Name
Username
Email Address
Password
Create Account
HTML Form
POST
Server
200
201

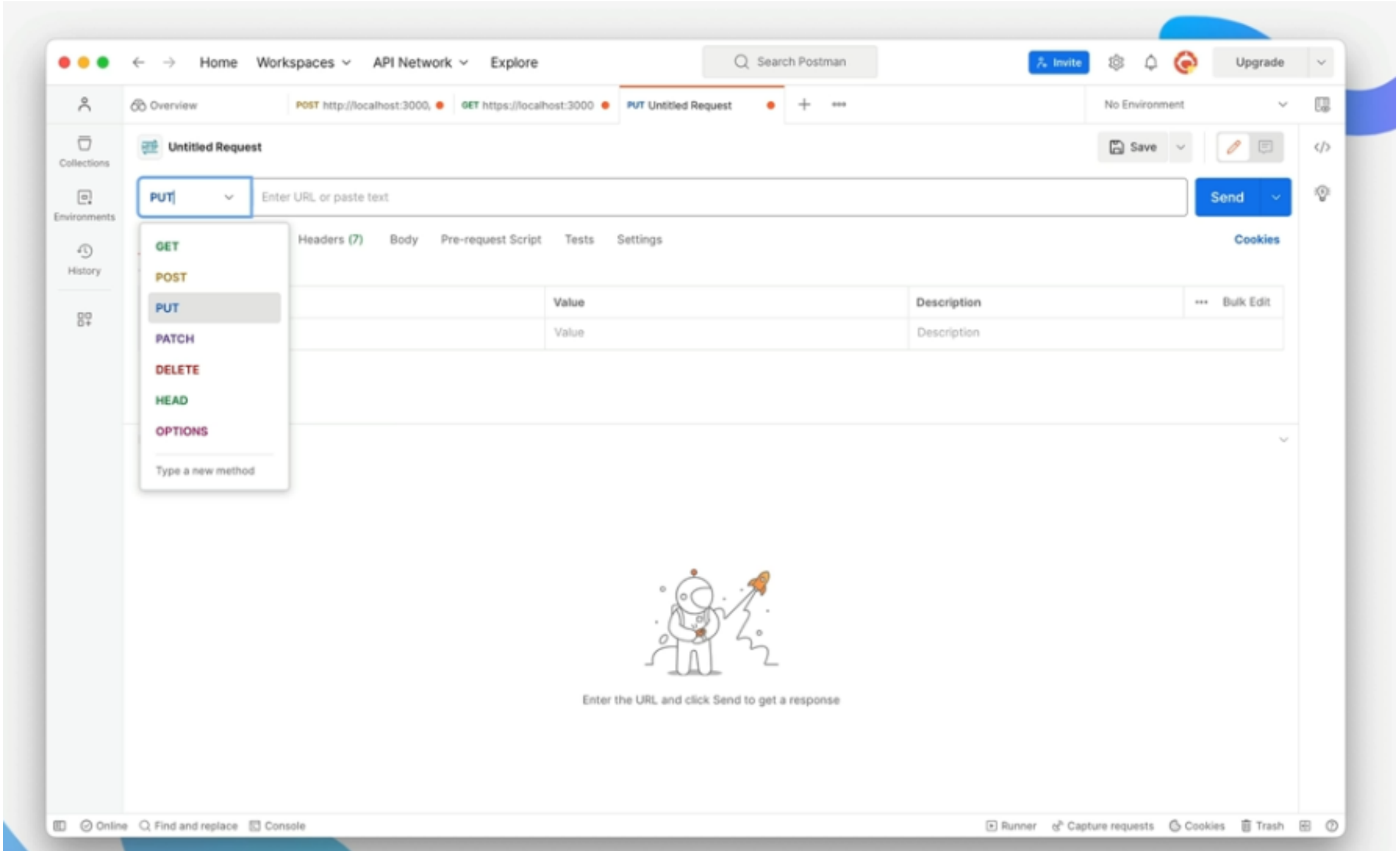
Create a backend without a frontend

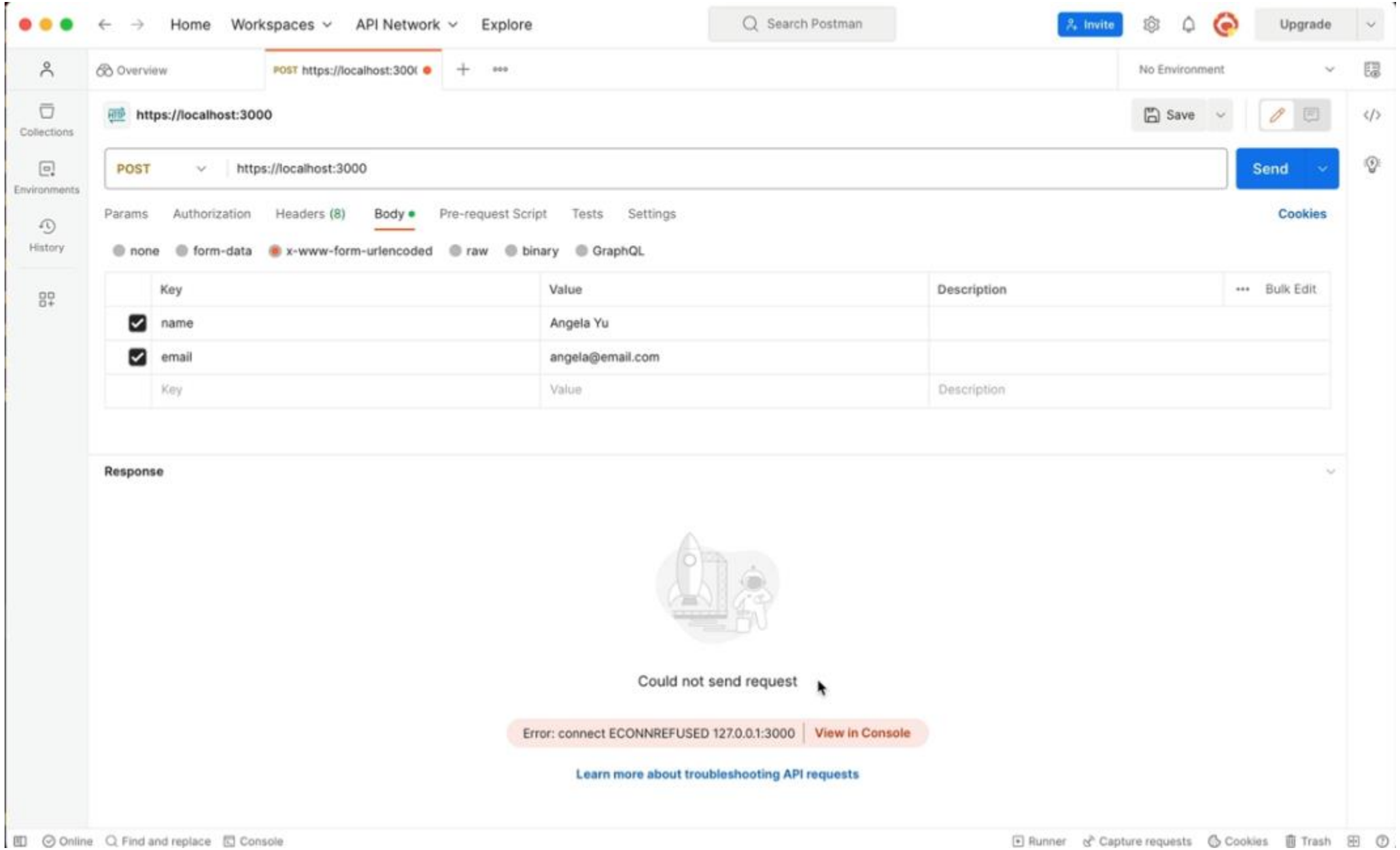


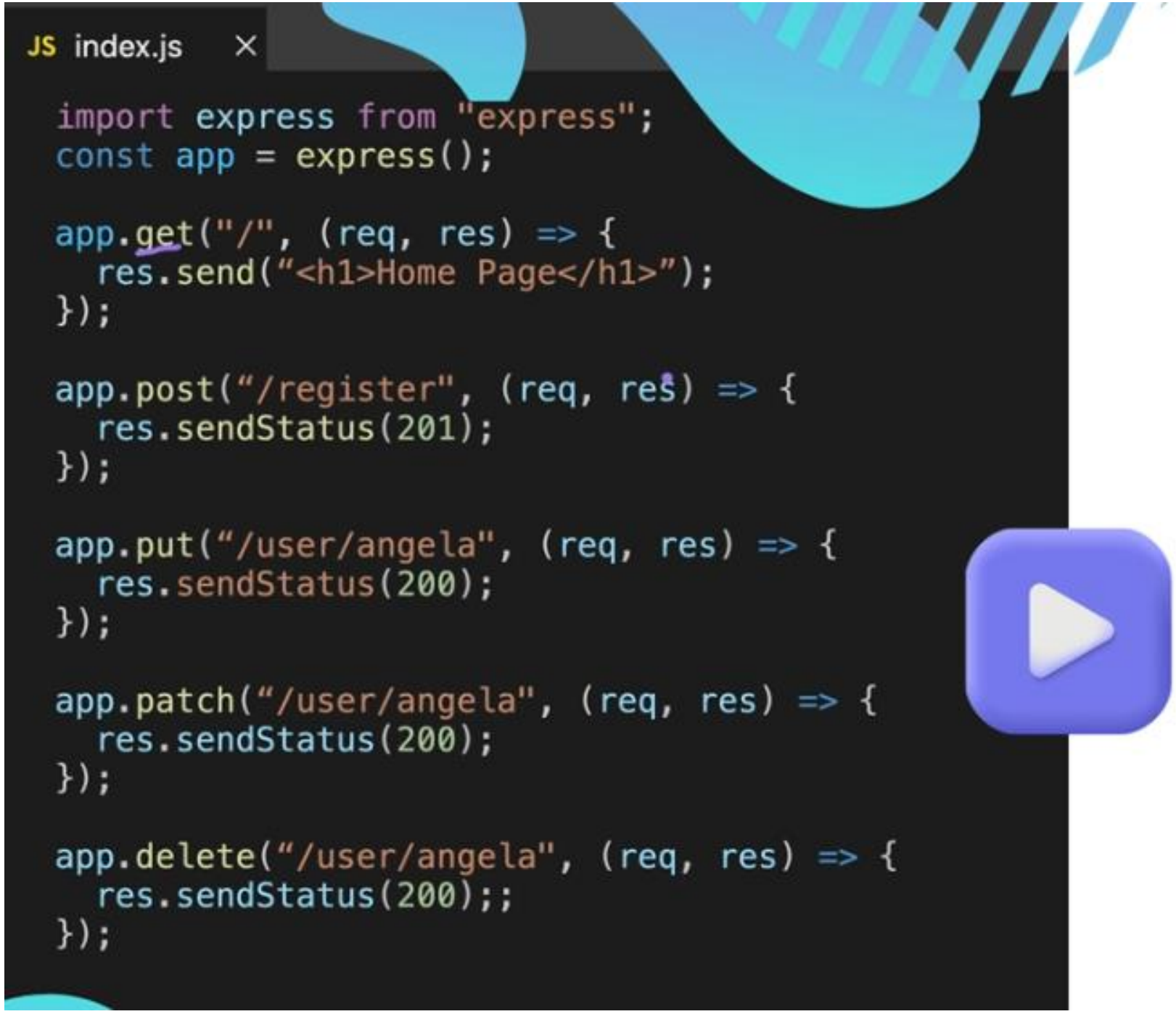
Postman



- https://www.postman.com/downloads/


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
POST http://localhost:3000,
GET https://localhost:3000
PUT Untitled Request
No Environment
Collections
Environments
History
Untitled Request
Save
PUT
Enter URL or paste text
Send
GET
POST
PUT
PATCH
DELETE
HEAD
OPTIONS
Type a new method
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Value
Description
Bulk Edit
Value
Description
Enter the URL and click Send to get a response
Online
Find and replace
Console
Runner
Capture requests
Cookies
Trash


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
POST https://localhost:300(
No Environment
Collections
Environments
History
https://localhost:3000
Save
POST
https://localhost:3000
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
Key
Value
Description
Bulk Edit
name
Angela Yu
email
angela@email.com
Key
Value
Description
Response
Could not send request
Error: connect ECONNREFUSED 127.0.0.1:3000
View in Console
Learn more about troubleshooting API requests
Online
Find and replace
Console
Runner
Capture requests
Cookies
Trash

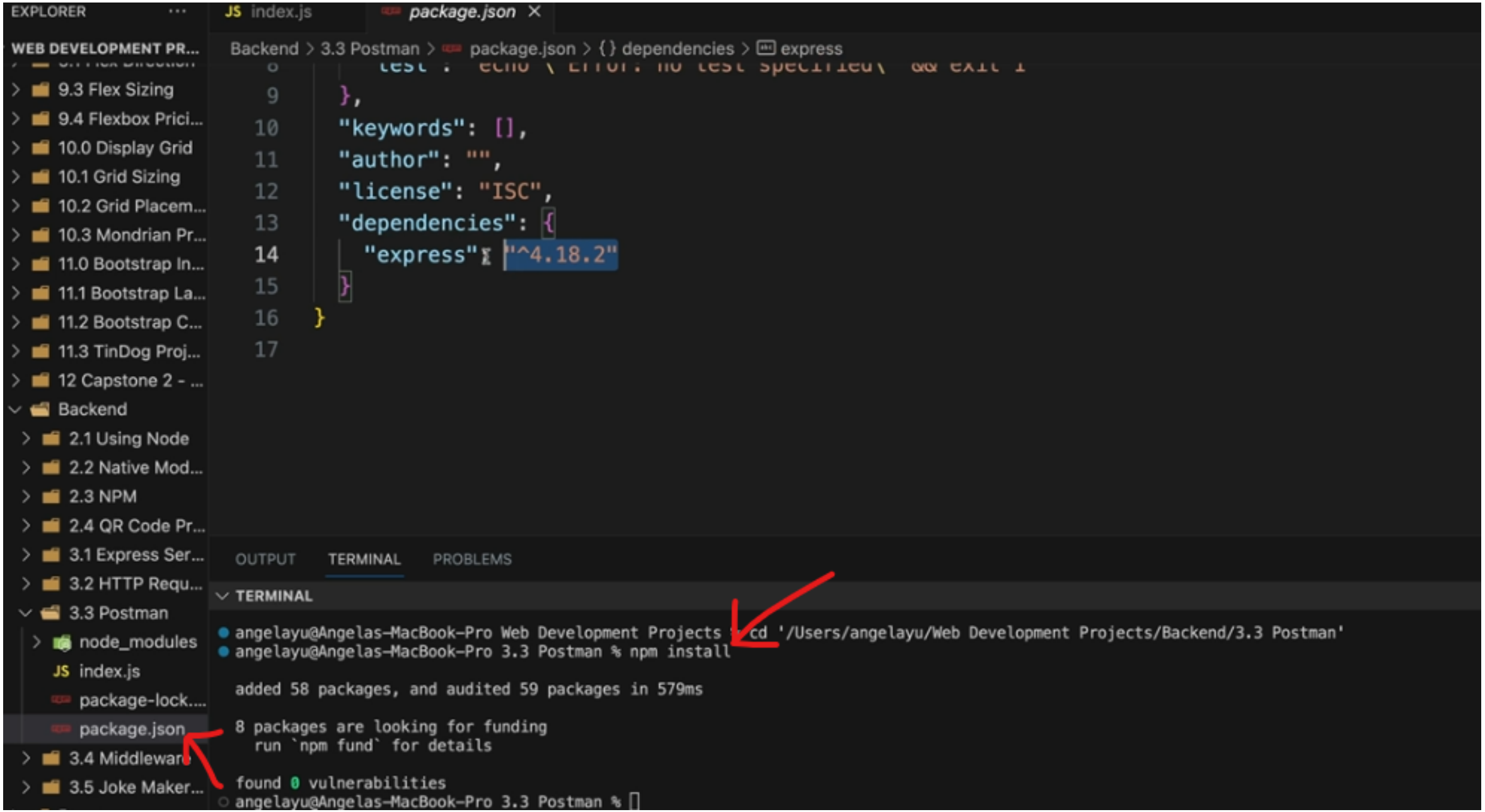

JS index.js
import express from "express";
const app = express();
app.get("/", (req, res) => {
res.send("<h1>Home Page</h1>");
});
app.post("/register", (req, res) => {
res.sendStatus(201);
});
app.put("/user/angela", (req, res) => {
res.sendStatus(200);
});
app.patch("/user/angela", (req, res) => {
res.sendStatus(200);
});
app.delete("/user/angela", (req, res) => {
res.sendStatus(200);;
});

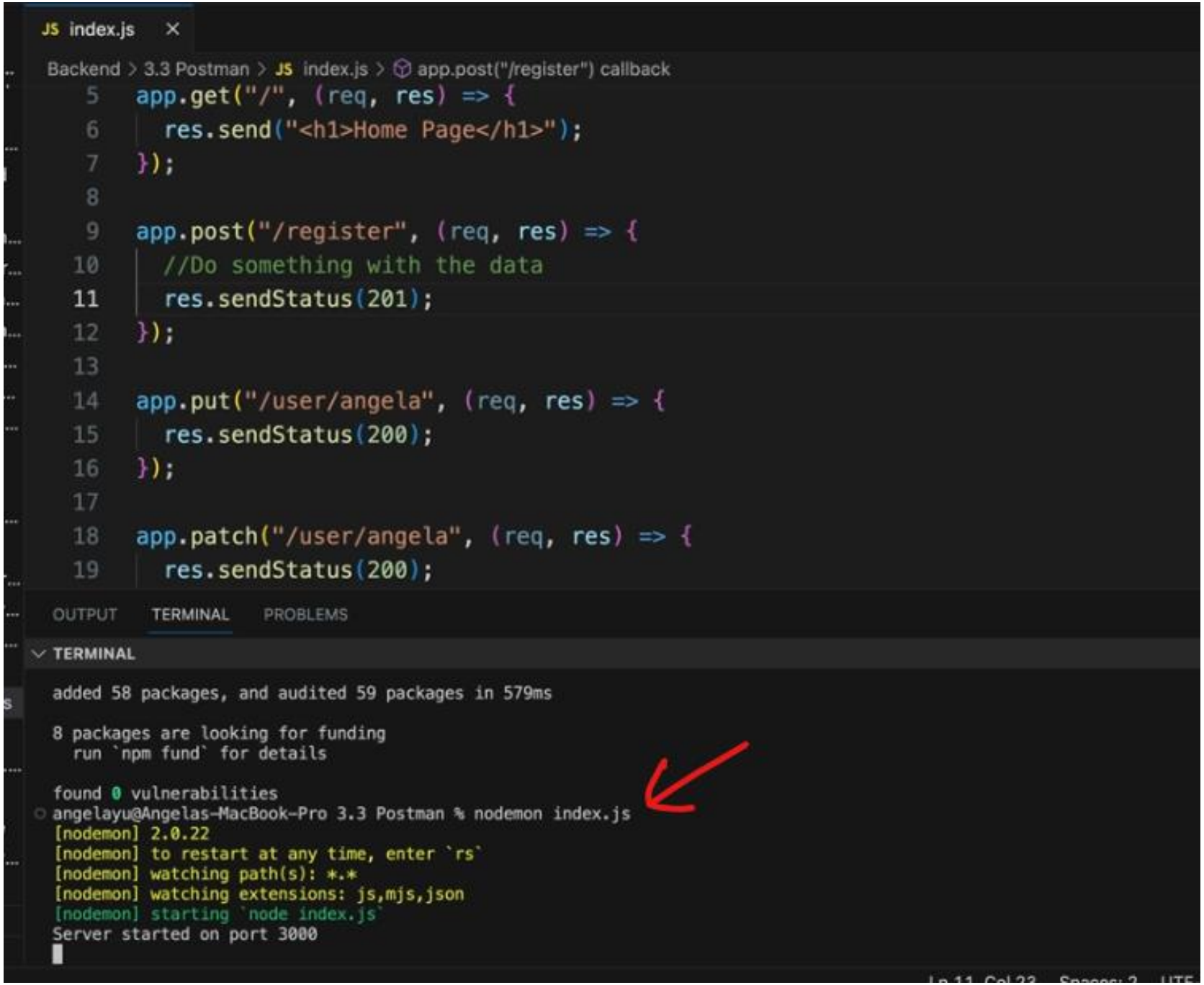
Exercise

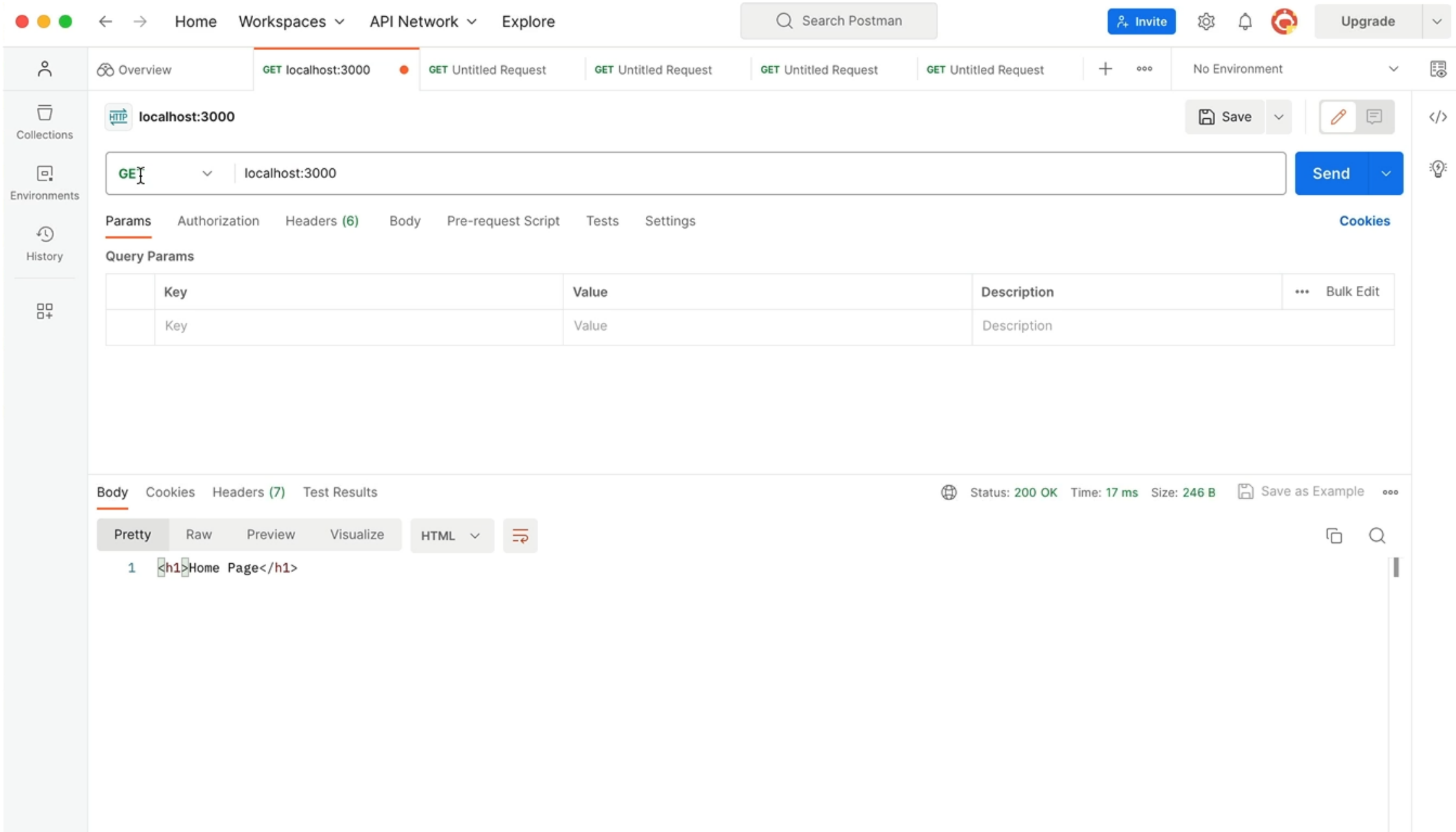
Try an Exercise

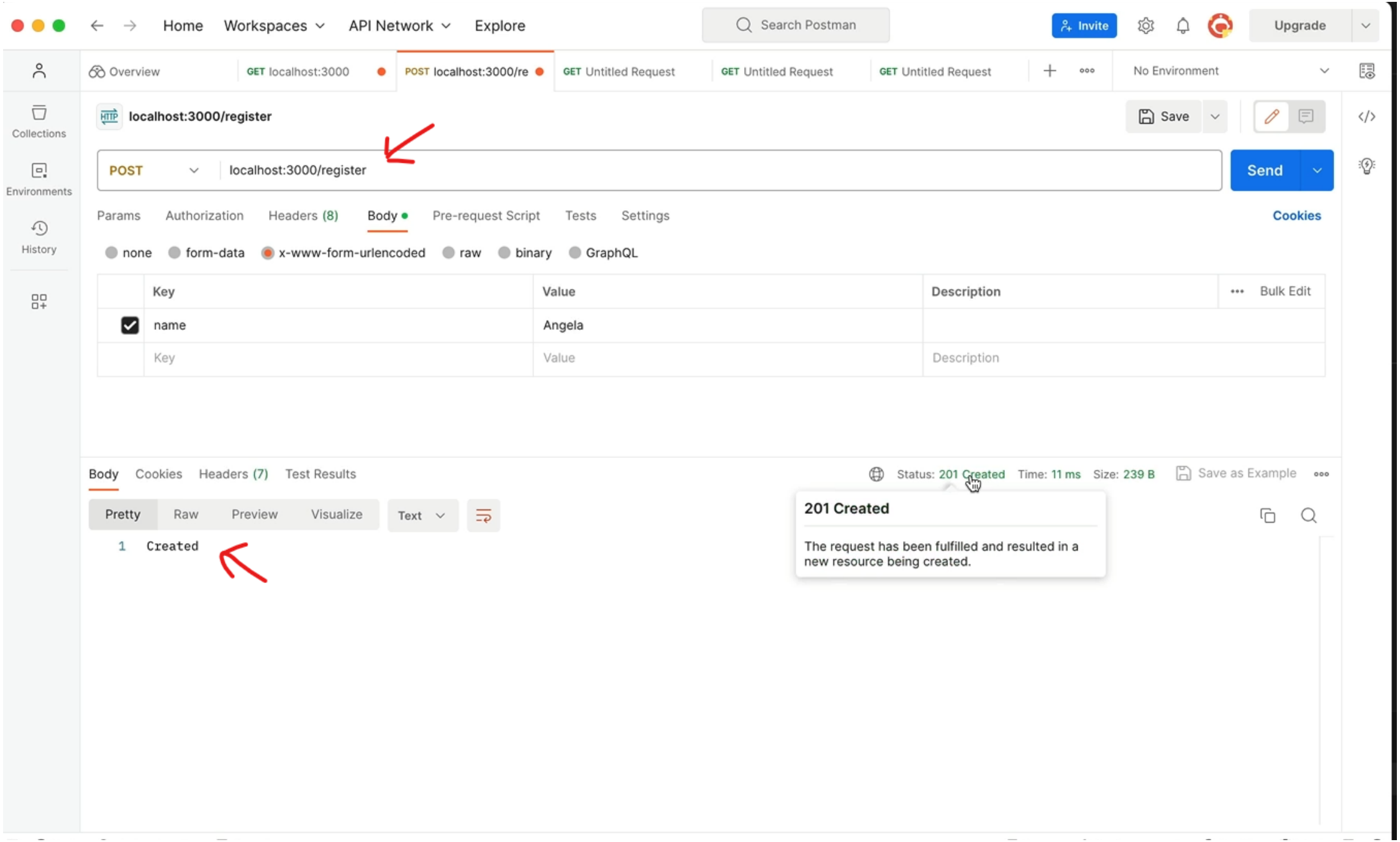


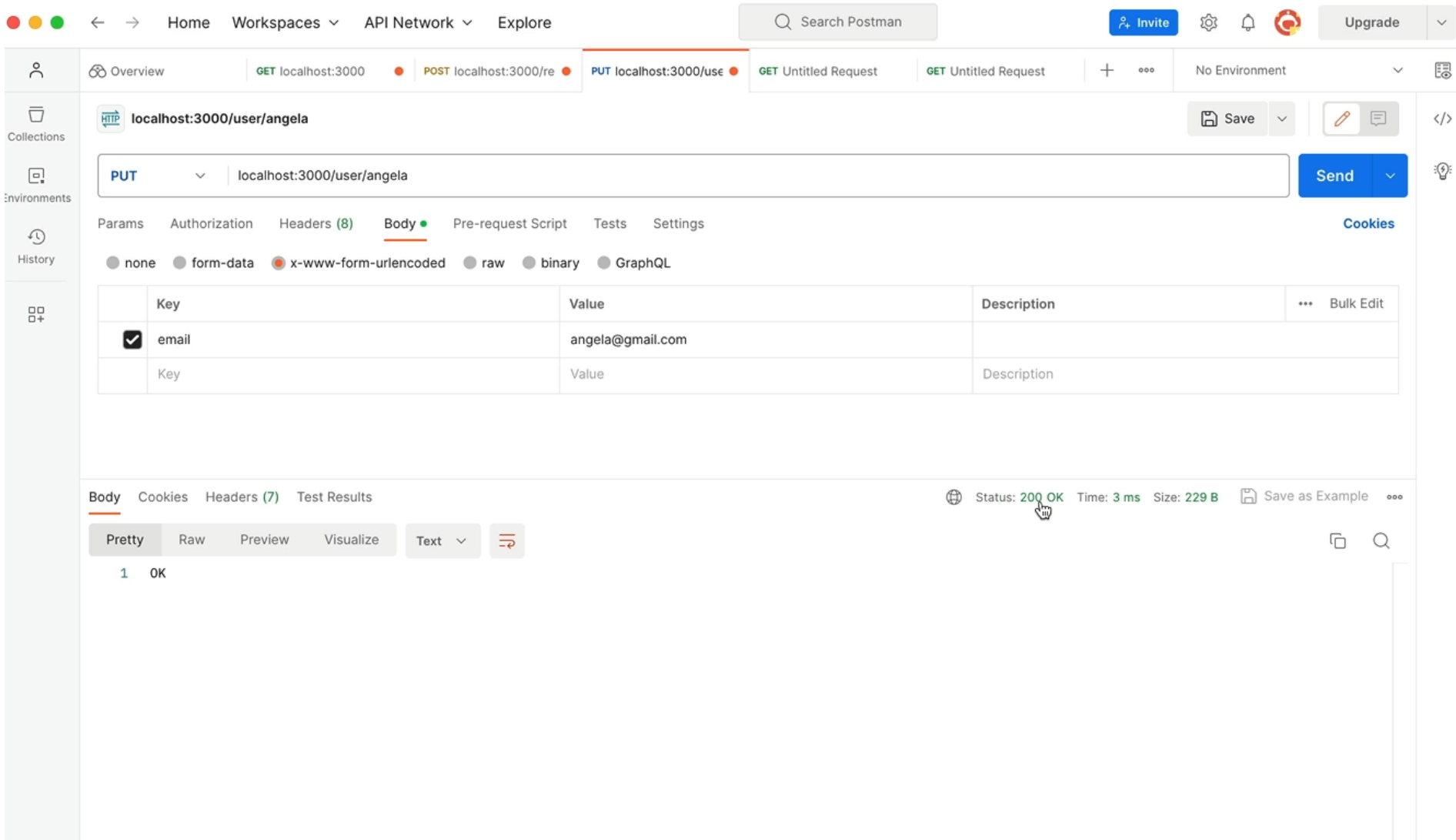
3.3 Postman.zip

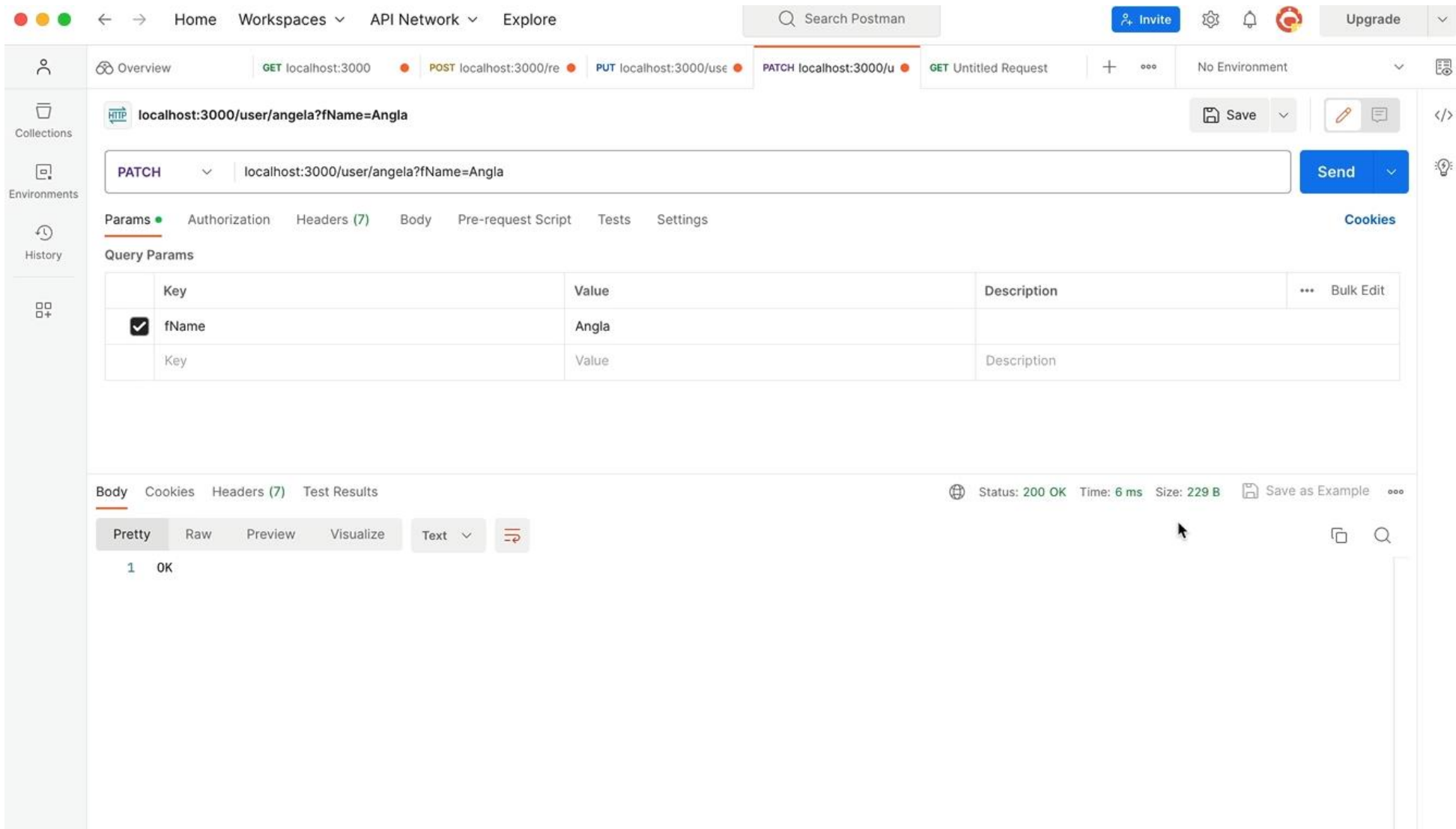

EXPLORER
WEB DEVELOPMENT PR...
9.3 Flex Sizing
9.4 Flexbox Prici...
10.0 Display Grid
10.1 Grid Sizing
10.2 Grid Placem...
10.3 Mondrian Pr...
11.0 Bootstrap In...
11.1 Bootstrap La...
11.2 Bootstrap C...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
node_modules
index.js
package-lock....
package.json
3.4 Middlewar
3.5 Joke Maker...
JS index.js
package.json
Backend > 3.3 Postman > package.json > { } dependencies > express
},
"keywords": [],
"author": "",
"license": "ISC",
"dependencies": {
"express": "^4.18.2"
}
}
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro Web Development Projects % cd '/Users/angelayu/Web Development Projects/Backend/3.3 Postman'
angelayu@Angelas-MacBook-Pro 3.3 Postman % npm install
added 58 packages, and audited 59 packages in 579ms
8 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
angelayu@Angelas-MacBook-Pro 3.3 Postman %

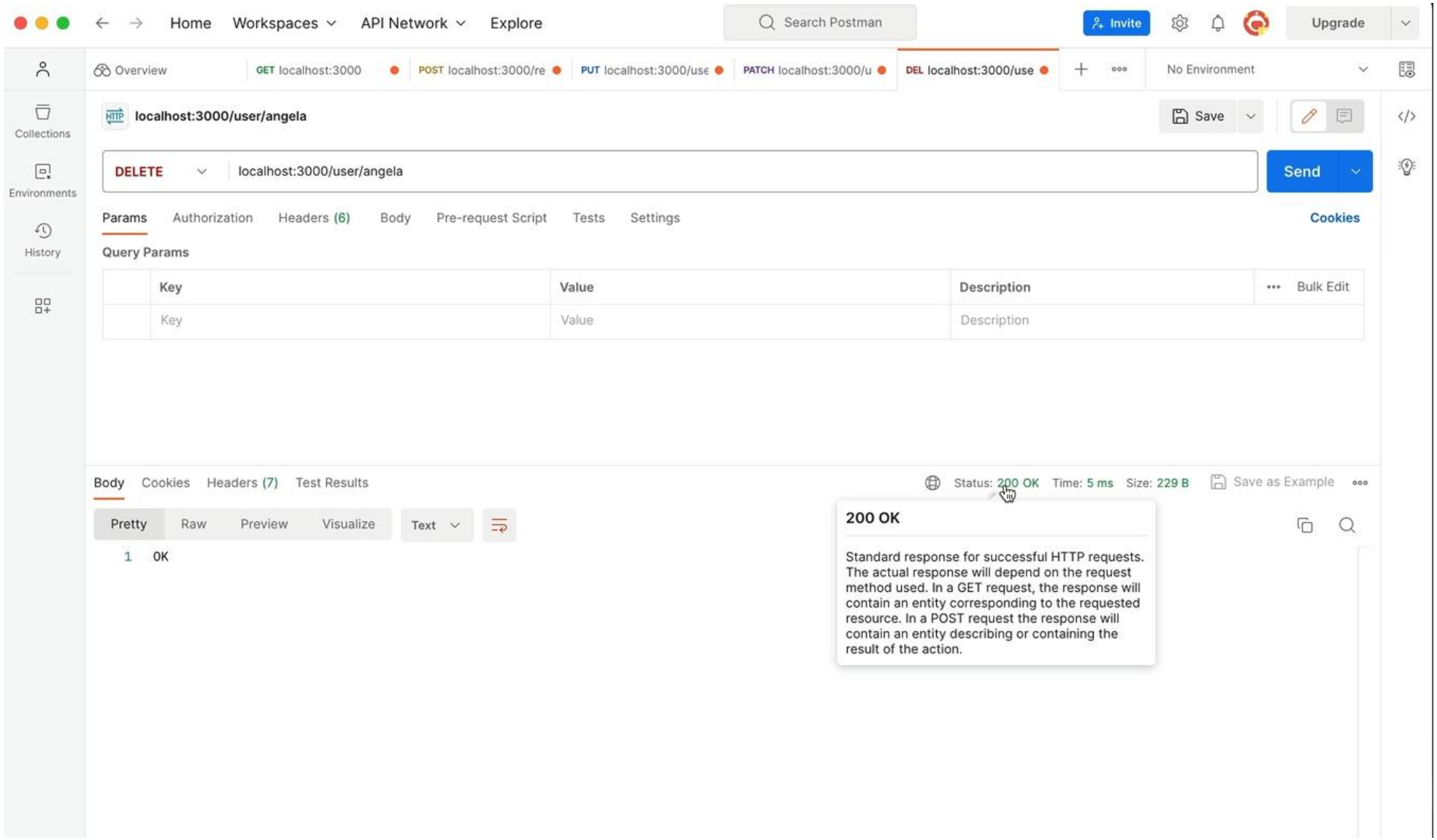

index.js
Backend > 3.3 Postman > index.js > app.post("/register") callback
app.get("/", (req, res) => {
res.send("<h1>Home Page</h1>");
});
app.post("/register", (req, res) => {
//Do something with the data
res.sendStatus(201);
});
app.put("/user/angela", (req, res) => {
res.sendStatus(200);
});
app.patch("/user/angela", (req, res) => {
res.sendStatus(200);
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
added 58 packages, and audited 59 packages in 579ms
8 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
angelayu@Angelas-MacBook-Pro 3.3 Postman % nodemon index.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Server started on port 3000


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
GET localhost:3000
GET Untitled Request
GET Untitled Request
GET Untitled Request
GET Untitled Request
No Environment
Collections
Environments
History
localhost:3000
Save
GET
localhost:3000
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 17 ms
Size: 246 B
Save as Example
Pretty
Raw
Preview
Visualize
HTML
1 <h1>Home Page</h1>


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
GET localhost:3000
POST localhost:3000/re
GET Untitled Request
GET Untitled Request
GET Untitled Request
No Environment
Collections
Environments
History
localhost:3000/register
Save
POST
localhost:3000/register
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
Key
Value
Description
Bulk Edit
name
Angela
Key
Value
Description
Body
Cookies
Headers (7)
Test Results
Status: 201 Created
Time: 11 ms
Size: 239 B
Save as Example
Pretty
Raw
Preview
Visualize
Text
1 Created
201 Created
The request has been fulfilled and resulted in a new resource being created.


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
GET localhost:3000
POST localhost:3000/re
PUT localhost:3000/use
GET Untitled Request
GET Untitled Request
No Environment
Collections
Environments
History
localhost:3000/user/angela
Save
PUT
localhost:3000/user/angela
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
Key
Value
Description
Bulk Edit
email
angela@gmail.com
Key
Value
Description
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 3 ms
Size: 229 B
Save as Example
Pretty
Raw
Preview
Visualize
Text
1 OK

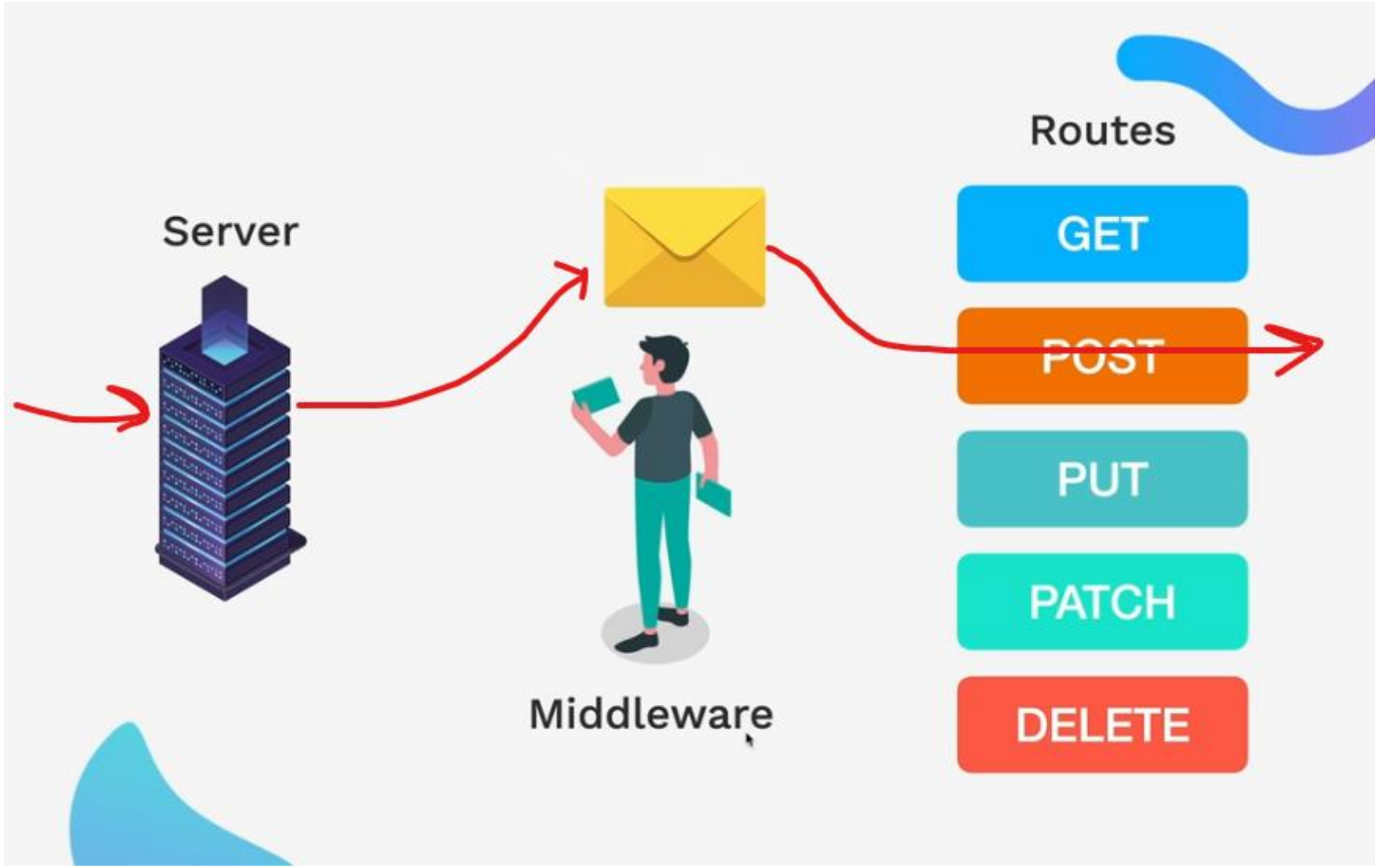

Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
GET localhost:3000
POST localhost:3000/re
PUT localhost:3000/use
PATCH localhost:3000/u
GET Untitled Request
No Environment
Collections
Environments
History
localhost:3000/user/angela?fName=Angla
Save
PATCH
localhost:3000/user/angela?fName=Angla
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
fName
Angla
Key
Value
Description
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 6 ms
Size: 229 B
Save as Example
Pretty
Raw
Preview
Visualize
Text
1 OK


Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
GET localhost:3000
POST localhost:3000/re
PUT localhost:3000/use
PATCH localhost:3000/u
DEL localhost:3000/use
No Environment
Collections
Environments
History
localhost:3000/user/angela
Save
DELETE
localhost:3000/user/angela
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 5 ms
Size: 229 B
Save as Example
Pretty
Raw
Preview
Visualize
Text
1 OK
200 OK
Standard response for successful HTTP requests. The actual response will depend on the request method used. In a GET request, the response will contain an entity corresponding to the requested resource. In a POST request the response will contain an entity describing or containing the result of the action.

Express Middleware

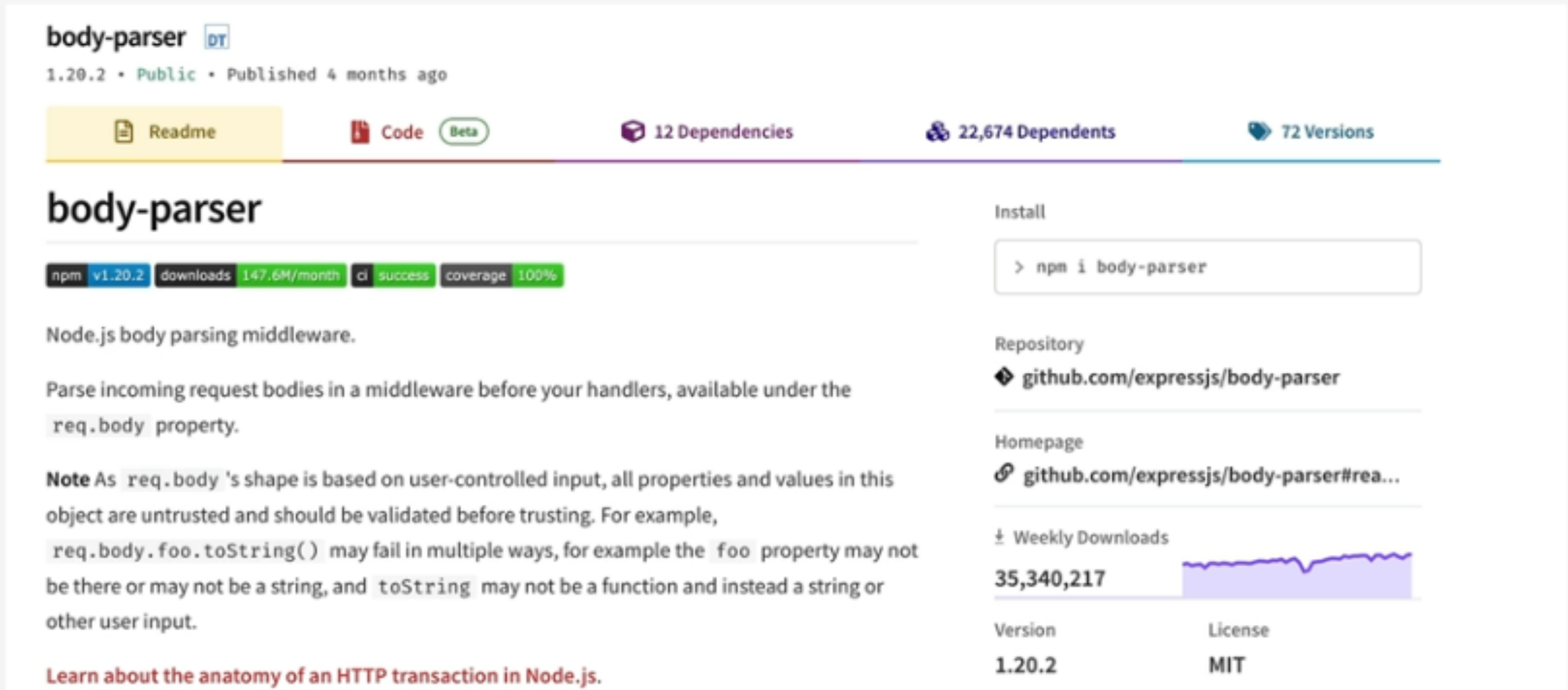
The man in the middle. Using body-parser together with HTML forms



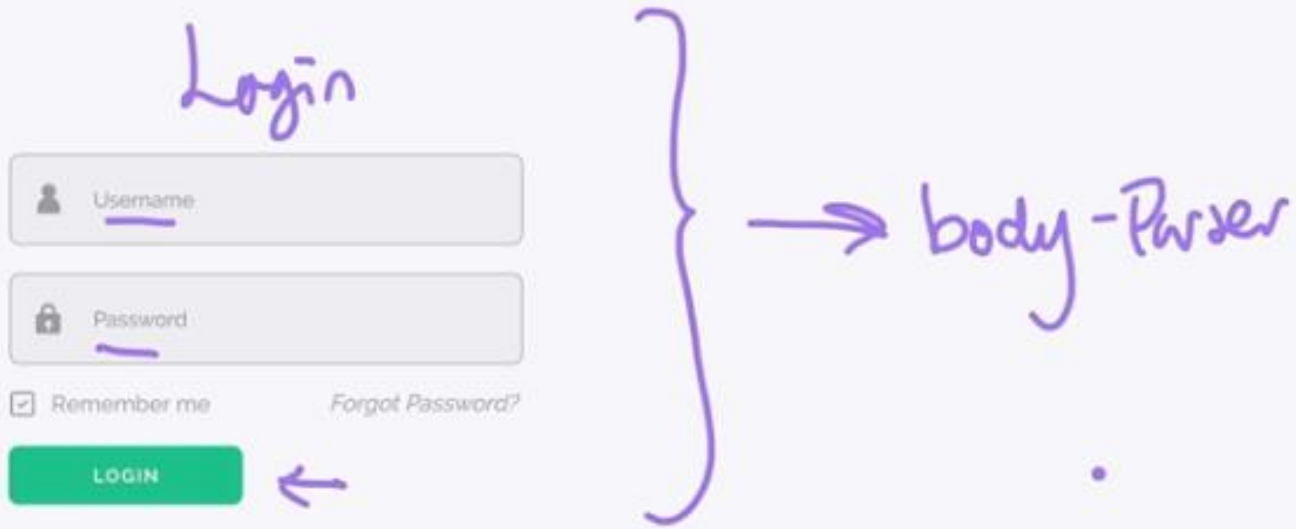

Routes
Server
GET
POST
PUT
PATCH
DELETE
Middleware

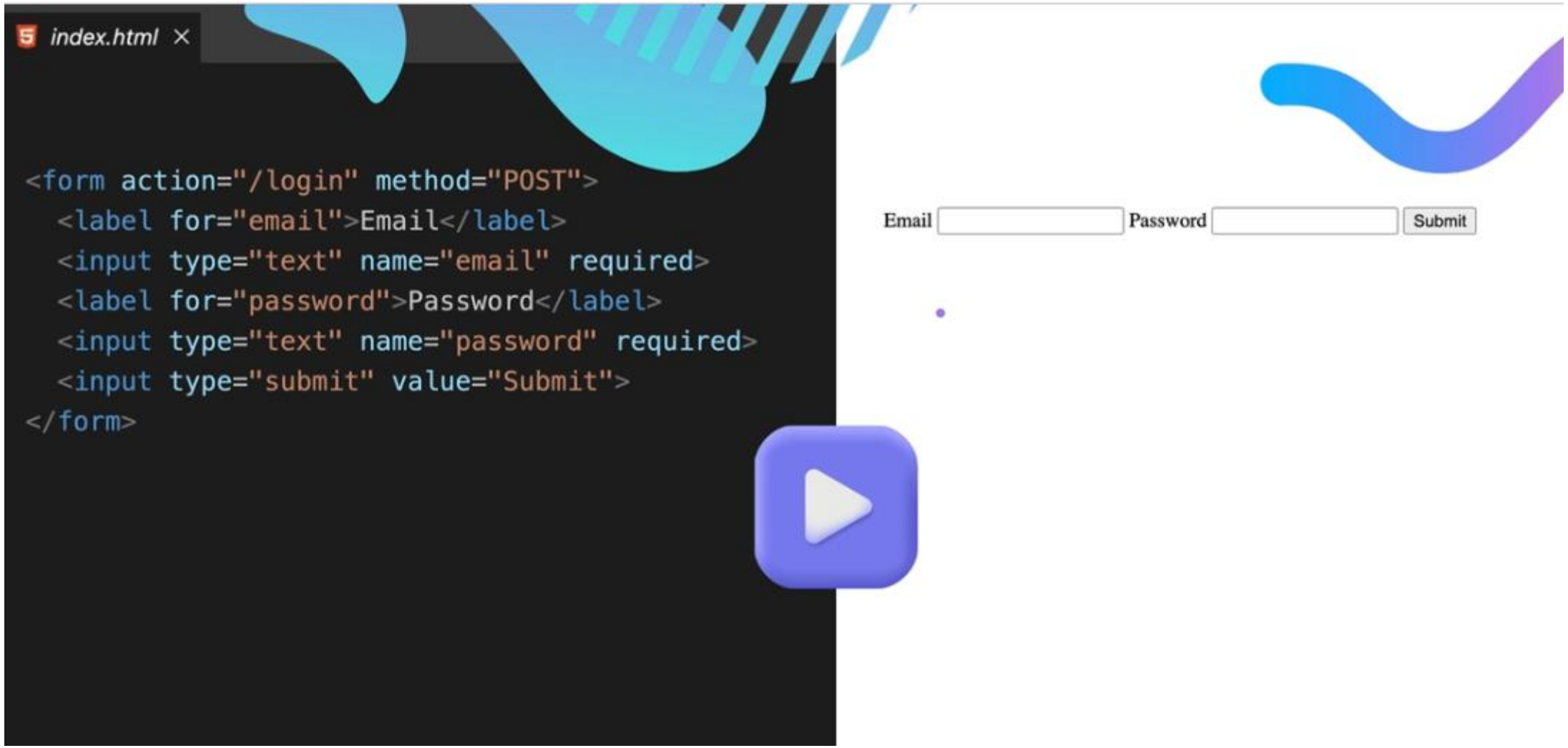

Middleware

Body Parser


body-parser
DT
1.20.2 • Public • Published 4 months ago
Readme
Code
Beta
12 Dependencies
22,674 Dependents
72 Versions
body-parser
npm v1.20.2
downloads 147.6M/month
ci success
coverage 100%
Node.js body parsing middleware.
Parse incoming request bodies in a middleware before your handlers, available under the req.body property.
Note As req.body 's shape is based on user-controlled input, all properties and values in this object are untrusted and should be validated before trusting. For example, req.body.foo.toString() may fail in multiple ways, for example the foo property may not be there or may not be a string, and toString may not be a function and instead a string or other user input.
Learn about the anatomy of an HTTP transaction in Node.js.
Install
> npm i body-parser
Repository
github.com/expressjs/body-parser
Homepage
github.com/expressjs/body-parser#rea...
Weekly Downloads
35,340,217
Version
1.20.2
License
MIT

HTML Forms


Login
Username
Password
Remember me
Forgot Password?
LOGIN
body-Parser

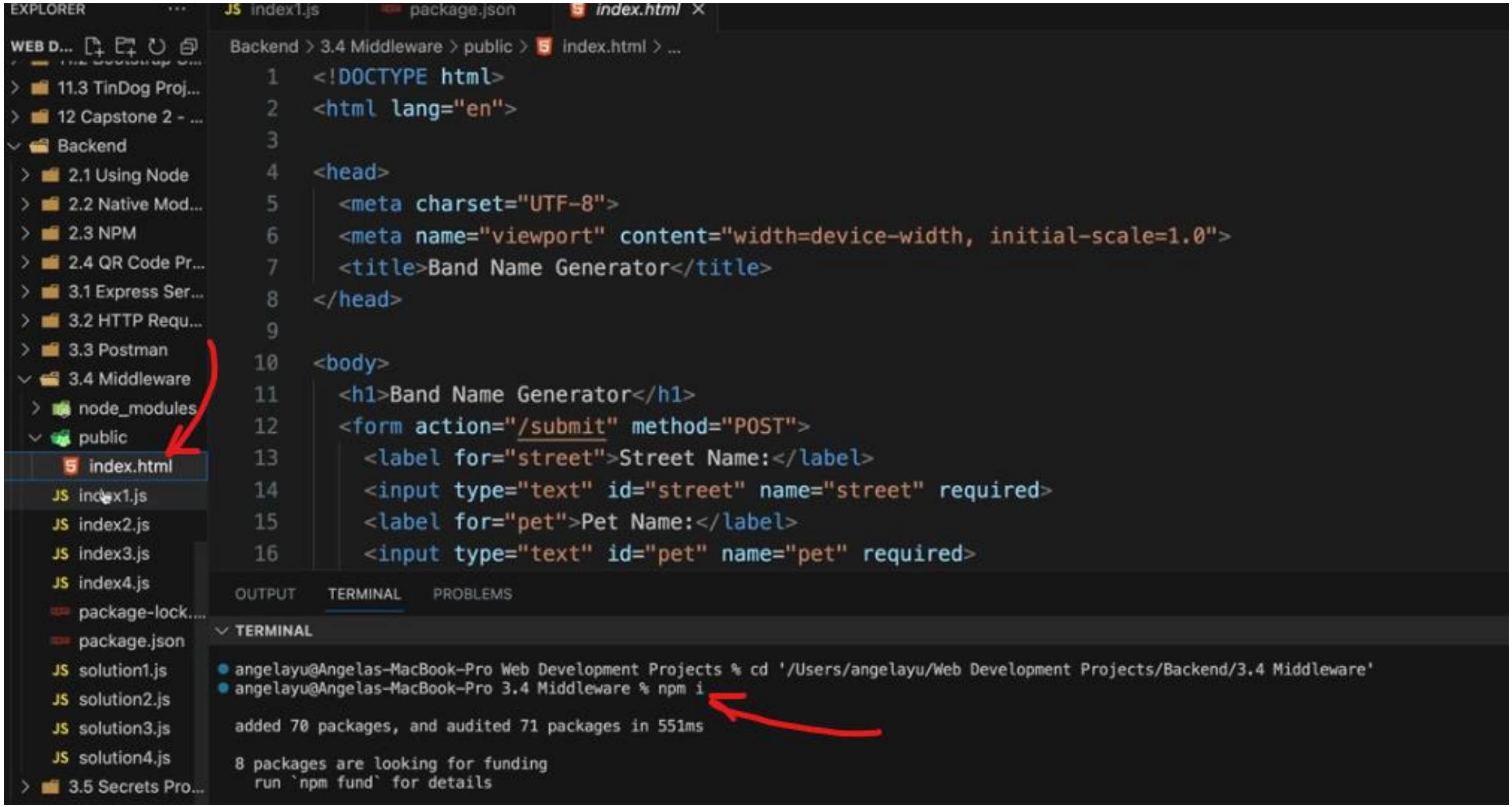

index.html
<form action="/login" method="POST">
<label for="email">Email</label>
<input type="text" name="email" required>
<label for="password">Password</label>
<input type="text" name="password" required>
<input type="submit" value="Submit">
</form>
Email
Password
Submit

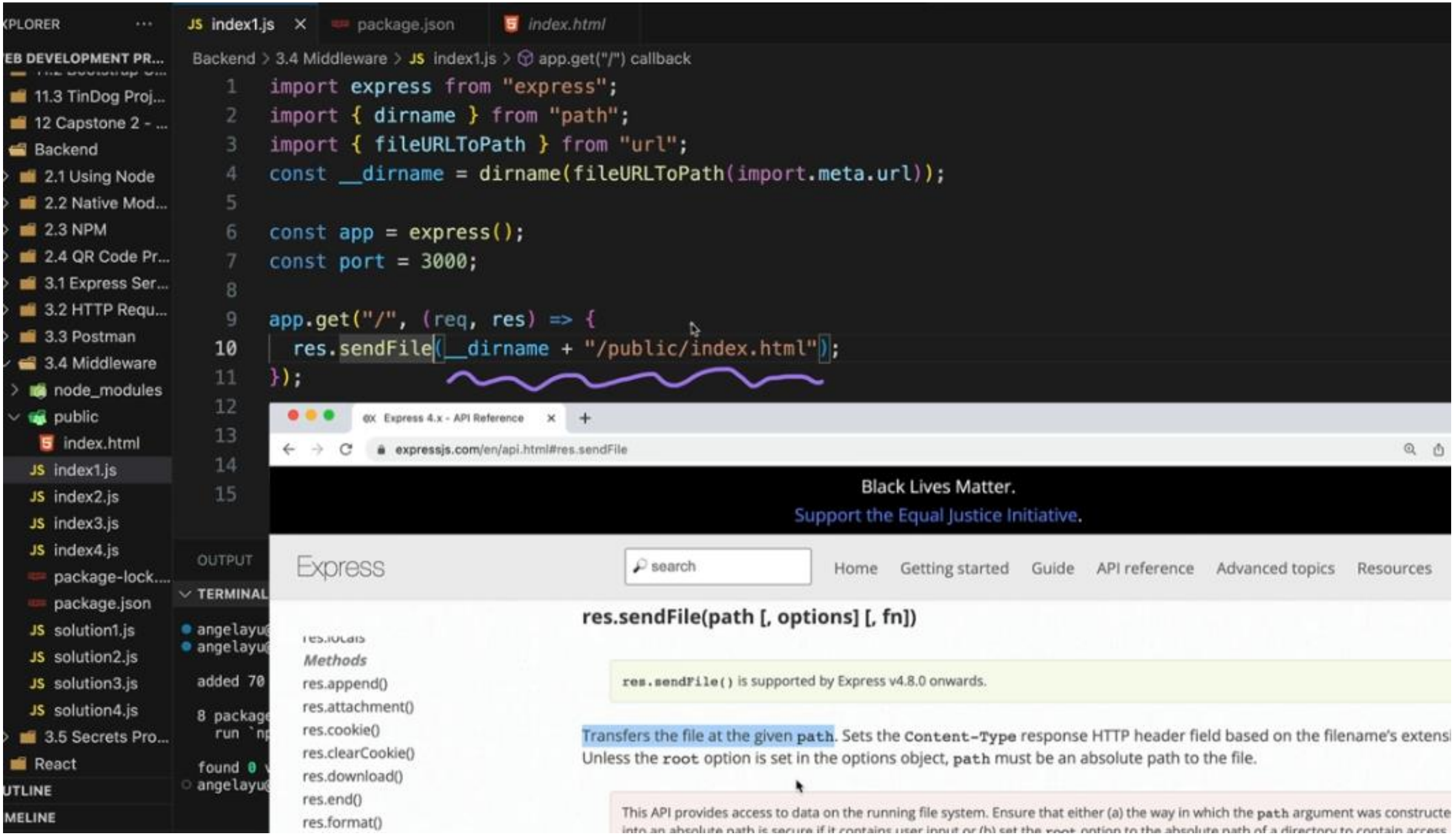
Exercise

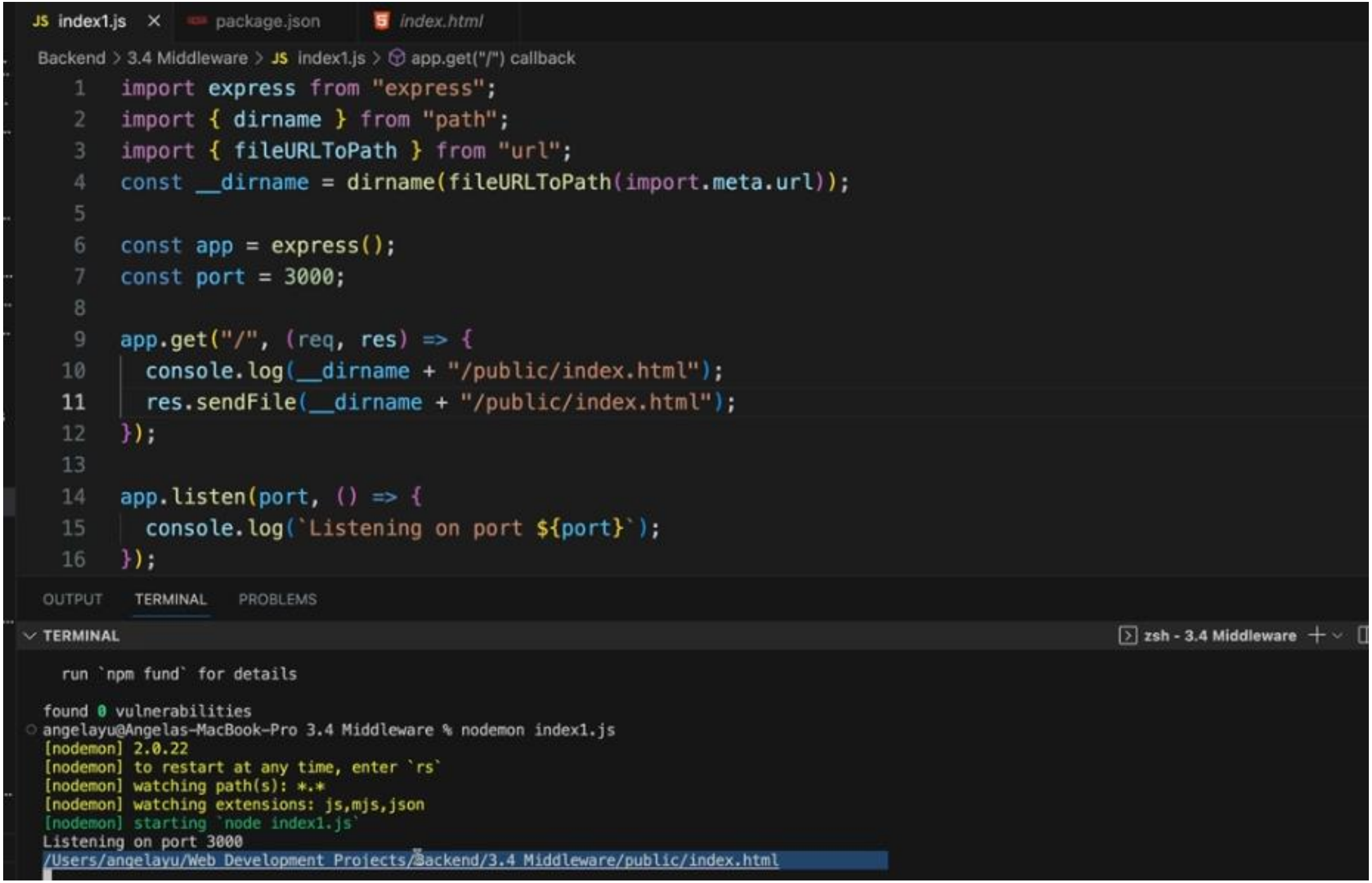
Try an Exercise

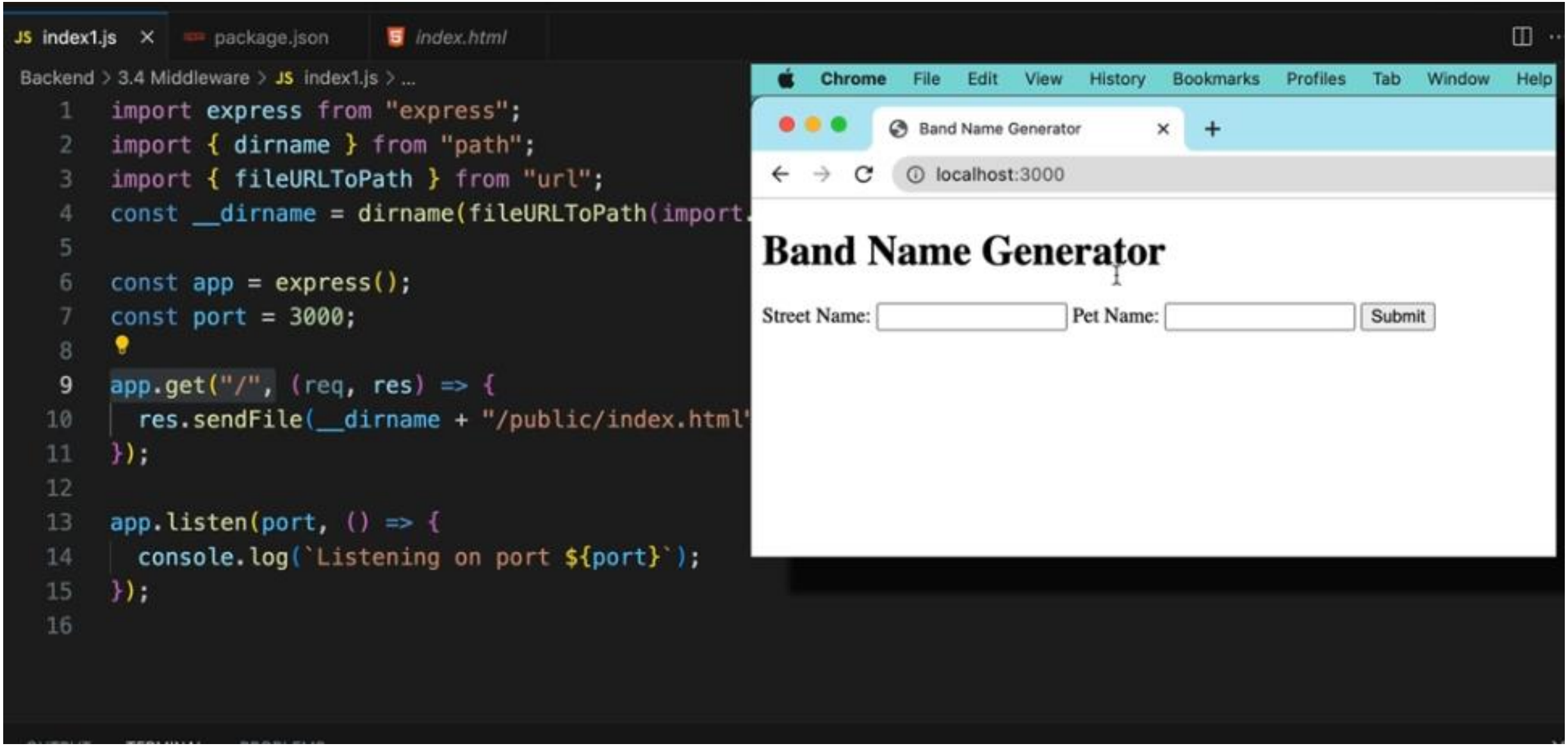


3.4 Middleware.zip

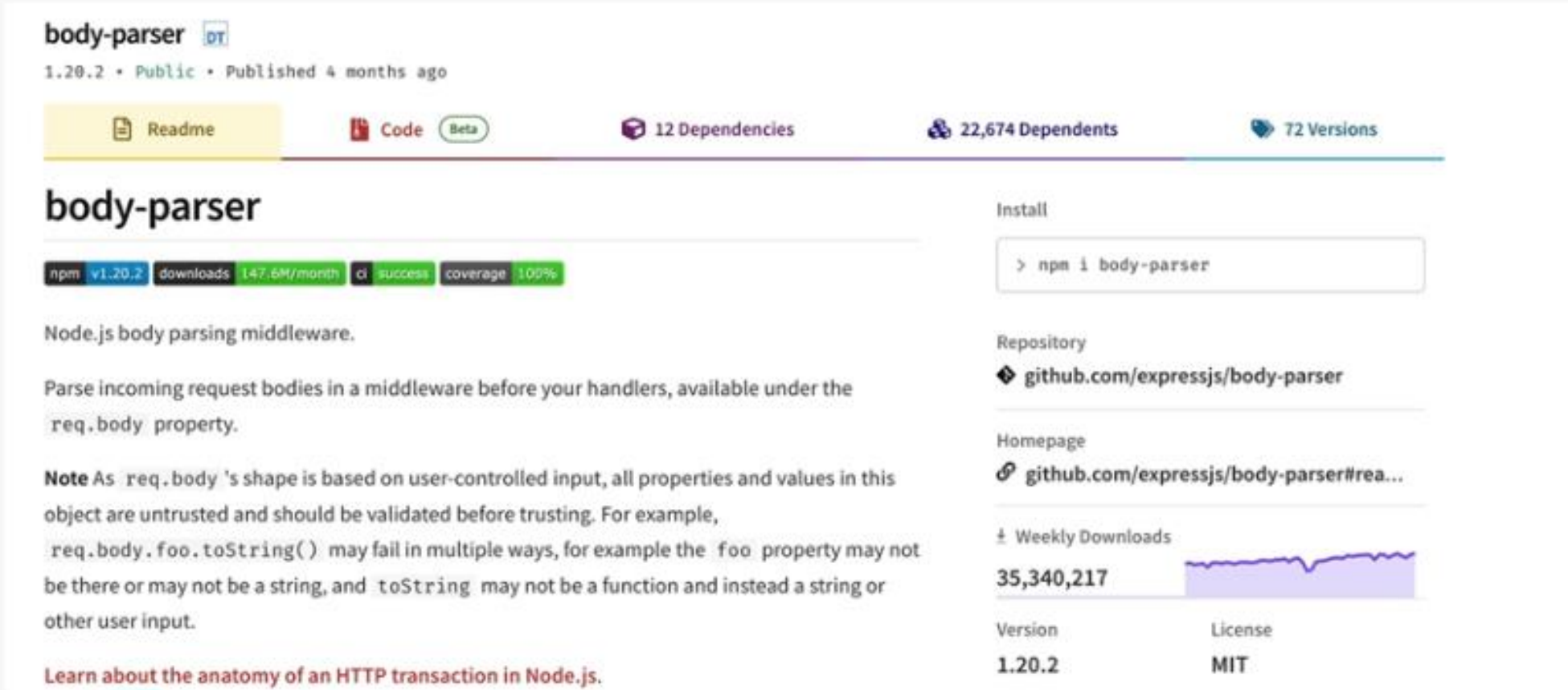

EXPLORER
WEB D...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
node_modules
public
index.html
index1.js
index2.js
index3.js
index4.js
package-lock....
package.json
solution1.js
solution2.js
solution3.js
solution4.js
3.5 Secrets Pro...
index1.js
package.json
index.html
Backend > 3.4 Middleware > public > index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5 <meta charset="UTF-8">
6 <meta name="viewport" content="width=device-width, initial-scale=1.0">
7 <title>Band Name Generator</title>
8 </head>
9
10 <body>
11 <h1>Band Name Generator</h1>
12 <form action="/submit" method="POST">
13 <label for="street">Street Name:</label>
14 <input type="text" id="street" name="street" required>
15 <label for="pet">Pet Name:</label>
16 <input type="text" id="pet" name="pet" required>
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro Web Development Projects % cd '/Users/angelayu/Web Development Projects/Backend/3.4 Middleware'
angelayu@Angelas-MacBook-Pro 3.4 Middleware % npm i
added 70 packages, and audited 71 packages in 551ms
8 packages are looking for funding
run `npm fund` for details


index1.js
package.json
index.html
Backend > 3.4 Middleware > index1.js > app.get("/") callback
import express from "express";
import { dirname } from "path";
import { fileURLToPath } from "url";
const __dirname = dirname(fileURLToPath(import.meta.url));
const app = express();
const port = 3000;
app.get("/", (req, res) => {
res.sendFile(__dirname + "/public/index.html");
});
Express 4.x - API Reference
expressjs.com/en/api.html#res.sendFile
Black Lives Matter.
Support the Equal Justice Initiative.
Express
search
Home
Getting started
Guide
API reference
Advanced topics
Resources
res.sendFile(path [, options] [, fn])
Methods
res.append()
res.attachment()
res.cookie()
res.clearCookie()
res.download()
res.end()
res.format()
res.sendFile() is supported by Express v4.8.0 onwards.
Transfers the file at the given path. Sets the Content-Type response HTTP header field based on the filename's extens
Unless the root option is set in the options object, path must be an absolute path to the file.
This API provides access to data on the running file system. Ensure that either (a) the way in which the path argument was constructe
OUTPUT
TERMINAL
added 70
found 0


index1.js
package.json
index.html
Backend > 3.4 Middleware > index1.js > app.get("/") callback
1 import express from "express";
2 import { dirname } from "path";
3 import { fileURLToPath } from "url";
4 const __dirname = dirname(fileURLToPath(import.meta.url));
5
6 const app = express();
7 const port = 3000;
8
9 app.get("/", (req, res) => {
10 console.log(__dirname + "/public/index.html");
11 res.sendFile(__dirname + "/public/index.html");
12 });
13
14 app.listen(port, () => {
15 console.log(`Listening on port ${port}`);
16 });
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
zsh - 3.4 Middleware
run `npm fund` for details
found 0 vulnerabilities
angelayu@Angelas-MacBook-Pro 3.4 Middleware % nodemon index1.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index1.js`
Listening on port 3000
/Users/angelayu/Web Development Projects/Backend/3.4 Middleware/public/index.html


index1.js
package.json
index.html
Backend > 3.4 Middleware > index1.js > ...
1 import express from "express";
2 import { dirname } from "path";
3 import { fileURLToPath } from "url";
4 const __dirname = dirname(fileURLToPath(import.
5
6 const app = express();
7 const port = 3000;
8
9 app.get("/", (req, res) => {
10 res.sendFile(__dirname + "/public/index.html"
11 });
12
13 app.listen(port, () => {
14 console.log(`Listening on port ${port}`);
15 });
16
Chrome File Edit View History Bookmarks Profiles Tab Window Help
Band Name Generator
localhost:3000
Band Name Generator
Street Name:
Pet Name:
Submit

Body Parser


body-parser
DT
1.20.2 • Public • Published 4 months ago
Readme
Code
Beta
12 Dependencies
22,674 Dependents
72 Versions
body-parser
npm v1.20.2
downloads 147.6M/month
ci success
coverage 100%
Node.js body parsing middleware.
Parse incoming request bodies in a middleware before your handlers, available under the req.body property.
Note As req.body 's shape is based on user-controlled input, all properties and values in this object are untrusted and should be validated before trusting. For example, req.body.foo.toString() may fail in multiple ways, for example the foo property may not be there or may not be a string, and toString may not be a function and instead a string or other user input.
Learn about the anatomy of an HTTP transaction in Node.js.
Install
> npm i body-parser
Repository
github.com/expressjs/body-parser
Homepage
github.com/expressjs/body-parser#rea...
Weekly Downloads
35,340,217
Version
1.20.2
License
MIT

JS index.js

```
import express from "express";
import bodyParser from "body-parser";

const app = express();
const port = 3000;

app.use(bodyParser.urlencoded({ extended: true }));

app.post("/submit", (req, res) => {
  console.log(req.body);
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```



JS index.js

```
import express from "express";
import bodyParser from "body-parser";

const app = express();
const port = 3000;

app.use(bodyParser.urlencoded({ extended: true }));

app.post("/submit", (req, res) => {
  console.log(req.body);
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

Exercise

Band Name Generator

Street Name: Pet Name: Submit

1. Use npm to install the `body-parser` module
2. Run `index1.js` with nodemon.
3. Import the `body-parser` module.
4. Mount the middleware using the Express `.use()` method.
5. Specify `.urlencoded({extended: true})` to create a body for URL-encoded requests like our form submission.
6. Write a `.post("/submit")` handler where you `console.log()` the form contents when the user clicks the submit button.

- npm i body-parser

```
15  });
16
```

OUTPUT TERMINAL PROBLEMS

TERMINAL

```
Listening on port 3000
/Users/angelayu/Web Development Projects/Backend/3.4 Middleware/public/index.html
[nodemon] restarting due to changes...
[nodemon] starting `node index1.js`
Listening on port 3000
^C
angelayu@Angelas-MacBook-Pro 3.4 Middleware % npm i body-parser
(########) ⠸ idealTree: timing idealTree Completed in 166ms
```

Ln 15, Co

index1.js × index.html

Backend > 3.4 Middleware > JS index1.js > ...

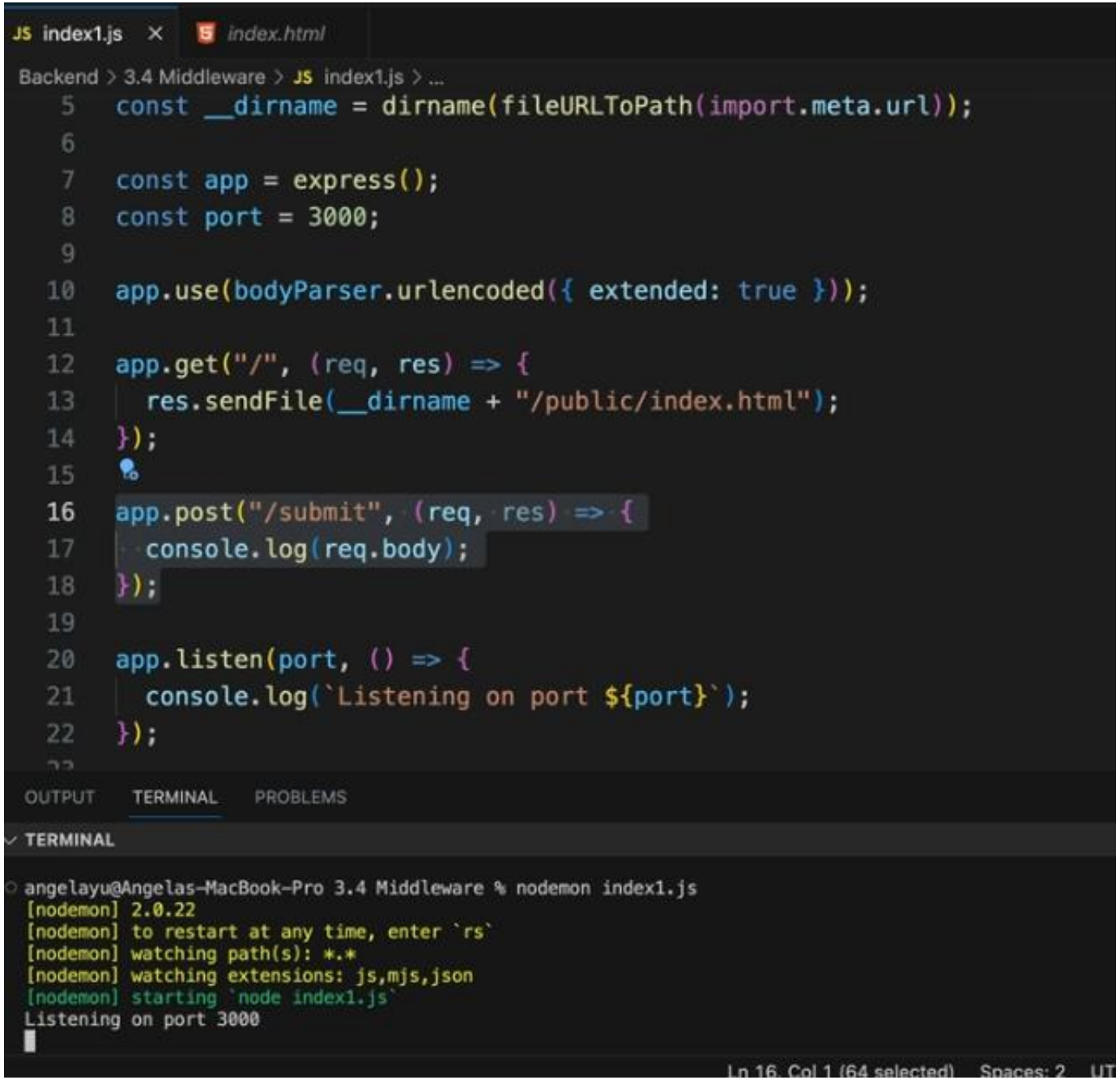
```
1  import express from "express";
2  import bodyParser from "body-parser";
3  import { dirname } from "path";
4  import { fileURLToPath } from "url";
5  const __dirname = dirname(fileURLToPath(import.meta.url));
6
7  const app = express();
8  const port = 3000;
9
10 app.use(bodyParser.urlencoded({ extended: true }));
11
12 app.get("/", (req, res) => {
13   res.sendFile(__dirname + "/public/index.html");
14 });
15
16 app.listen(port, () => {
17   console.log(`Listening on port ${port}`);
18 });
```

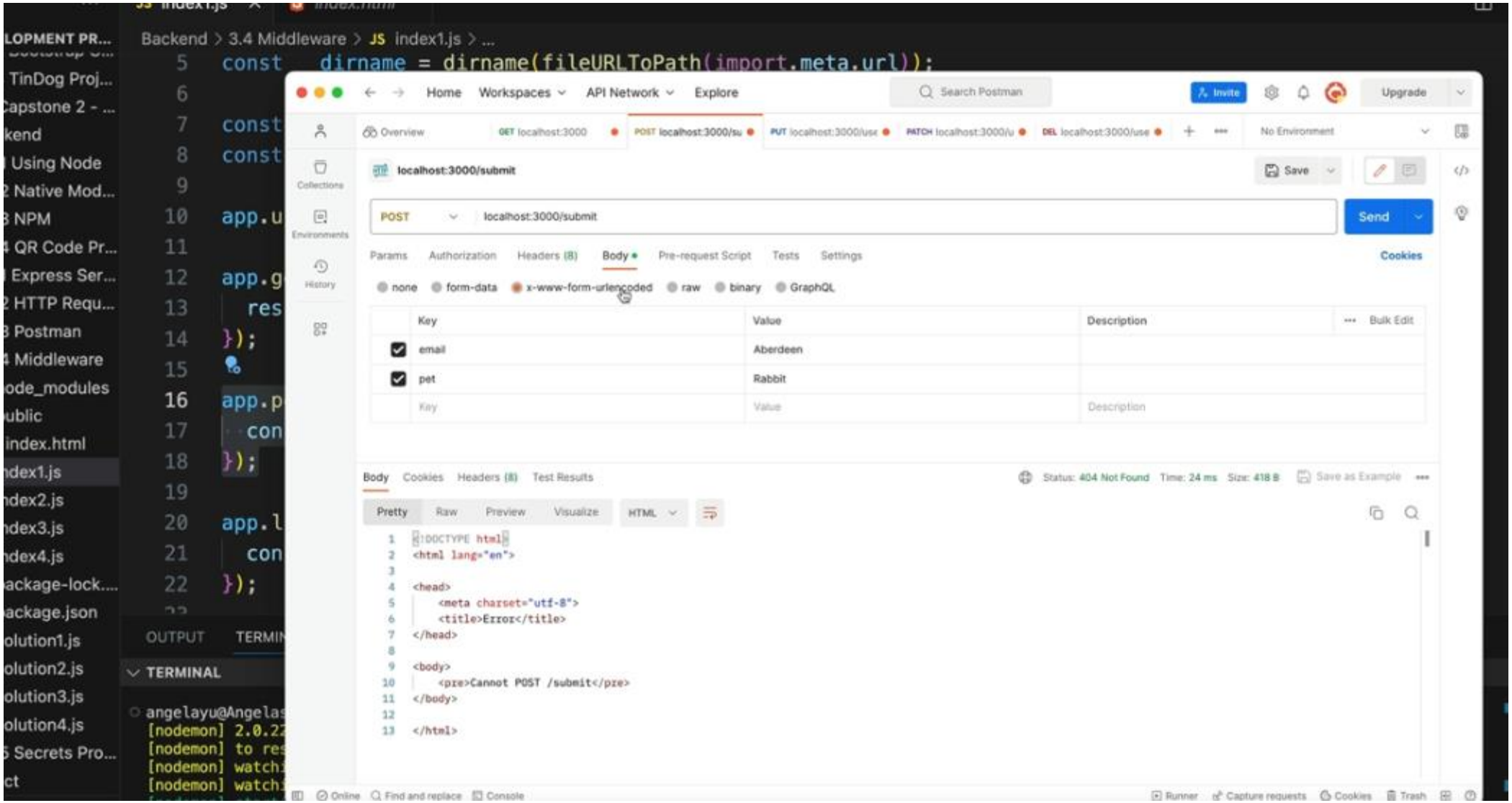
OUTPUT TERMINAL PROBLEMS

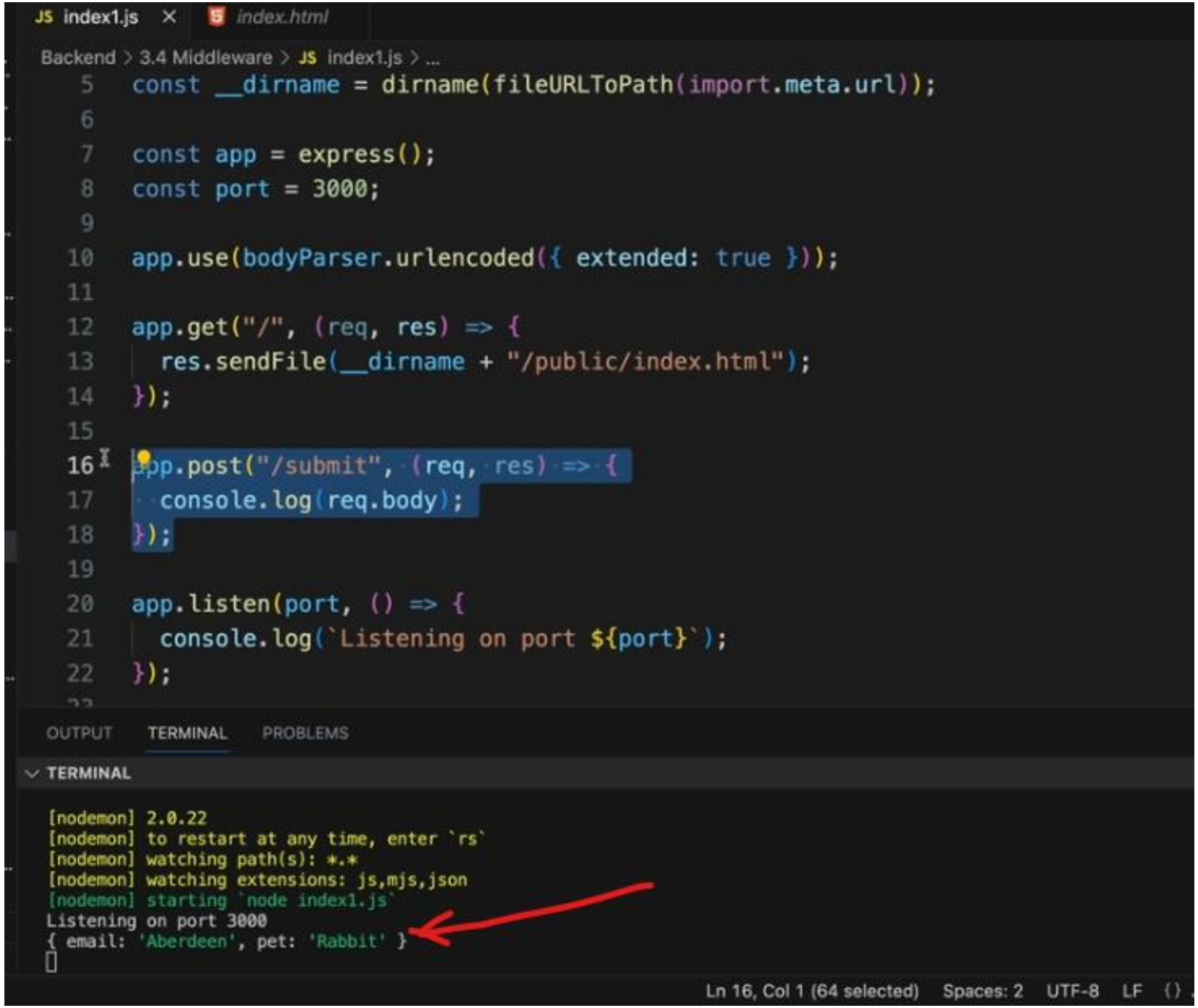
TERMINAL

```
removed 5 packages, and audited 66 packages in 565ms

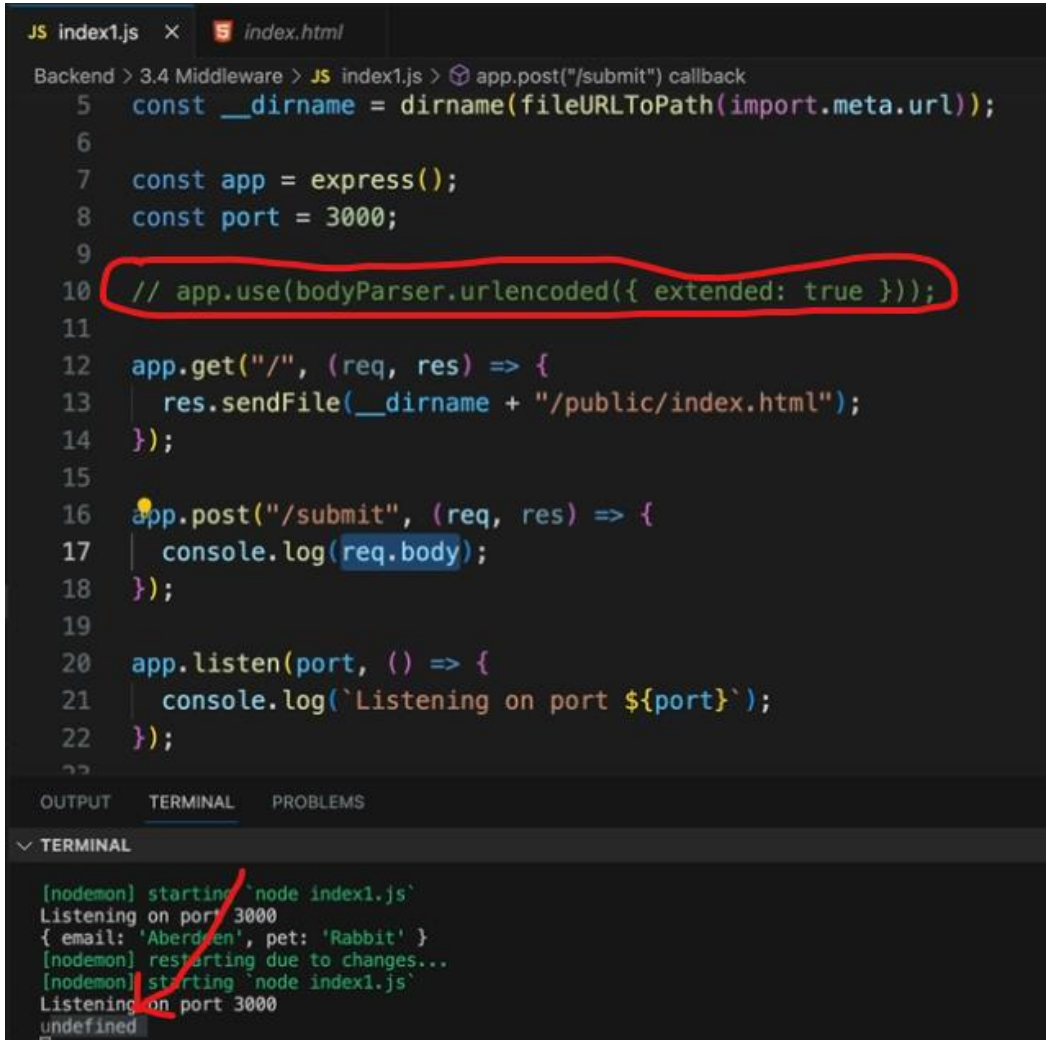
8 packages are looking for funding
```

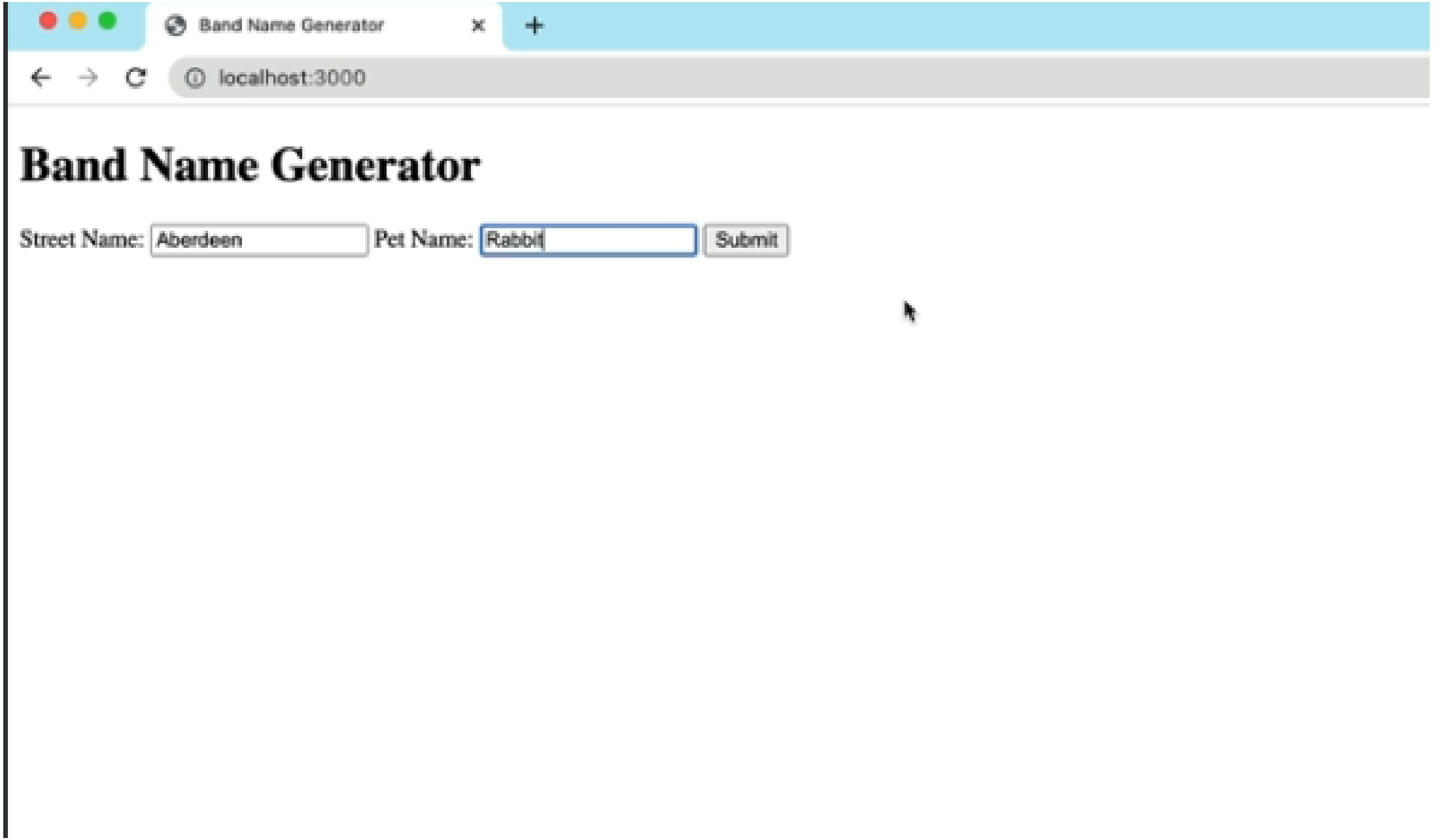

index1.js
index.html
Backend > 3.4 Middleware > index1.js > ...
5 const __dirname = dirname(fileURLToPath(import.meta.url));
6
7 const app = express();
8 const port = 3000;
9
10 app.use(bodyParser.urlencoded({ extended: true }));
11
12 app.get("/", (req, res) => {
13 res.sendFile(__dirname + "/public/index.html");
14 });
15
16 app.post("/submit", (req, res) => {
17 console.log(req.body);
18 });
19
20 app.listen(port, () => {
21 console.log(`Listening on port ${port}`);
22 });
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro 3.4 Middleware % nodemon index1.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index1.js`
Listening on port 3000
Ln 16, Col 1 (64 selected)
Spaces: 2

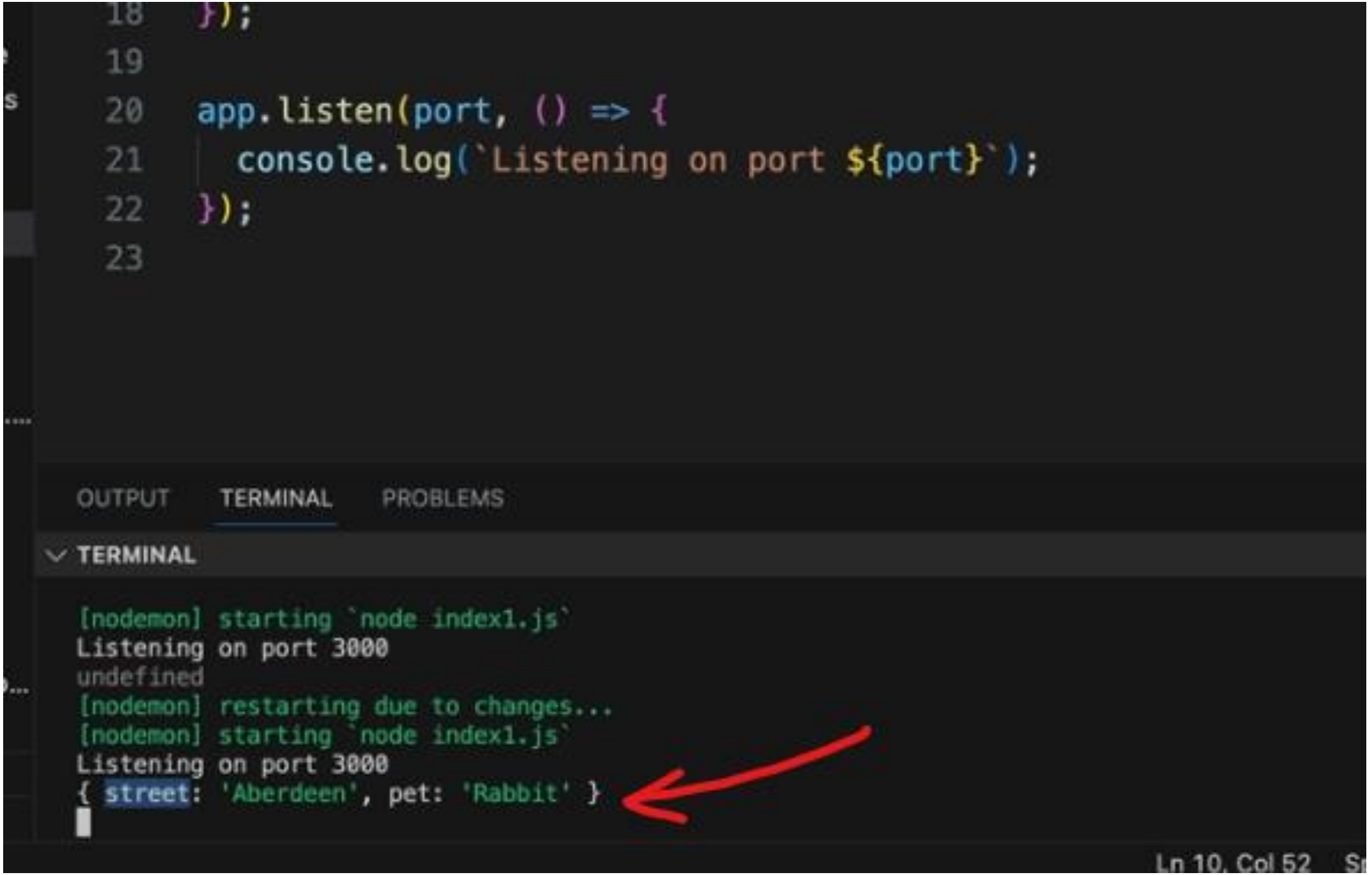

Backend > 3.4 Middleware > JS index1.js > ...
const __dirname = dirname(fileURLToPath(import.meta.url));
Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
localhost:3000/submit
POST
localhost:3000/submit
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
Key
Value
Description
Bulk Edit
email
Aberdeen
pet
Rabbit
Body
Cookies
Headers (8)
Test Results
Status: 404 Not Found
Time: 24 ms
Size: 418 B
Save as Example
Pretty
Raw
Preview
Visualize
HTML
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot POST /submit</pre>
</body>
</html>
OUTPUT
TERMINAL
[nodemon] 2.0.22
[nodemon] watching


index1.js
index.html
Backend > 3.4 Middleware > index1.js > ...
const __dirname = dirname(fileURLToPath(import.meta.url));
const app = express();
const port = 3000;
app.use(bodyParser.urlencoded({ extended: true }));
app.get("/", (req, res) => {
res.sendFile(__dirname + "/public/index.html");
});
app.post("/submit", (req, res) => {
console.log(req.body);
});
app.listen(port, () => {
console.log(`Listening on port ${port}`);
});
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index1.js`
Listening on port 3000
{ email: 'Aberdeen', pet: 'Rabbit' }
Ln 16, Col 1 (64 selected)
Spaces: 2
UTF-8
LF

Type to comment app.use…




Band Name Generator
localhost:3000
Band Name Generator
Street Name: Aberdeen
Pet Name: Rabbit
Submit


18 });
19
20 app.listen(port, () => {
21 console.log(`Listening on port ${port}`);
22 });
23
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
[nodemon] starting `node index1.js`
Listening on port 3000
undefined
[nodemon] restarting due to changes...
[nodemon] starting `node index1.js`
Listening on port 3000
{ street: 'Aberdeen', pet: 'Rabbit' }
Ln 10, Col 52

Custom Middleware

How to use logging middleware as well as how to create your own







Middleware



body-parser

Pre-processing

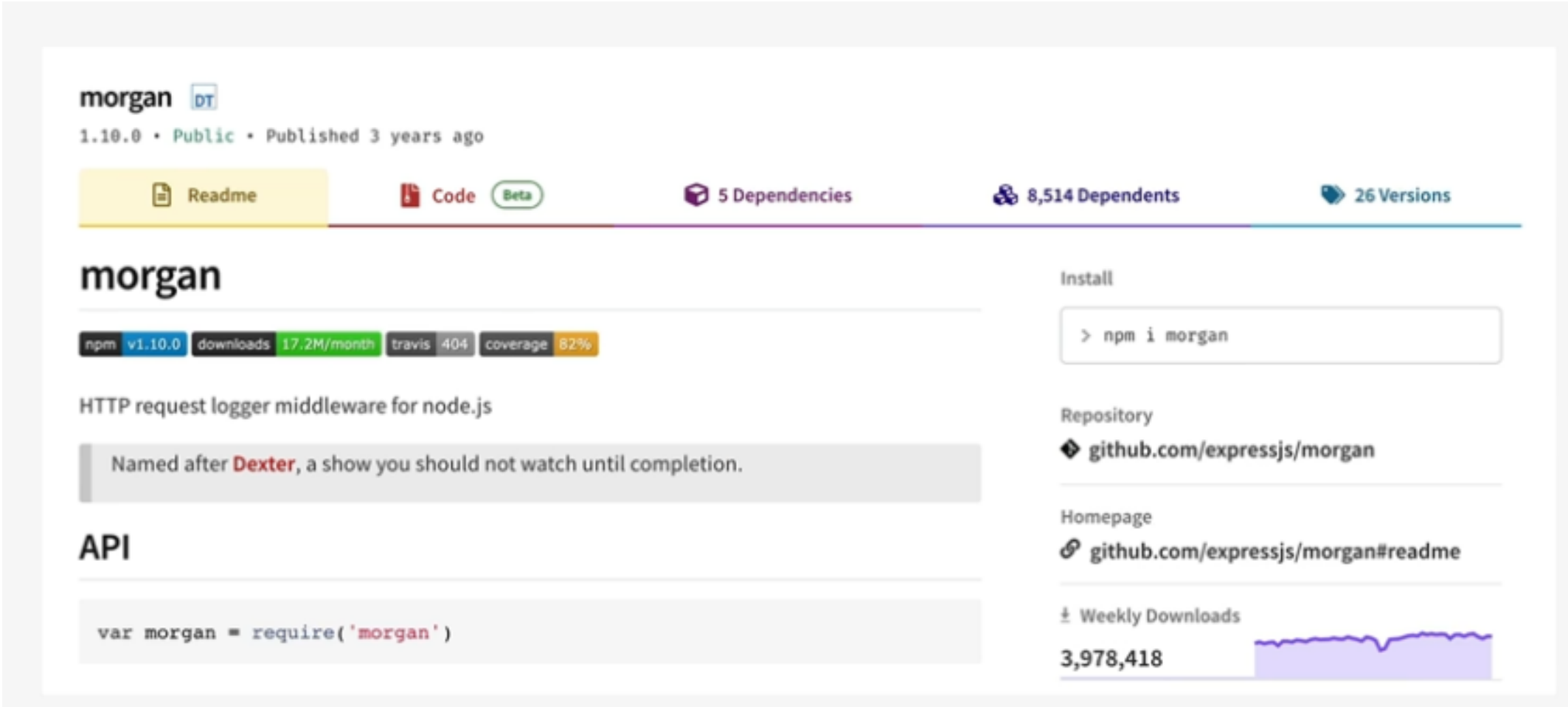


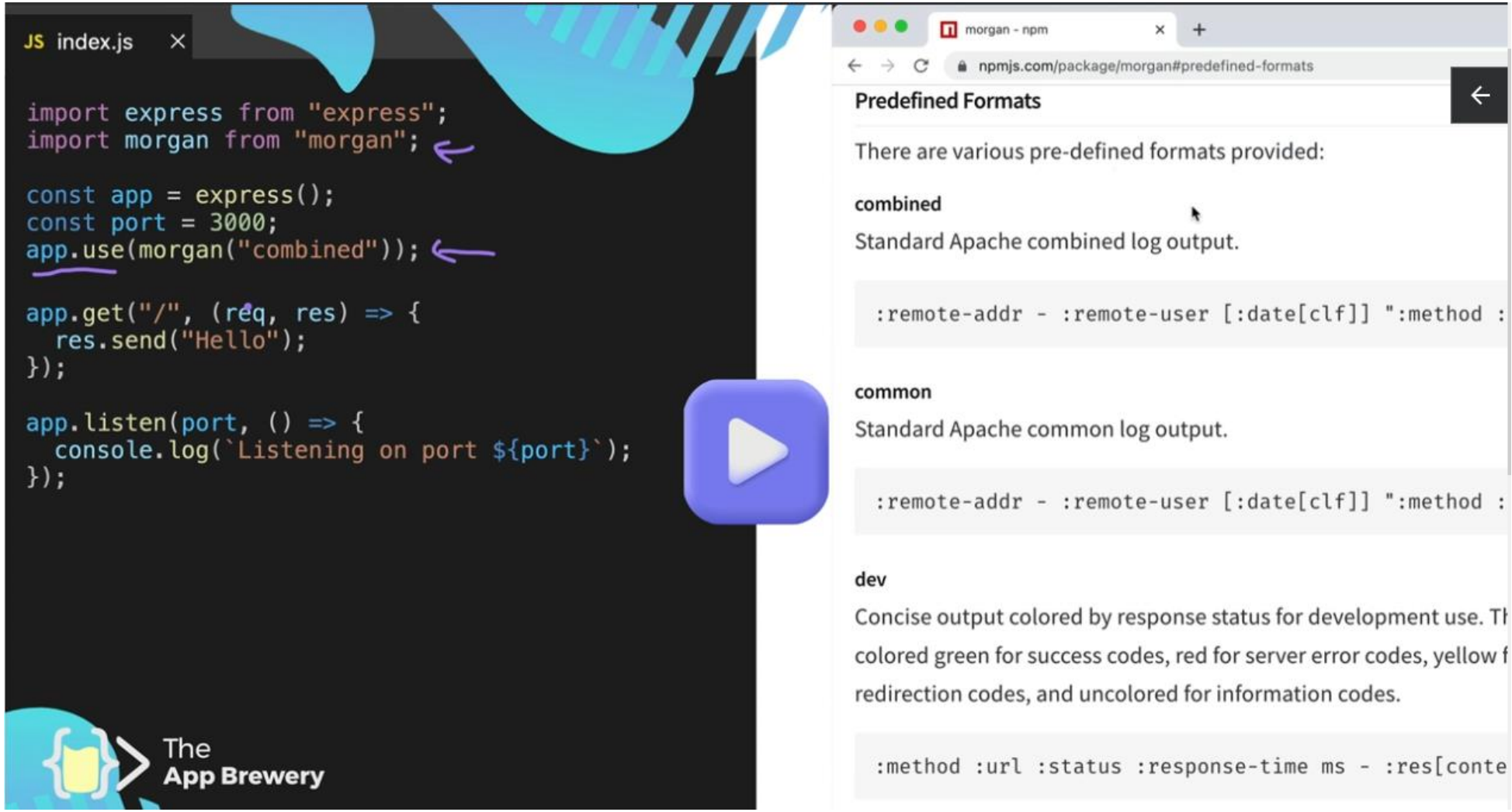
Auth

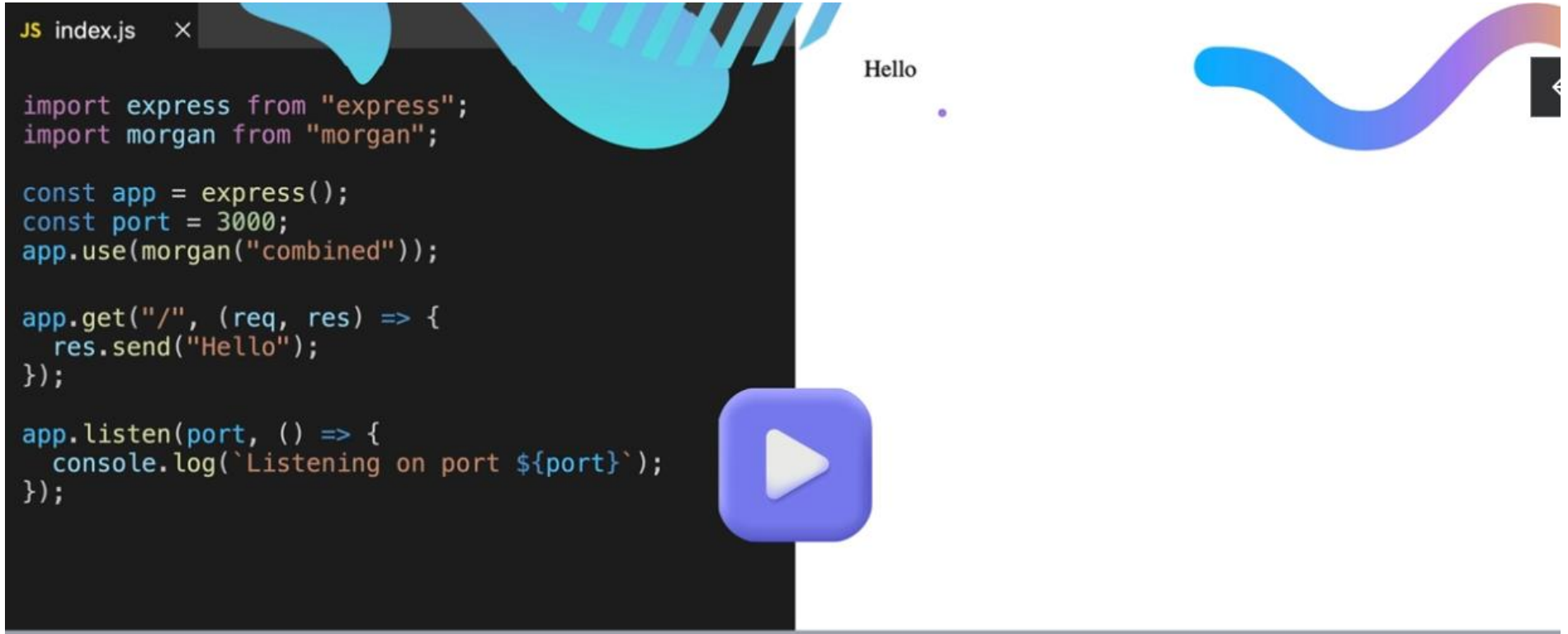


Error

Logger middleware




JS index.js
import express from "express";
import morgan from "morgan";
const app = express();
const port = 3000;
app.use(morgan("combined"));
app.get("/", (req, res) => {
res.send("Hello");
});
app.listen(port, () => {
console.log(`Listening on port ${port}`);
});
The
App Brewery
morgan - npm
npmjs.com/package/morgan#predefined-formats
Predefined Formats
There are various pre-defined formats provided:
combined
Standard Apache combined log output.
:remote-addr - :remote-user [:date[clf]] ":method :
common
Standard Apache common log output.
:remote-addr - :remote-user [:date[clf]] ":method :
dev
Concise output colored by response status for development use. Th
colored green for success codes, red for server error codes, yellow f
redirection codes, and uncolored for information codes.
:method :url :status :response-time ms - :res[conte


JS index.js
import express from "express";
import morgan from "morgan";
const app = express();
const port = 3000;
app.use(morgan("combined"));
app.get("/", (req, res) => {
res.send("Hello");
});
app.listen(port, () => {
console.log(`Listening on port ${port}`);
});
Hello
::1 - - [15/Jun/2023:09:56:13 +0000] "GET / HTTP/1.1" 200 5 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36"

Predefined Formats

There are various pre-defined formats provided:

combined

Standard Apache combined log output.

```
:remote-addr - :remote-user [:date[clf]] ":method :url HTTP/:http-version"
```

common

Standard Apache common log output.

```
:remote-addr - :remote-user [:date[clf]] ":method :url HTTP/:http-version"
```

dev

Concise output colored by response status for development use. The `:status` token will be colored green for success codes, red for server error codes, yellow for client error codes, cyan for redirection codes, and uncolored for information codes.

```
:method :url :status :response-time ms - :res[content-length]
```

short

Shorter than default, also including response time.

```
:remote-addr :remote-user :method :url HTTP/:http-version :status :res[cont
```

tiny

The minimal output.

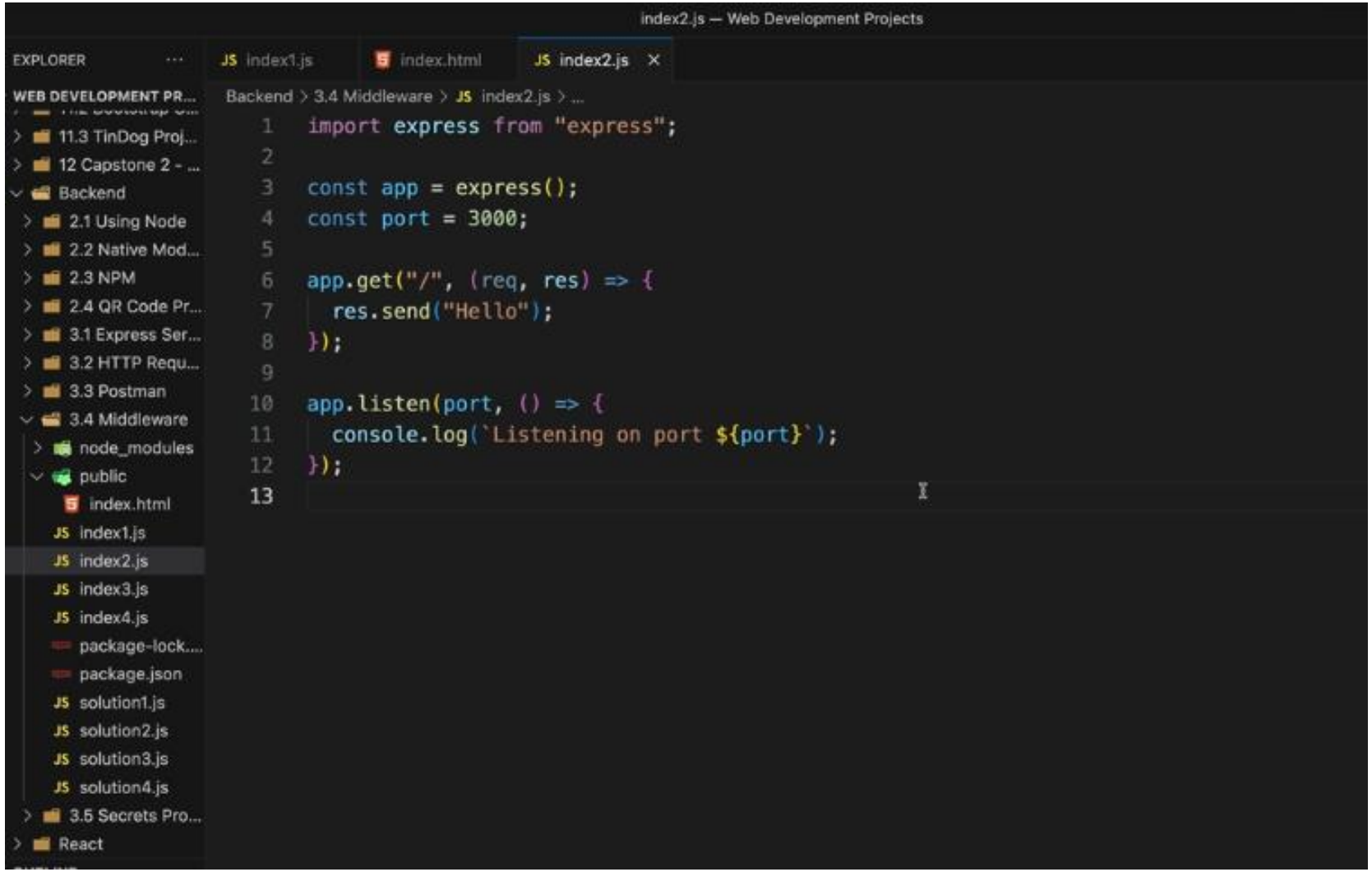
```
:method :url :status :res[content-length] - :response-time ms
```

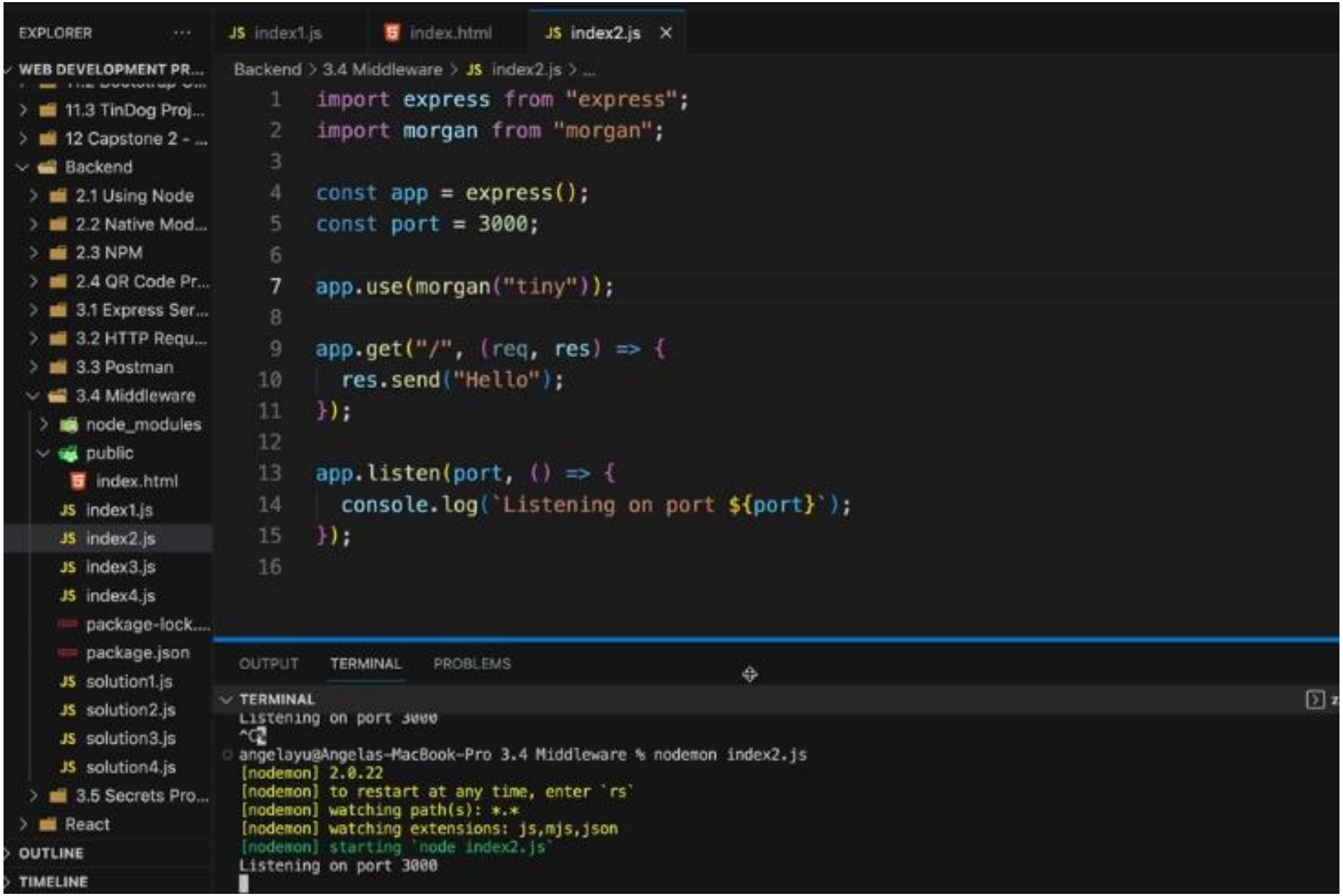
Tokens

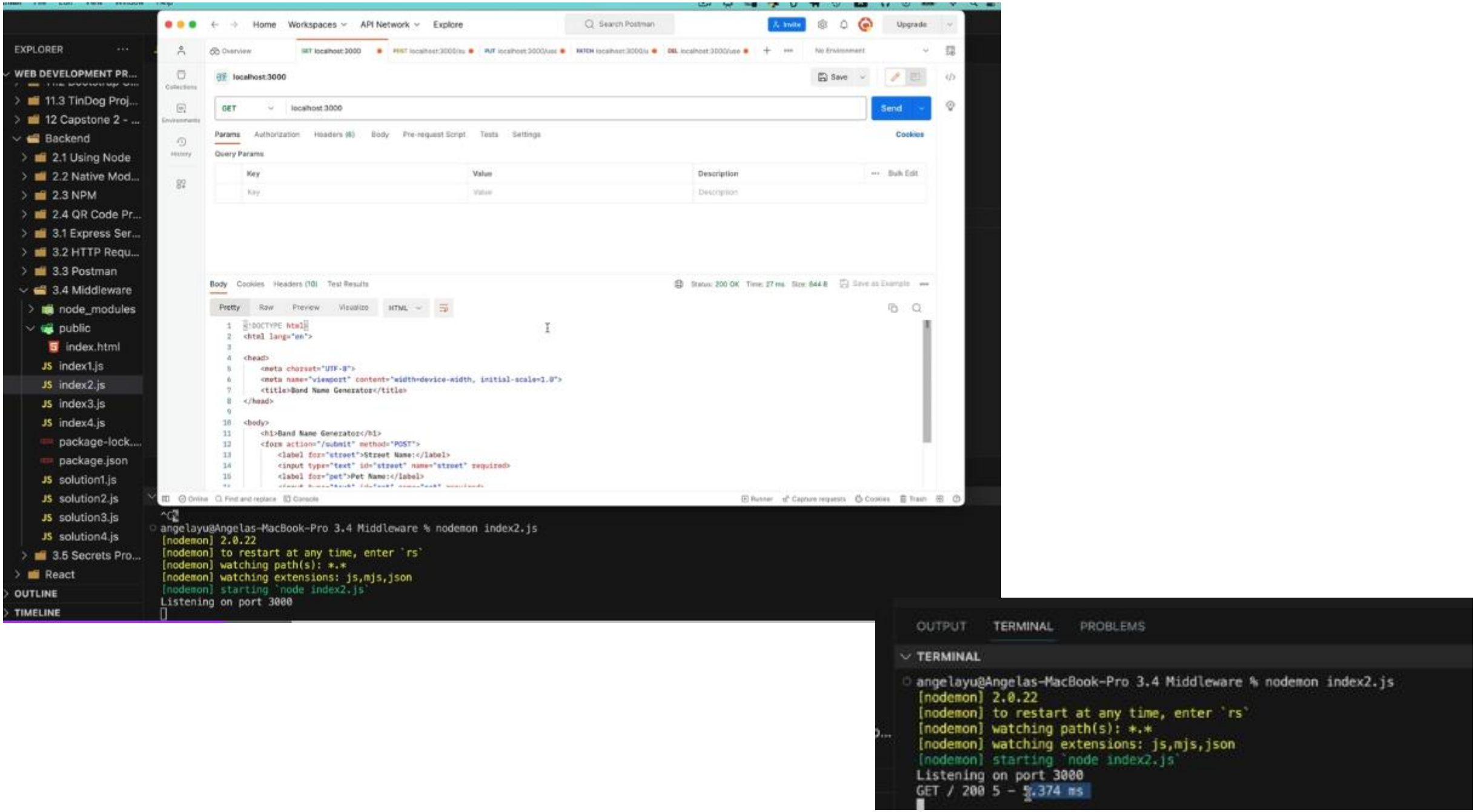
Creating new tokens

To define a token, simply invoke `morgan.token()` with the name and a callback function. This

Index2.js

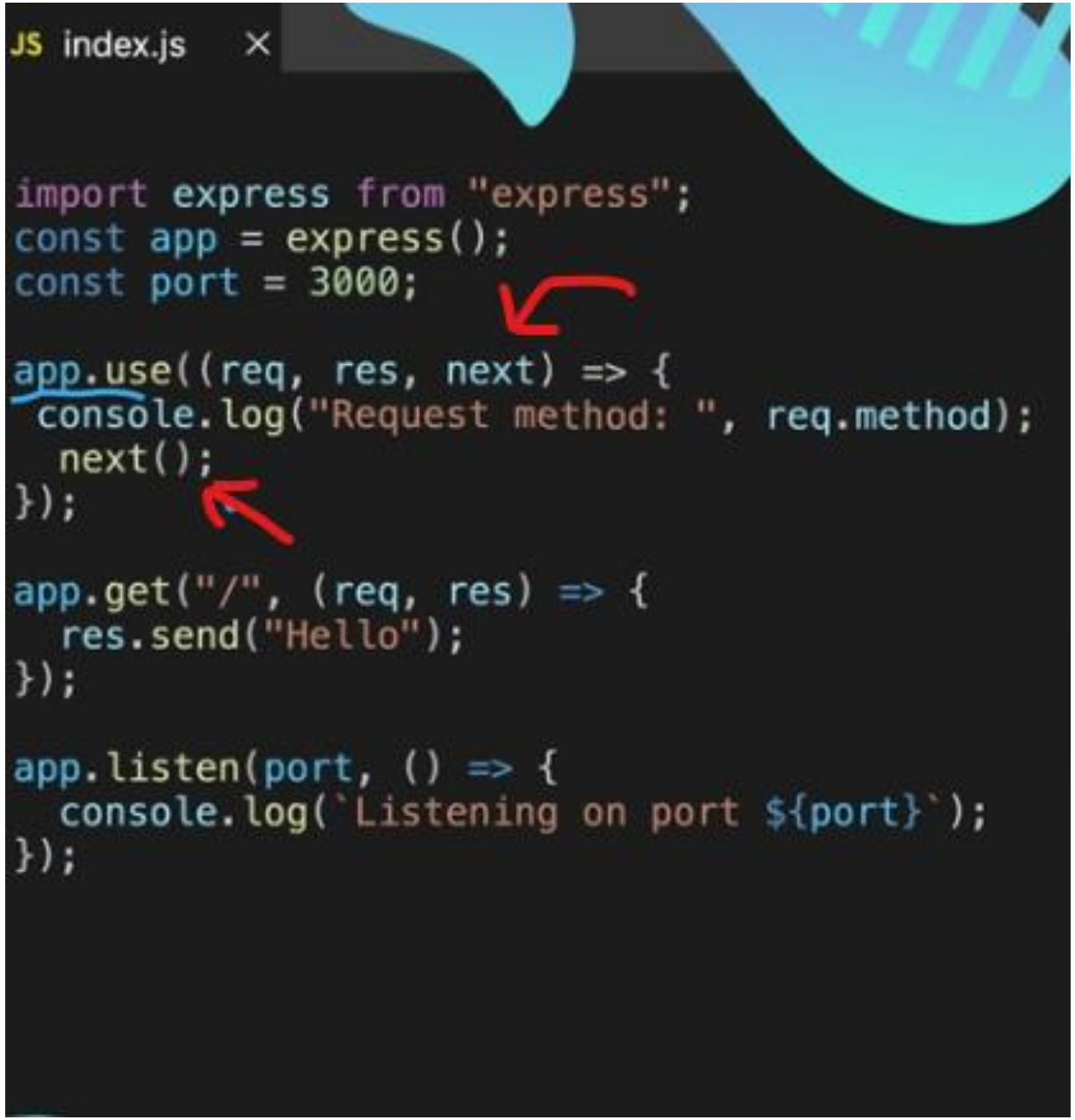

index2.js — Web Development Projects
EXPLORER
index1.js
index.html
index2.js
Backend > 3.4 Middleware > index2.js > ...
1 import express from "express";
2
3 const app = express();
4 const port = 3000;
5
6 app.get("/", (req, res) => {
7 res.send("Hello");
8 });
9
10 app.listen(port, () => {
11 console.log(`Listening on port ${port}`);
12 });
13
WEB DEVELOPMENT PR...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
node_modules
public
index.html
index1.js
index2.js
index3.js
index4.js
package-lock....
package.json
solution1.js
solution2.js
solution3.js
solution4.js
3.5 Secrets Pro...
React


EXPLORER
WEB DEVELOPMENT PR...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
node_modules
public
index.html
index1.js
index2.js
index3.js
index4.js
package-lock....
package.json
solution1.js
solution2.js
solution3.js
solution4.js
3.5 Secrets Pro...
React
OUTLINE
TIMELINE
index1.js
index.html
index2.js
Backend > 3.4 Middleware > index2.js > ...
1 import express from "express";
2 import morgan from "morgan";
3
4 const app = express();
5 const port = 3000;
6
7 app.use(morgan("tiny"));
8
9 app.get("/", (req, res) => {
10 res.send("Hello");
11 });
12
13 app.listen(port, () => {
14 console.log(`Listening on port ${port}`);
15 });
16
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
Listening on port 3000
^C
angelayu@Angelas-MacBook-Pro 3.4 Middleware % nodemon index2.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index2.js`
Listening on port 3000

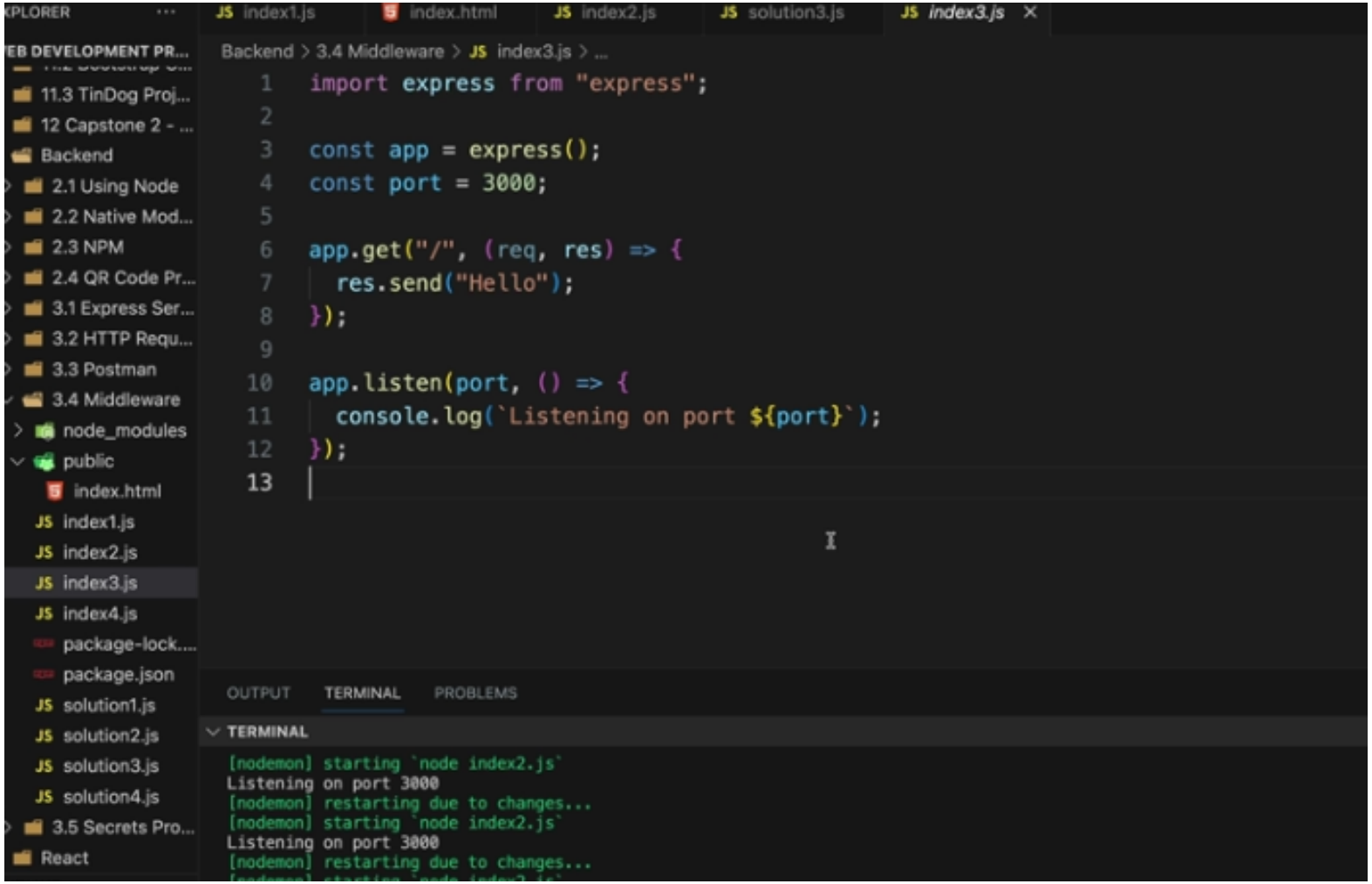

EXPLORER
WEB DEVELOPMENT PR...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
node_modules
public
index.html
index1.js
index2.js
index3.js
index4.js
package-lock....
package.json
solution1.js
solution2.js
solution3.js
solution4.js
3.5 Secrets Pro...
React
OUTLINE
TIMELINE
Home
Workspaces
API Network
Explore
Search Postman
Invite
Upgrade
Overview
GET localhost:3000
No Environment
localhost:3000
Save
GET
localhost:3000
Send
Params
Authorization
Headers (6)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (10)
Test Results
Status: 200 OK
Time: 27 ms
Size: 644 B
Pretty
Raw
Preview
Visualize
HTML
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Band Name Generator</title>
</head>
<body>
<h1>Band Name Generator</h1>
<form action="/submit" method="POST">
<label for="street">Street Name:</label>
<input type="text" id="street" name="street" required>
<label for="pet">Pet Name:</label>
angelayu@Angelas-MacBook-Pro 3.4 Middleware % nodemon index2.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index2.js`
Listening on port 3000
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
angelayu@Angelas-MacBook-Pro 3.4 Middleware % nodemon index2.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index2.js`
Listening on port 3000
GET / 200 5 - 5.374 ms

DIY Middleware




JS index.js
import express from "express";
const app = express();
const port = 3000;
app.use((req, res, next) => {
console.log("Request method: ", req.method);
next();
});
app.get("/", (req, res) => {
res.send("Hello");
});
app.listen(port, () => {
console.log(`Listening on port ${port}`);
});

Index3.js


index1.js
index.html
index2.js
solution3.js
index3.js
Backend > 3.4 Middleware > index3.js > ...
1 import express from "express";
2
3 const app = express();
4 const port = 3000;
5
6 app.get("/", (req, res) => {
7 res.send("Hello");
8 });
9
10 app.listen(port, () => {
11 console.log(`Listening on port ${port}`);
12 });
13
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
node_modules
public
index.html
index1.js
index2.js
index3.js
index4.js
package-lock....
package.json
solution1.js
solution2.js
solution3.js
solution4.js
3.5 Secrets Pro...
React
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
[nodemon] starting `node index2.js`
Listening on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index2.js`
Listening on port 3000
[nodemon] restarting due to changes...

JS index.js

```
import express from "express";
const app = express();
const port = 3000;

// Your custom function here

app.use(custom_function);

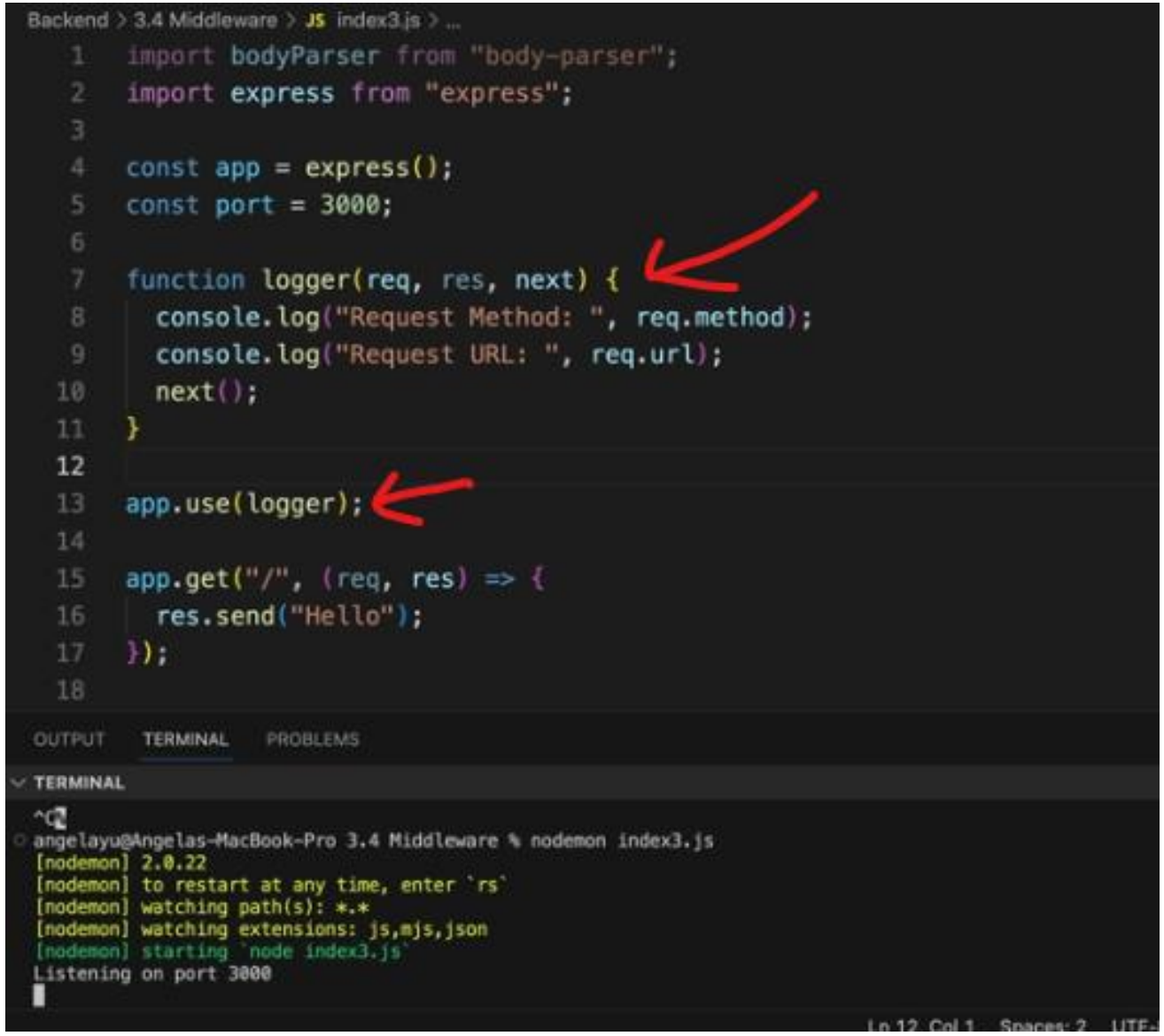
app.get("/", (req, res) => {
  res.send("Hello");
});

app.listen(port, () => {
  console.log(`Listening on port ${port}`);
});
```

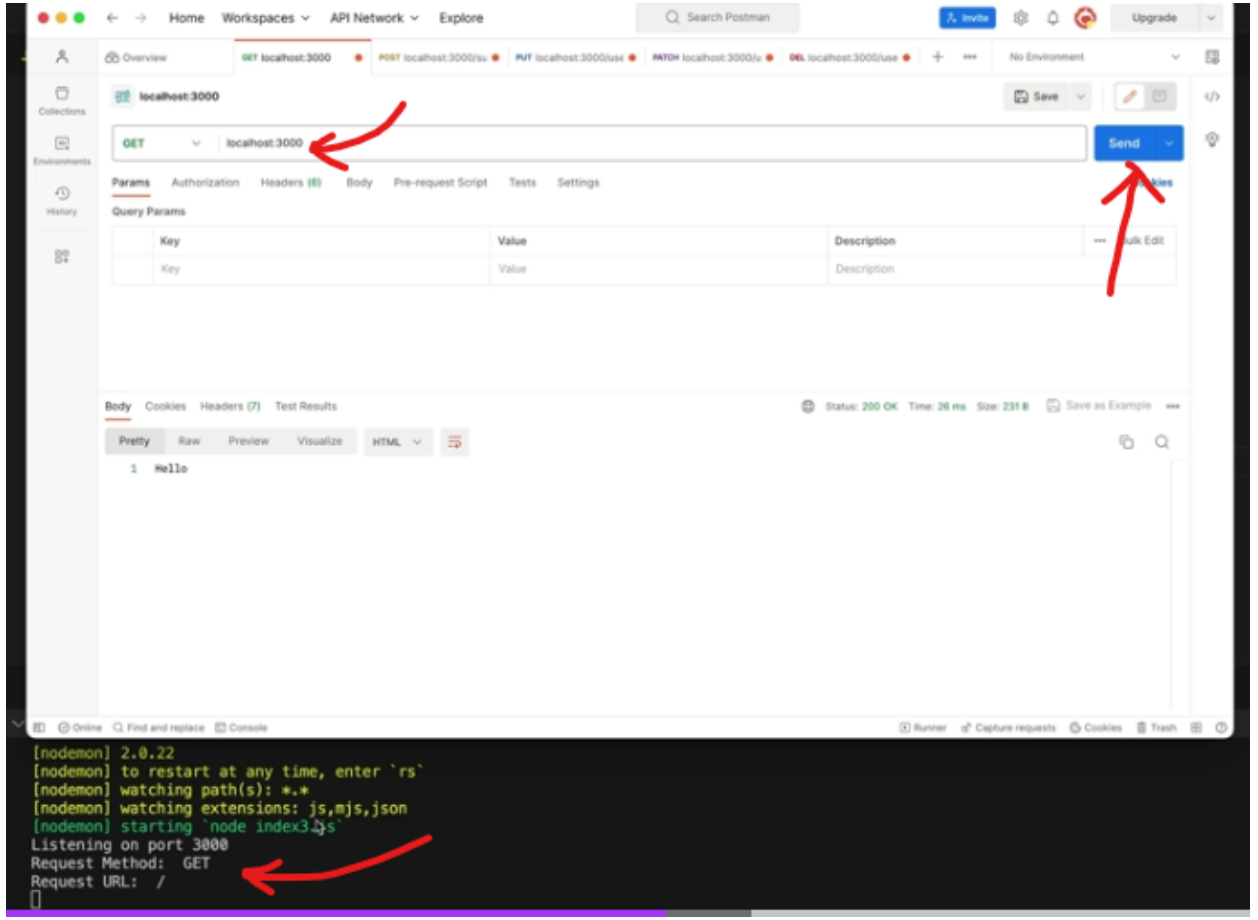
Exercise

Hello

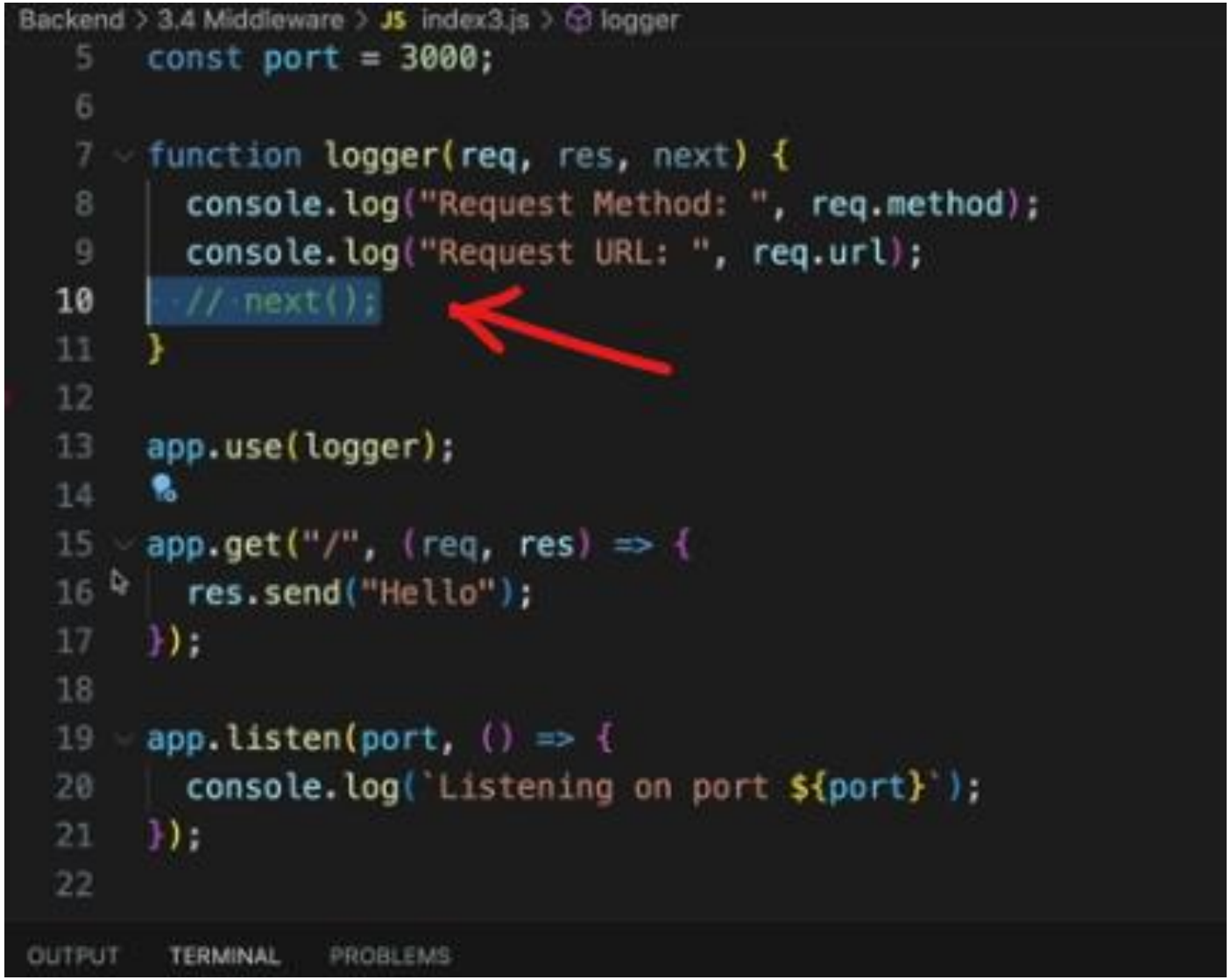
1. Run `index3.js` with nodemon.
2. Create your own function called `logger(req, res, next)`.
3. Inside your function `console.log()` the request method and the request URL.
4. Remember to call `next()`.
5. Mount your middleware function using the Express `.use()` method.
6. Test the logging on localhost and with Postman.


Backend > 3.4 Middleware > JS index3.js > ...
1 import bodyParser from "body-parser";
2 import express from "express";
3
4 const app = express();
5 const port = 3000;
6
7 function logger(req, res, next) {
8 console.log("Request Method: ", req.method);
9 console.log("Request URL: ", req.url);
10 next();
11 }
12
13 app.use(logger);
14
15 app.get("/", (req, res) => {
16 res.send("Hello");
17 });
18
OUTPUT TERMINAL PROBLEMS
TERMINAL
^C
angelayu@Angelas-MacBook-Pro 3.4 Middleware % nodemon index3.js
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index3.js`
Listening on port 3000

Testing request



Try to comment next();



Bandname Generator





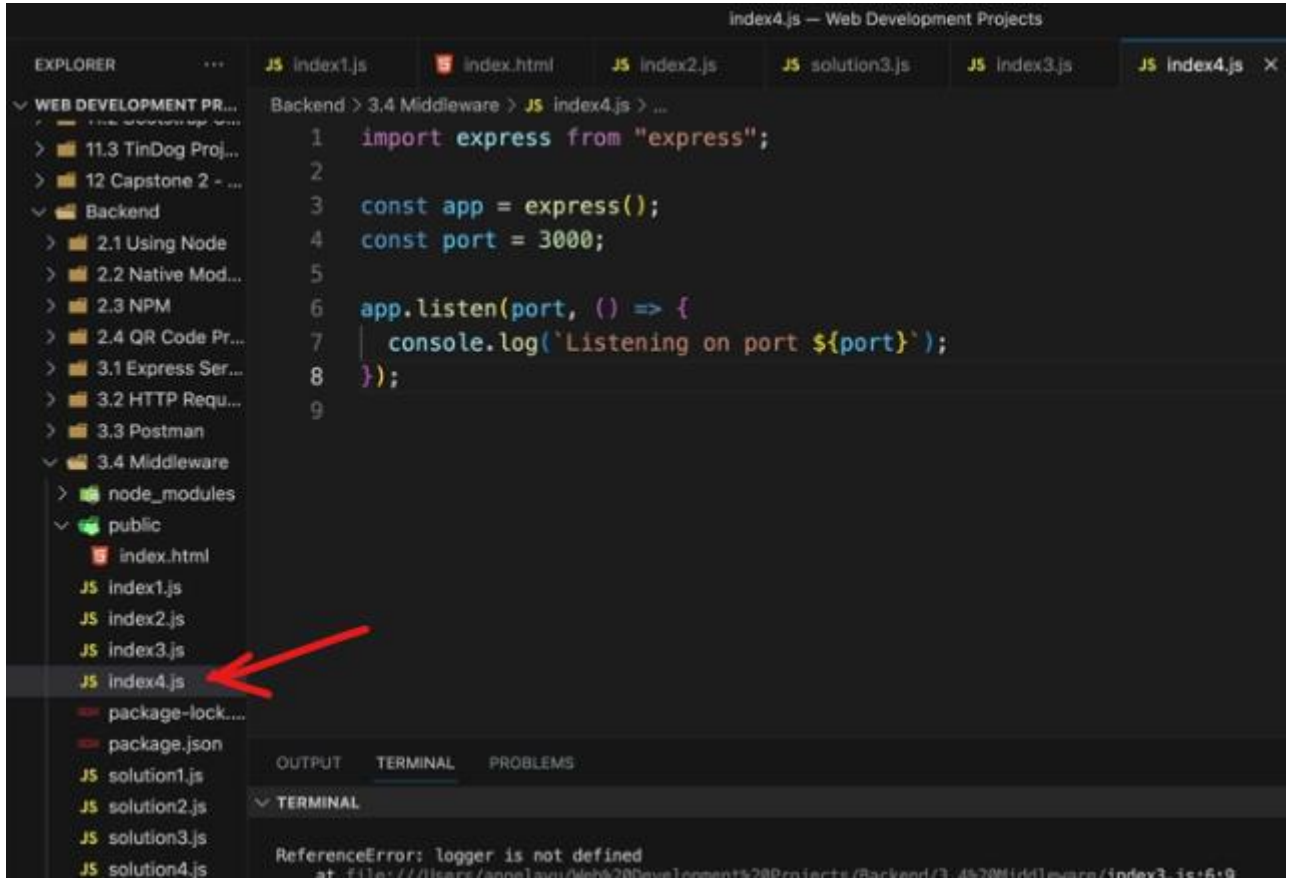


Try an Exercise

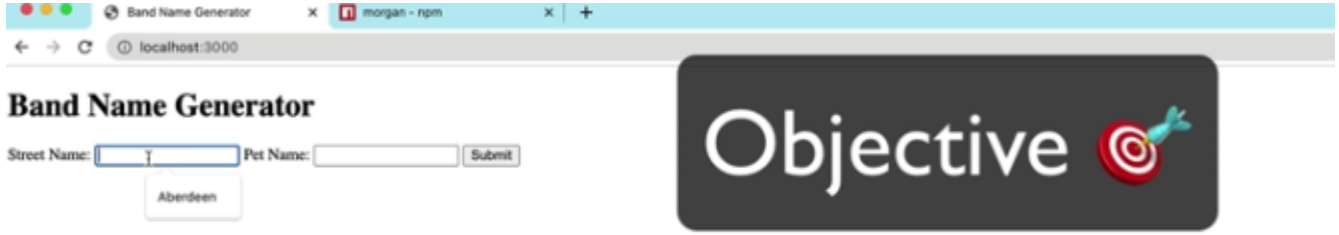


3.4 Middleware.zip

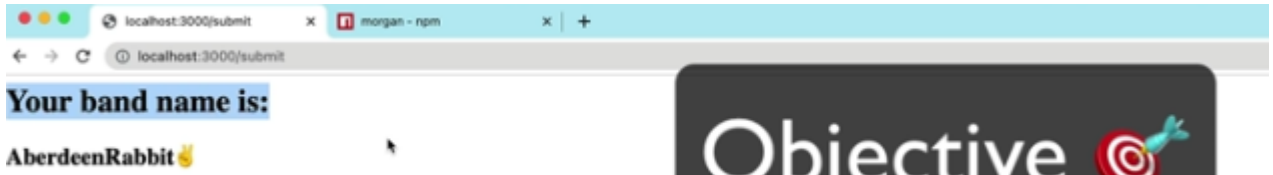
Index4.js


index4.js — Web Development Projects
EXPLORER
index1.js
index.html
index2.js
solution3.js
index3.js
index4.js
Backend > 3.4 Middleware > index4.js > ...
1 import express from "express";
2
3 const app = express();
4 const port = 3000;
5
6 app.listen(port, () => {
7 console.log(`Listening on port ${port}`);
8 });
9
WEB DEVELOPMENT PR...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
node_modules
public
index.html
index1.js
index2.js
index3.js
index4.js
package-lock....
package.json
solution1.js
solution2.js
solution3.js
solution4.js
OUTPUT
TERMINAL
PROBLEMS
TERMINAL
ReferenceError: logger is not defined

solution4


Band Name Generator
morgan - npm
localhost:3000
Band Name Generator
Street Name:
Pet Name:
Submit
Aberdeen

Objective


localhost:3000/submit
morgan - npm
localhost:3000/submit
Your band name is:
AberdeenRabbit


Objective

Backend > 3.4 Middleware > JS solution4.js > ...

```
1  import express from "express";
2  import bodyParser from "body-parser";
3  import { dirname } from "path";
4  import { fileURLToPath } from "url";
5
6  const __dirname = dirname(fileURLToPath(import.meta.url));
7
8  const app = express();
9  const port = 3000;
10 var bandName = "";
11
12 app.use(bodyParser.urlencoded({ extended: true }));
13
14 function bandNameGenerator(req, res, next) {
15   console.log(req.body);
16   bandName = req.body["street"] + req.body["pet"];
17   next();
18 }
19
20 app.use(bandNameGenerator);
21
22 app.get("/", (req, res) => {
23   res.sendFile(__dirname + "/public/index.html");
24 });
25
```

Backend > 3.4 Middleware > JS solution4.js > ...

```
8   const app = express();
9   const port = 3000;
10  var bandName = "";
11
12  app.use(bodyParser.urlencoded({ extended: true }));
13
14  function bandNameGenerator(req, res, next) {
15    console.log(req.body);
16    bandName = req.body["street"] + req.body["pet"];
17    next();
18  }
19
20  app.use(bandNameGenerator);
21
22  app.get("/", (req, res) => {
23    res.sendFile(__dirname + "/public/index.html");
24  });
25
26  app.post("/submit", (req, res) => {
27    res.send(`<h1>Your band name is:</h1><h2>${bandName}</h2>`);
28  });
29
30  app.listen(port, () => {
31    console.log(`Listening on port ${port}`);
32  });
```

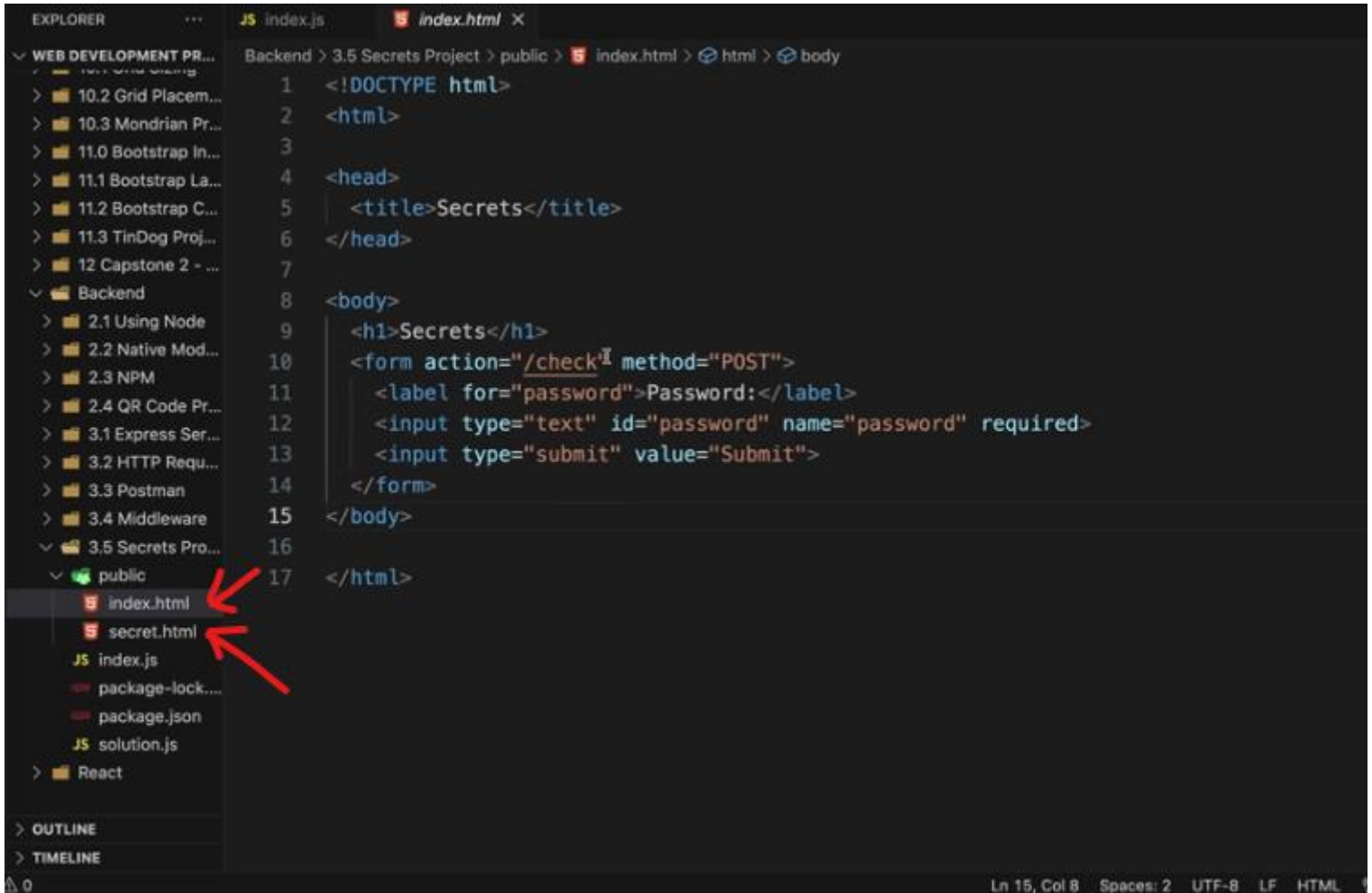
Secrets Project

A website full of secrets




ZIP
Secrets
Project.zip

Public/html


EXPLORER
index.js
index.html
Backend > 3.5 Secrets Project > public > index.html > html > body
WEB DEVELOPMENT PR...
10.2 Grid Placem...
10.3 Mondrian Pr...
11.0 Bootstrap In...
11.1 Bootstrap La...
11.2 Bootstrap C...
11.3 TinDog Proj...
12 Capstone 2 - ...
Backend
2.1 Using Node
2.2 Native Mod...
2.3 NPM
2.4 QR Code Pr...
3.1 Express Ser...
3.2 HTTP Requ...
3.3 Postman
3.4 Middleware
3.5 Secrets Pro...
public
index.html
secret.html
index.js
package-lock....
package.json
solution.js
React
OUTLINE
TIMELINE
1 <!DOCTYPE html>
2 <html>
3
4 <head>
5 <title>Secrets</title>
6 </head>
7
8 <body>
9 <h1>Secrets</h1>
10 <form action="/check" method="POST">
11 <label for="password">Password:</label>
12 <input type="text" id="password" name="password" required>
13 <input type="submit" value="Submit">
14 </form>
15 </body>
16
17 </html>
Ln 15, Col 8
Spaces: 2
UTF-8
LF
HTML

Step

- Run npm I
- Run nodemon solution.js
- Open the page

